Computer Architecture
A step-by-step beginner guide

Introduction and Chapters 1-5

113 small lessons that connect all six supplied HCMUT decks to clear explanations, original diagrams, tables, and carefully traced Assembly examples.

Start with the big picture. Learn registers and memory separately. Follow one instruction at a time. Then combine the steps into arrays, loops, procedures, and machine encodings.

112 practice questions appear at the end, with working space and no answer key in this document. Solutions are provided in a separate PDF.

Print specification	Setting
Paper	A4; print at actual size / 100%
Main text, tables, code	12 pt Candara
Lesson / section headings	19.2 / 21.6 pt
Subheadings	14.4 pt
Source notes / running text	10.2 / 9.2 pt
Color	Pure black and white

Prepared from the supplied slide decks. 14 October 2026.

Contents

Reading plan | Learn one small idea at a time 10

Introduction | Start with the big picture 12

I.1 What a computer does, and what architecture means . . . 12
I.2 Your course map and a manageable study routine . . . 13
I.3 How to read a worked example . . . 14

Chapter 1 | Technology and performance 15

1.1 From large machines to chips . . . 15
1.2 Different computer classes, familiar components . . . 16
1.3 The layers below a program . . . 17
1.4 Response time, throughput, and CPU time . . . 18
1.5 Comparing performance without reversing the ratio . . . 19
1.6 Clock period, frequency, and cycles . . . 20
1.7 Instruction count and CPI . . . 21
1.8 Weighted CPI: count the work in each class . . . 22
1.9 Improving only part of a program . . . 23
1.10 Power, voltage, and the power wall . . . 24
1.11 Multiple cores and the limits of parallel work . . . 25
1.12 Benchmarks and a final performance checklist . . . 26

Chapter 2 | Learn MIPS one action at a time 27

2.1 The ISA and the stored-program machine . . . 27
2.2 A small foundation: bits, binary, hex, and signed values . . . 28
2.3 Registers, immediates, and instruction syntax . . . 29
2.4 Arithmetic: destination first, then sources . . . 30
2.5 Memory addresses, offsets, and alignment . . . 31
2.6 Loads, stores, extension, and large constants . . . 32
2.7 Byte order and a complete memory trace . . . 33
2.8 Word arrays: load, calculate, store . . . 34
2.9 Variable indexes and nested array access . . . 35
2.10 Shifts: move bits within a fixed width . . . 36
2.11 Bitwise logic and masks . . . 37

2.12 Branches, labels, and a tiny loop trace . . . 38
2.13 If / else: make both paths explicit . . . 39
2.14 Less-than tests and signed versus unsigned comparisons . . . 40
2.15 While loops and for loops . . . 41
2.16 Pseudoinstructions and assembler conveniences . . . 43
2.17 Calls, returns, and the execution-model choice . . . 44
2.18 Calling conventions and the stack . . . 45
2.19 A leaf procedure, one phase at a time . . . 46
2.20 Non-leaf calls and recursive factorial . . . 47
2.21 Character arrays and string copying . . . 49
2.22 Machine code and the three formats . . . 50
2.23 R-format encoding and decoding . . . 51
2.24 I-format, negative immediates, and memory operands . . . 52
2.25 Branch displacements: begin at PC + 4 . . . 53
2.26 J-format and the shared address region . . . 54
2.27 Two complete Assembly-to-machine-code examples . . . 55
2.28 Far branches and a debugging routine . . . 56

Chapter 3 | Computer arithmetic **57**

3.1 Bits are patterns; an interpretation gives them meaning . . . 57
3.2 Binary addition and signed overflow . . . 58
3.3 Subtraction is addition of a negated operand . . . 59
3.4 What an overflow exception means . . . 60
3.5 Multiplication by partial products . . . 61
3.6 Shift-and-add multiplication hardware . . . 62
3.7 HI and LO store a wide multiplication result . . . 63
3.8 Division: build the quotient one bit at a time . . . 64
3.9 Restoring division and its hardware variants . . . 65
3.10 MIPS division, quotient, remainder, and checks . . . 66
3.11 Binary fractions and scientific notation . . . 67
3.12 IEEE-754 normal formats: sign, exponent, fraction . . . 68
3.13 Decode a floating-point word and understand its range . . . 69
3.14 Encode a decimal number, including rounding . . . 70
3.15 Reserved patterns, rounding modes, and extra bits . . . 71
3.16 Floating-point addition: align before adding . . . 72

3.17 Floating-point multiplication: combine signs and scales . . . 73
3.18 Floating-point registers and MIPS operations . . . 74
3.19 Trace an FP if / else and Fahrenheit conversion . . . 75
3.20 FP machine fields and an arithmetic problem checklist . . . 76

Chapter 4 | Microarchitecture: datapath, control, and pipelining **77**

4.1 One instruction, five kinds of work . . . 77
4.2 Read a datapath as boxes, routes, and switches . . . 78
4.3 IF and ID: fetch bits, extract fields, read values . . . 79
4.4 EX and MEM: calculate first, access memory second . . . 80
4.5 WB and the main control signals . . . 81
4.6 ALUOp and ALU control are two levels of decoding . . . 82
4.7 Worked functional-unit and control traces . . . 83
4.8 The next-PC paths: sequential, branch, and jump . . . 84
4.9 Why a single-cycle clock waits for the slowest instruction . . . 85
4.10 Pipelining overlaps work; it does not make a task disappear . . . 86
4.11 Count ideal pipeline cycles and compute time . . . 87
4.12 Read a pipeline diagram in two directions . . . 88
4.13 Pipeline registers preserve each instruction's information . . . 89
4.14 Controls travel with the instruction . . . 90
4.15 Three hazard types and structural conflicts . . . 91
4.16 RAW dependencies without forwarding . . . 92
4.17 Reschedule code only when meaning stays unchanged . . . 93
4.18 Insert stalls and count the slide's four-instruction example . . . 94
4.19 Forwarding supplies a result before WB . . . 95
4.20 Forwarding MUX values and the newest-result rule . . . 96
4.21 Why an immediate load-use pair still needs a stall . . . 97
4.22 Hazard detection: freeze the front, inject a bubble . . . 98
4.23 Branch hazards, prediction, and wrong-path flushing . . . 99
4.24 Static and dynamic prediction; a complete pipeline recipe . . . 100

Chapter 5 | Memory hierarchy: caches and virtual memory **101**

5.1 Why memory has several levels . . . 101
5.2 Temporal and spatial locality . . . 102
5.3 SRAM, DRAM, flash, and storage . . . 103

5.4 Blocks, hits, misses, and miss penalties . . . 104
5.5 Direct mapping: one possible line per memory block . . . 105
5.6 Tags, valid bits, and the slide's eight-access trace . . . 106
5.7 Split a byte address into tag, index, and offset . . . 107
5.8 Larger blocks and the address-1200 example . . . 108
5.9 Split caches and the FastMATH example . . . 109
5.10 Write-through and a write buffer . . . 110
5.11 Write-back, dirty bits, and write allocation . . . 111
5.12 Memory bandwidth: narrow, wide, and interleaved transfers . . . 112
5.13 Cache misses add to CPI . . . 113
5.14 Average memory access time is not automatically CPI . . . 114
5.15 Associativity changes where a block may go . . . 115
5.16 Trace direct, 2-way, and fully associative caches . . . 116
5.17 Replacement: invalid first, then LRU or an approximation . . . 117
5.18 L1 and L2: distinguish local and global miss rates . . . 118
5.19 Cache stalls interact with CPU speed and program dependencies . . . 119
5.20 Matrix access and blocking keep useful data nearby . . . 120
5.21 Virtual addresses, physical addresses, and pages . . . 121
5.22 Page tables and permission information . . . 122
5.23 TLB miss, page fault, and cache miss are different . . . 123
5.24 Page faults, replacement, dirty pages, and protection . . . 124
5.25 Work through the deck's TLB exercise one request at a time . . . 125
5.26 Four hierarchy questions and the three kinds of cache miss . . . 127

Source map and corrections **128**

Practice | Make an attempt before opening the key **133**

Q01. What is architecture? . . . 133
Q02. Find the computer class and component . . . 133
Q03. Connect the representations . . . 133
Q04. Interpret a technology claim . . . 134
Q05. Distinguish the kinds of time . . . 134
Q06. Relative performance and percentages . . . 134
Q07. Clock units . . . 134
Q08. A redesign changes the cycle count . . . 135

Q09. The full CPU-time equation . . . 135
Q10. Weighted CPI . . . 135
Q11. A partial improvement . . . 135
Q12. Dynamic power ratios . . . 136
Q13. Parallelism with a serial portion . . . 136
Q14. A benchmark summary . . . 136
Q15. Binary and hex . . . 136
Q16. The same byte, two interpretations . . . 137
Q17. A register trace . . . 137
Q18. Translate an expression . . . 137
Q19. Addresses and alignment . . . 137
Q20. Load widths and extension . . . 138
Q21. Partial stores, executed in sequence . . . 138
Q22. Big-endian and little-endian layouts . . . 138
Q23. A constant array index . . . 138
Q24. A variable array index . . . 139
Q25. A nested array index . . . 139
Q26. Valid shifts and a malformed instruction . . . 139
Q27. Bitwise operations . . . 139
Q28. Build a full-word constant . . . 140
Q29. Follow a loop and count its branches . . . 140
Q30. Translate a signed if / else . . . 140
Q31. Signed and unsigned comparison . . . 141
Q32. A sign-extended unsigned comparison . . . 141
Q33. A bounded search loop . . . 141
Q34. A for loop that accumulates a sum . . . 141
Q35. Expand conveniences . . . 142
Q36. Calls in two execution models . . . 142
Q37. Who preserves the value? . . . 142
Q38. A leaf procedure using a saved register . . . 142
Q39. A stack-frame address calculation . . . 143
Q40. Repair a non-leaf return . . . 143
Q41. Trace recursive factorial . . . 143
Q42. Copy a terminator . . . 144
Q43. Choose formats . . . 144

Q44. Encode R-format instructions . . . 144
Q45. Encode I-format instructions . . . 145
Q46. Encode backward and forward branches . . . 145
Q47. A jump at an address-region boundary . . . 145
Q48. Encode and trace a complete sequence . . . 146

Practice | Chapter 3 arithmetic **147**

Q49. Range and interpretation . . . 147
Q50. Carry and signed overflow . . . 147
Q51. Subtraction using two's complement . . . 147
Q52. Trapping versus non-trapping arithmetic . . . 148
Q53. Partial-product multiplication . . . 148
Q54. A shift-and-add trace . . . 148
Q55. Unsigned HI and LO . . . 149
Q56. Signed multiplication fit . . . 149
Q57. Signed quotient and remainder . . . 149
Q58. Binary long division . . . 150
Q59. Special-result registers and division safety . . . 150
Q60. Normalize a binary fraction . . . 150
Q61. Decode IEEE-754 . . . 151
Q62. Encode IEEE-754 . . . 151
Q63. Reserved floating-point patterns . . . 151
Q64. Ties-to-even rounding . . . 151
Q65. Floating-point addition steps . . . 152
Q66. Floating-point multiplication steps . . . 152
Q67. FP comparison and control flow . . . 152
Q68. FP instruction fields . . . 153

Practice | Chapter 4 microarchitecture **154**

Q69. Follow a load through five stages . . . 154
Q70. MUX selection and write enable . . . 154
Q71. Extract fields versus register contents . . . 154
Q72. Load path and control . . . 155
Q73. Store path and control . . . 155
Q74. Branch signal versus taken signal . . . 155

Q75. Two levels of ALU decoding . . . 156
Q76. Single-cycle critical path . . . 156
Q77. Finite pipeline speedup . . . 156
Q78. Which instruction controls cycle 5? . . . 157
Q79. Preserve the destination number . . . 157
Q80. RAW without forwarding . . . 157
Q81. Fill a load-use gap safely . . . 158
Q82. Forward from two different boundaries . . . 158
Q83. Choose the latest forwarded value . . . 158
Q84. An immediate load-use pair . . . 159
Q85. Repair incorrect stall control . . . 159
Q86. A field match that is not a source dependency . . . 159
Q87. Branch penalties and the endpoint . . . 160
Q88. A one-bit predictor . . . 160

Practice | Chapter 5 memory hierarchy **161**

Q89. Identify locality . . . 161
Q90. Set count and address widths . . . 161
Q91. Map a specific byte address . . . 161
Q92. Direct-mapped state trace . . . 162
Q93. Valid is part of the hit test . . . 162
Q94. Include metadata storage . . . 162
Q95. Placement choices . . . 163
Q96. LRU updates on hits . . . 163
Q97. Dirty write-back behavior . . . 163
Q98. Allocation is separate from write propagation . . . 164
Q99. Block-transfer bandwidth . . . 164
Q100. Miss contributions to CPI . . . 164
Q101. AMAT and miss-time conventions . . . 165
Q102. Local and global L2 miss rates . . . 165
Q103. More GHz does not halve a fixed memory wait . . . 165
Q104. Matrix layout and tile footprint . . . 166
Q105. Virtual-to-physical translation . . . 166
Q106. Change the page size . . . 166
Q107. A flat page-table size . . . 167

Q108. Three different misses 167
Q109. A small TLB LRU trace 167
Q110. A dirty page victim 168
Q111. Classify the three Cs 168
Q112. Combine translation and a data-cache trace 168

Reading plan | Learn one small idea at a time

You do not need to understand a full processor diagram before learning your first instruction. This guide builds the pieces in order and revisits them when they become useful. Every numbered lesson begins on a new page; a longer lesson can continue onto the following page. Use the contents and PDF bookmarks to return to a particular step.

A manageable way to study

1. Read one lesson's first explanation and name the main idea in your own words.

2. Follow its diagram or table before reading the complete worked trace.

3. Cover the next row of the trace, predict it, then compare your result.

4. Write down why each register, memory location, field, or cycle changes.

5. Read the common mistakes and final reminder. Return to an earlier prerequisite if the notation is unfamiliar.

6. Attempt the related numbered practice questions at the end. Keep the separate answer key closed until you have tried.

Part	What it helps you understand	Practice questions
Introduction: I.1-I.3	What architecture studies; how to approach the course	Q01-Q03
Chapter 1: 1.1-1.12	Technology, performance, clock, CPI, power	Q04-Q14, with foundation review Q01-Q03
Chapter 2: 2.1-2.28	Registers, memory, Assembly, calls, machine encoding	Q15-Q48
Chapter 3: 3.1-3.20	Integer algorithms, HI/LO, IEEE-754, FP code	Q49-Q68
Chapter 4: 4.1-4.24	Datapath routes, controls, pipeline timing and hazards	Q69-Q88
Chapter 5: 5.1-5.26	Caches, miss costs, page tables, TLBs, translation	Q89-Q112

Conventions used throughout

Use 32-bit ordinary MIPS registers, byte-addressed memory, and four-byte words. Memory layout examples follow the slides' big-endian model unless another order is stated. A MIPS implementation or simulator can choose a different byte order; use the question's convention.

Assembly teaching fragments use no architectural delay slots unless explicitly discussing classic delayed MIPS. Thus jal links to the following instruction in these fragments; the classic reference card instead uses PC+8 and a delay slot. Branch and J-format reconstruction use the instruction address plus 4 in both address calculations.

Pipeline examples state their forwarding and branch-resolution model. The default five-stage register-file convention permits same-cycle WB write and ID read. For a full-drain count, stores still occupy a final stage even though their last useful memory update occurred earlier. Do not use a

no-forwarding timing answer for a forwarding machine.

For cache and virtual-memory calculations, distinguish bytes, words, blocks, pages, sets, and frames. KiB means 1024 bytes. A miss penalty is additional delay after lookup unless a question gives a total miss time. Rate denominators and initial state are part of the problem, not optional details.

Notation that will recur

Notation	Meaning
0x...	Hexadecimal; one digit represents four bits
2^n	Two raised to power n
IC / CPI	Executed instruction count / cycles per instruction
P or PC	Instruction byte address; P avoids confusion with an already incremented PC
R[register] / memory[address]	Stored value in a register / at a location
X in a control table	Don't-care choice; the corresponding result is unused
HI:LO	Concatenated upper and lower result registers
VPN / PA / PTE / TLB	Virtual page number / physical address / page-table entry / translation cache

The lessons paraphrase the supplied decks and add original teaching examples. Source-page notes use PDF page numbers. Title/divider/end slides are mapped with their neighboring topic; historical charts are treated as dated illustrations. The final source map records both coverage and corrections.

Introduction | Start with the big picture

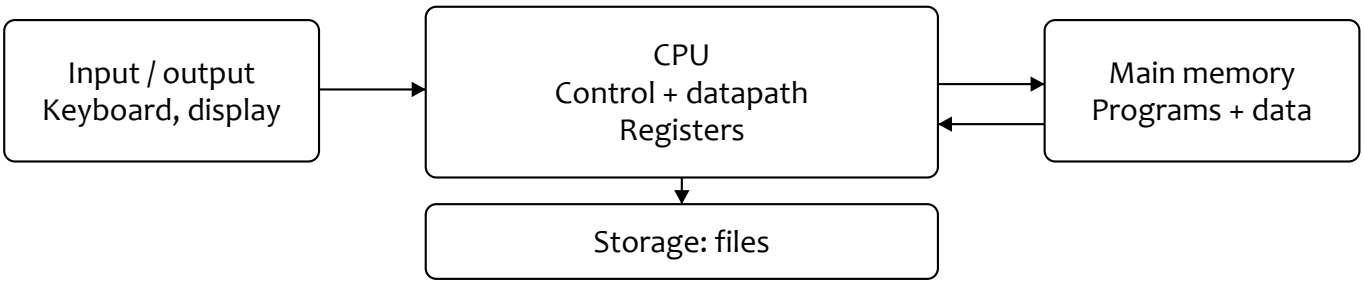
I.1 What a computer does, and what architecture means

Source: Introduction.pdf pp. 3-4.

Goal: Describe a computer without needing to know any Assembly syntax yet.

A computer stores information and follows instructions that operate on it. The information might represent a number, photo, sound, or memory location. The processor works with bit patterns according to its instructions; it need not understand their human meaning.

Think about a calculator program. You enter 7 and 5. Input hardware brings those values into the system. Memory holds the program and its data. The processor adds the values. Output hardware shows 12. A more complicated application repeats many such small operations.



Part	Beginner meaning	In the calculator example
Processor / CPU	The part that follows instructions	Performs the addition
Datapath	The circuits that move and operate on values	Carries operands to an adder
Control	The circuits that select the required actions	Selects addition and the destination
Memory	Storage the processor can access	Holds instructions and working data
Input / output	The system's connections to the outside world	Keyboard input and display output

Computer architecture concerns the choices that make these parts work together while meeting goals such as correct behavior, speed, cost, and energy use. A design for a phone and a design for a scientific computer can prioritize different goals.

An **instruction set architecture (ISA)** defines programmer-visible instructions, registers, data types, and addressing rules. **Organization** is their implementation. Two processors can implement the same ISA with different circuits and execution times.

Walk through the idea: Identify inputs, operation, and destination: 7 and 5; addition; output 12. Use the same habit for add $t0, $t1, $t2 later.

Remember: A computer program is a sequence of small, precisely defined actions. You can learn those actions one at a time.

I.2 Your course map and a manageable study routine

Source: Introduction.pdf pp. 1-2 and 5-9.

Goal: Know what these documents cover and how to study without trying to absorb everything at once.

The introduction identifies Computer Architecture at HCMUT and instructor Cuong Pham-Quoc. This guide covers all six requested decks, from system foundations and Assembly to arithmetic, processor implementation, and memory hierarchy. Some broader course areas extend beyond those decks.

Course area shown in the introduction	Where it fits
Performance evaluation	Chapter 1 in this guide
Instruction set architecture	Chapter 2 in this guide
Computer arithmetic	Chapter 3 in this guide
Datapath and control	Chapter 4 in this guide
Memory and I/O systems	Memory hierarchy in Chapter 5; further I/O material lies beyond these decks
Multicores, multiprocessors, clusters	Introduced in Chapter 1; developed later

The outcomes ask you to understand the structure of a computer and to write or improve small pieces of machine-level code. You do not need to memorize every instruction before writing your first three-line program.

Administrative information recorded in the deck

Item	What the introduction slide records
Assignments	30%
Labs	10%, mandatory
Midterm	20%; multiple-choice / written; one A4 sheet of notes
Final	40%; multiple-choice / written; two A4 sheets of notes
Learning resources	BKEL slides / handouts, instructor resources, textbooks

These are the rules recorded in this supplied deck, rather than a verification of the rules for your current class. Check your current course announcement for any updates. The last introduction slide uses pictures for classroom expectations; its icons suggest avoiding disruptive conversation, phone use, and food or drink, and bringing handouts. Because it has no explanatory text, treat those descriptions as an interpretation of its pictures.

A repeatable learning routine

Use the reading plan near the beginning: learn one idea, predict the next trace row, compare it, then attempt the related practice question before opening the key.

Remember: Slow, accurate traces build the understanding needed for faster work later.

I.3 How to read a worked example

Source: Introduction.pdf p. 6; learning method added for this guide.

Goal: Use a small table to make an unfamiliar program understandable.

An instruction changes part of the machine's **state**. State means the values currently stored in registers and memory, together with the address of the instruction to execute. You do not need to track the entire machine. Track only the values that the example uses.

Suppose three registers are small labeled boxes. Before the instruction, $t1 holds 7, $t2 holds 5, and $t0 holds 99. The instruction below says to add the two source values and place the result in the destination.

```
add $t0, $t1, $t2
```

Point in time	$t0	$t1	$t2
Before	99	7	5
After	12	7	5

Read it in this order: **operation**, **destination**, **sources**. The destination is first in this arithmetic syntax. Reading $t0 first does not mean that its old value is added automatically. The source list determines what is read.

When you get stuck, answer these four questions on paper: Which instruction is next? Which existing values does it read? What calculation or decision does it make? Which one or two things change? Leave every other entry unchanged.

For performance mathematics, use a similar habit: list the given numbers with their units, write the formula, substitute the values, and check the unit of the result. A number without its unit can be misleading.

Conventions used throughout this guide

All code uses a 32-bit MIPS teaching model. A word is 4 bytes. Unless stated otherwise, memory examples use **big-endian** byte order to match the Chapter 2 slides. Branches and calls in the worked traces use the slides' **no-delay-slot model**: a taken branch immediately changes the next instruction; jal saves the following instruction's address, PC + 4.

Classic delayed MIPS instead executes a delay-slot instruction and saves PC + 8 for jal, as your reference card records. Instruction encodings and the branch target formula still use the branch instruction's address plus 4. Lesson 2.17 explains the difference. Do not mix the two execution models in a single trace.

Arithmetic examples assume that their signed results fit in 32 bits unless overflow is the topic. Register names are written in lower case. Labels are names for instruction addresses; they are not stored as words in a register.

Chapter 1 | Technology and performance

1.1 From large machines to chips

Source: Chapter1.pdf pp. 1-8, 10 and 14.

Goal: Understand what the history and technology graphs are trying to show.

A transistor can act as a tiny electronic switch. Many connected switches can implement logic, arithmetic, and storage. An integrated circuit places many electronic components together on a chip. Increasing the number of useful components that can fit on a chip has enabled systems that would have been impractical with earlier technology.

Stage in the slide overview	Main idea	Why the change mattered
Vacuum tubes	Large electronic switching components	Electronic computation, but high size and power
Transistors	Smaller solid-state switching components	Better size, energy use, and reliability
Integrated circuits	Many components manufactured together	More functionality at lower cost per function
Highly integrated chips and personal computers	Very large collections of components	Computing became widely accessible

Generation dates are broad teaching categories because transitions overlap. Better components can change capacity, cost, and performance; all machines do not change generation on one exact day.

The ENIAC slides show early computing's scale: a room-sized system with thousands of tubes and substantial power. "First computer" depends on the definition. ENIAC was an early programmable, general-purpose electronic digital computer; it was not the first computing device of any kind [W1].

Moore's Law, in plain language

The slide describes transistor counts doubling roughly every 18-24 months. This is a historical observation and prediction about integration, rather than a law guaranteeing that every program doubles its speed. Moore's 1965 projection used a one-year interval; his 1975 revision used about two years [W2].

Illustrative doubling	Starting count 100	Meaning
Start	100	Initial capacity
One doubling	200	Twice as many
Two doublings	400	Four times the original
Three doublings	800	Eight times the original

More transistors can support larger caches or more cores; program speed still depends on their use. The product table is a Q1/2023 snapshot. Shipment and DRAM graphs are historical illustrations.

Remember: Technology creates opportunities. Architecture decides how to use them.

1.2 Different computer classes, familiar components

Source: Chapter1.pdf pp. 9-11.

Goal: Recognize different design goals and the parts shared by different computers.

Class	Typical job	A major design concern
Personal computer	Run varied applications for a person	Useful performance at an affordable cost
Server	Provide services over a network	Capacity, availability, reliability
Supercomputer	Large scientific or engineering calculations	Very high computational capability
Embedded computer	Control a component inside another product	Power, cost, deadlines, physical size

A washing machine controller and a large server are very different products, but both contain processing, storage, and connections to their surroundings. A phone contains several kinds of processors; the categories are useful descriptions, not walls separating every real product.

Distinguish the storage locations

Location	What to imagine	Role
Register	A small working box inside the CPU	Immediately used operands and results
Cache	A nearby collection of frequently needed information	Reduce the cost of many memory accesses
Main memory / RAM	A larger workspace for running programs	Instructions and active data
Storage device	Longer-term storage	Files and programs retained across shutdown

These are roles, not interchangeable names. A register number such as 8 identifies a CPU register. A memory address such as 8 identifies a byte location in memory. They belong to different spaces, even when the numbers happen to match.

Worked thought experiment: You open a photo editor. The program and photo are brought from storage into memory. The CPU repeatedly moves needed values into registers and computes results, often helped by caches. The display receives an image through output hardware. Saving the photo writes it back to storage.

The datapath performs movements and calculations. The controller selects the movements and operations. Later datapath chapters show the circuits in detail; here you only need their roles.

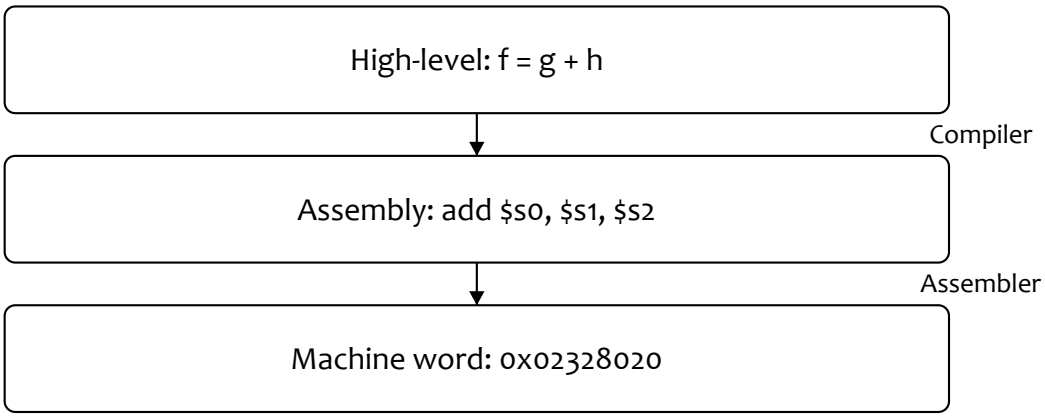
Remember: Find the job first. Then ask which component performs it.

1.3 The layers below a program

Source: Chapter1.pdf pp. 12-13 and 34; Chapter2.pdf pp. 2-4.

Goal: Connect a familiar high-level statement to the machine instructions you will learn.

High-level languages let you express a problem using variables, expressions, functions, and loops. Assembly uses names for individual machine instructions and registers. Machine code represents the instructions as bits. The ISA connects software's expectations to hardware's behavior.



For example, a high-level expression might be f = g + h. If the variables have already been assigned to registers, one possible MIPS instruction is add $s0, $s1, $s2. The names are meaningful to the programmer; the machine instruction contains register numbers and operation fields.

1. A compiler decides how to translate high-level work into lower-level operations.

2. An assembler encodes Assembly instructions and resolves labels. Some compiler paths generate machine code directly, so a visible Assembly file is not compulsory.

3. A linker combines separately produced pieces and resolves references between them.

4. A loader and operating system prepare the program to run. The CPU executes its machine instructions.

The operating system also manages memory, storage, I/O, and the sharing of processor time between tasks. It does not translate every source-code line while your already compiled program runs.

Why abstraction helps

You can learn how to drive without knowing the chemistry of fuel. In the same way, a programmer can use a function without knowing its machine instructions. Assembly study deliberately looks below that familiar layer, but you can still separate the levels while learning.

Question	Suitable level
What does the application need to calculate?	Algorithm / high-level program
Which registers and instructions implement it?	ISA / Assembly
How are the instruction fields represented?	Machine code
Which circuits execute those fields?	Processor organization

Remember: The same computation can have several representations. Representation changes; the intended result stays consistent.

1.4 Response time, throughput, and CPU time

Source: Chapter1.pdf pp. 16-19.

Goal: Choose the measurement that answers the question being asked.

The airplane slide asks which aircraft has the best performance. Speed, passenger capacity, and range can give different answers. Computers have the same issue: "best" requires a workload and a measurement.

Response time, also called latency, is the time needed for one task. **Throughput** is the amount of work completed per unit time. Imagine a shop: how long one customer's order takes is response time; how many orders the shop completes per hour is throughput.

Change	Possible effect on one task	Possible effect on total work
Faster processor	Often lowers its CPU processing time	Often increases throughput
More processors	Helps if the task can use them, or if waiting is reduced	Often helps with many independent tasks
Faster storage	Helps tasks that wait for storage	Helps when storage was a bottleneck

There is no guarantee that adding processors shortens an entirely sequential task. There is also no guarantee that a faster CPU fixes a long network wait.

Two clocks you might use to measure a program

Elapsed time is the wall-clock time from start to finish, including waiting, I/O, and other overhead. **CPU time** measures processor time spent on the job. User CPU time belongs to the program's execution; system CPU time is operating-system work performed on its behalf.

Worked timeline: A program computes for 2 seconds, waits for disk input for 3 seconds, and computes for another 1 second. With no other work in this simplified example, elapsed time is 6 seconds; CPU time is 3 seconds. Halving only the computation makes elapsed time 1 + 3 + 0.5 = 4.5 seconds.

The simplified timeline treats the intervals as non-overlapping. On a machine running multiple threads, CPU times across cores can add up to more than elapsed time. Always identify what a measurement includes.

Remember: Count seconds for one task; count tasks per second for throughput.

1.5 Comparing performance without reversing the ratio

Source: Chapter1.pdf p. 18.

Goal: Calculate how much faster one system is for the same completed work.

For a fixed workload, performance is proportional to 1 / execution time. A shorter time means greater performance. To compare X against Y, place Y's time in the numerator and X's time in the denominator.

> Performance X / Performance Y = Time Y / Time X
> Speedup from old to new = Old time / New time

The slide's example runs in 10 seconds on A and 15 seconds on B. A's relative performance is 15 / 10 = 1.5. Say "A has 1.5 times B's performance for this workload." A takes two-thirds as long as B.

Description	Calculation	Result
Performance increase from B to A	(1.5 - 1) x 100%	50%
Execution-time reduction from B to A	(15 - 10) / 15 x 100%	33.33%

These percentages differ because one compares rates and the other compares times. Specify which one you mean. A speedup of 1 means no change; greater than 1 means improvement; less than 1 means slowdown.

A safe four-step calculation

1. Check that both times refer to the same completed workload and compatible measurement.

2. Mark the system you want to call faster.

3. Divide the slower system's time by the faster system's time.

4. Check that the result is greater than 1.

Do not compare one system's CPU time with another system's elapsed time as though the measurements were identical. Do not compare a different input size without explaining the difference.

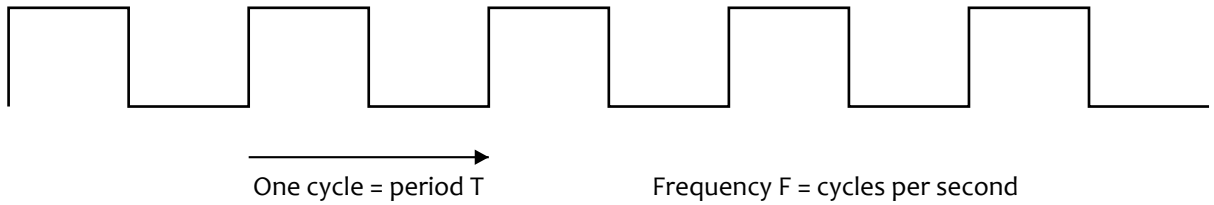
Remember: More performance corresponds to less time. That is why the time ratio reverses.

1.6 Clock period, frequency, and cycles

Source: Chapter1.pdf pp. 20-22.

Goal: Convert between a clock's speed and the time required by a cycle count.

A clock provides regularly spaced timing events. Its **period** is the duration of one cycle. Its **frequency** is the number of cycles per second. The clock does not mean that every instruction finishes in one cycle.



Period T = 1 / Frequency F
CPU time = Clock cycles x T = Clock cycles / F

Unit	Meaning	Useful conversion
GHz	Billion cycles per second	2 GHz = 2 x 10^9 Hz
MHz	Million cycles per second	500 MHz = 5 x 10^8 Hz
ns	Billionth of a second	1 ns = 10^-9 s
ps	Trillionth of a second	250 ps = 0.25 ns
microsecond	Millionth of a second	1 microsecond = 10^-6 s

At 2 GHz, a cycle lasts 1 / (2 x 10^9) seconds = 0.5 ns. If a job requires 20 cycles, it takes 20 x 0.5 ns = 10 ns in this simplified timing model.

Work through the slide's redesign example

Computer A runs at 2 GHz and takes 10 seconds. First calculate its cycle count: 10 s x 2 x 10^9 cycles/s = 20 x 10^9 cycles. Computer B's design requires 1.2 times as many cycles, so B needs 24 x 10^9 cycles. To finish in 6 seconds, B needs (24 x 10^9) / 6 = 4 x 10^9 Hz = **4 GHz**.

The extra cycles matter. Simply changing 2 GHz by the ratio 10/6 would ignore the design's 20% increase in cycle count.

Remember: Frequency tells you the price of one cycle in seconds. The cycle count tells you how many cycles you must pay for.

1.7 Instruction count and CPI

Source: Chapter1.pdf pp. 23-24 and 28.

Goal: Use all three factors that determine CPU time.

Instruction count (IC) is the number of instructions actually executed for the workload. It is a dynamic count. A loop containing four instruction lines can execute those lines hundreds of times. **Cycles per instruction (CPI)** is the average number of clock cycles per executed instruction.

Clock cycles = IC x average CPI CPU time = IC x CPI x cycle period = IC x CPI / frequency

Factor	Ask this question	Influences
IC	How many instructions are executed?	Algorithm, compiler, ISA, input
CPI	How many cycles does an instruction cost on average?	CPU implementation, instruction mix, stalls
Period	How many seconds does a cycle cost?	Clock frequency / hardware design

Worked example: A workload executes 2 million instructions with CPI 1.5 on a 3 GHz CPU. Cycles = 2,000,000 x 1.5 = 3,000,000. Time = 3,000,000 / 3,000,000,000 = 0.001 s = 1 ms.

The slide comparison

With the same dynamic IC, A has CPI 2 and period 250 ps; B has CPI 1.2 and period 500 ps. A pays 500 ps per instruction on average. B pays 600 ps. Therefore A is faster by 600/500 = 1.2 times, even though B has lower CPI.

The phrase "same ISA and compiler" is used in the slide to justify the shared count for its example. In real comparisons, confirm the same program, input, and executed path too. The same ISA by itself does not force all programs to have the same count.

Check units explicitly: instructions x cycles/instruction x seconds/cycle leaves seconds. This cancellation is a useful way to remember the equation.

Remember: A low CPI or high GHz number alone does not determine the winner.

1.8 Weighted CPI: count the work in each class

Source: Chapter1.pdf pp. 25-26.

Goal: Calculate an average that respects how often each instruction type occurs.

If different instruction classes have different costs, count their cycles separately. Add the class cycle counts, then divide by the total instruction count.

> Total cycles = sum of (Class instruction count x Class CPI)
> Average CPI = Total cycles / Total instructions
> Equivalent: Average CPI = sum of (Class fraction x Class CPI)

For classes A, B, C with CPIs 1, 2, 3, respectively, the slide compares these implementations:

Class	CPI	Implementation 1 count	Its cycles	Implementation 2 count	Its cycles
A	1	2	2	4	4
B	2	1	2	1	2
C	3	2	6	1	3
Total		5	10	6	9

Implementation 1 has average CPI 10/5 = 2. Implementation 2 has average CPI 9/6 = 1.5. At the same clock frequency, implementation 2 is faster because it uses 9 cycles instead of 10. It executes more instructions, but its instructions are cheaper overall.

Do not average 1, 2, and 3 without weights unless each class occurs equally often. Do not conclude that 5 instructions must be faster than 6 instructions.

A useful checking habit

First make sure the fractions add to 1, or the percentages add to 100%. Then calculate every row's contribution. The weighted average should lie between the smallest and largest class CPIs. If it falls outside that range, recheck the calculation.

Remember: The average cost is shaped by both instruction cost and instruction frequency.

1.9 Improving only part of a program

Source: Chapter1.pdf p. 27. The general partial-improvement formula is supplemental explanation.

Goal: Recalculate total time after a change instead of applying a local speedup to the entire job.

The slide's exercise has 1,000 instructions on a 2 GHz CPU. Load/store instructions are 30% at CPI 2.5; jumps are 10% at CPI 1; branches are 20% at CPI 1.5. Arithmetic instructions are the remaining 40%, at CPI 2.

Class	Instructions	CPI	Original cycles	Cycles after load/store cost is halved
Load/store	300	2.5	750	375
Jump	100	1	100	100
Branch	200	1.5	300	300
Arithmetic	400	2	800	800
Total	1,000		1,950	1,575

1. Original average CPI = 1,950/1,000 = 1.95.

2. Original CPU time = 1,950/(2 x 10^9) = 0.975 microseconds.

3. Improved time = 1,575/(2 x 10^9) = 0.7875 microseconds.

4. Overall speedup = 1,950/1,575 = 26/21, about **1.238 times**.

Only the load/store cost was halved. The total time was not halved. These calculations assume the count and frequency stay fixed and the change does not introduce other costs.

Connect this to Amdahl's Law

If fraction f of the original execution time is accelerated by factor s, the normalized new time is (1 - f) + f/s. Overall speedup is 1 / ((1 - f) + f/s).

Here the load/store time fraction is 750/1,950 = 5/13, about 38.46%. It is **not** the 30% instruction fraction, because those instructions have different CPI. Using f = 5/13 and s = 2 gives the same speedup, 26/21.

Imagine speeding up one part of a route while the rest stays the same. Even making the improved part instantaneous cannot remove the unchanged part's time.

Remember: Find the fraction of time affected, not merely the fraction of instructions.

1.10 Power, voltage, and the power wall

Source: Chapter1.pdf pp. 29-30.

Goal: Use the slide's power model and understand why clock increases face limits.

The slides use a simplified switching-power model for CMOS circuits:

> Dynamic power is proportional to C x V^2 x F
> New / old power = (Cnew / Cold) x (Vnew / Vold)^2 x (Fnew / Fold)

C represents effective capacitive load, V supply voltage, and F clock frequency. More generally the switching activity matters too. For the slide's ratio exercise, assume it stays constant. This model describes dynamic switching power; it does not account for every source of total chip power, such as leakage.

If voltage becomes 80% of its original value with other factors fixed, the model gives 0.8^2 = 0.64 of the original dynamic power. That is a 36% reduction, rather than a 20% reduction.

Work through the slide's example

The new CPU has 85% of the old capacitive load. Both voltage and frequency are reduced by 15%, so each becomes 0.85 of its original value. The ratio is 0.85 x 0.85^2 x 0.85 = 0.85^4 = 0.52200625. The new dynamic power is about 52.2% of the original, a reduction of about 47.8%.

Phrase in a question	Ratio to substitute
Reduced by 15%	0.85
Reduced to 15%	0.15
Increased by 15%	1.15

The **power wall** is the difficulty of continuing to increase performance using ever higher clocks and power. Heat removal and reliable operation constrain design. Voltage cannot be lowered arbitrarily while preserving the same behavior and timing.

Reducing frequency alone usually increases the CPU time for the same cycle count. A lower-power design and a faster design are not automatically the same design. This motivates more efficient implementations and parallel work.

Remember: Square the voltage ratio. Distinguish power consumed per second from energy used for an entire task.

1.11 Multiple cores and the limits of parallel work

Source: Chapter1.pdf pp. 31 and 34; simple numerical examples added for clarity.

Goal: Explain why more processors can help and why speedup is not guaranteed to equal core count.

A core is a processor execution engine. A multicore chip contains several cores. A multiprocessor system has multiple processing units working in one system. A program benefits when useful work can be divided among them.

Imagine four independent files, each requiring 2 seconds to process. One core doing them consecutively needs 8 seconds. Four cores could process one file each in about 2 seconds, assuming no extra overhead or shared bottleneck. This raises throughput.

Now imagine a single calculation in which each step needs the previous step's result. The steps cannot all run simultaneously. More cores do not automatically remove that dependency.

Source of limited speedup	Plain-language explanation
Serial work	Some steps must happen in order
Uneven work	One worker may finish later than the others
Communication / synchronization	Workers must exchange values or wait
Shared resources	Cores may compete for memory, storage, or another resource

As a supplemental calculation, suppose half of a task is completely serial and half divides perfectly across four cores. Normalized time is 0.5 + 0.5/4 = 0.625. Speedup is 1/0.625 = 1.6, even before communication overhead.

A programmer may need to change the algorithm to expose parallelism. A compiler cannot always turn a sequential dependency into independent work. The performance equation still matters: parallel execution changes the way the work and costs are distributed; it does not make all costs disappear.

Remember: Ask which parts can run independently before multiplying a speed by the number of cores.

1.12 Benchmarks and a final performance checklist

Source: Chapter1.pdf pp. 32-34.

Goal: Read a benchmark result as a measurement of specified work.

A benchmark is a program or collection of programs used to measure performance. A useful benchmark resembles the work you care about. A result for one workload does not prove that the same machine wins for every workload.

The slides discuss SPEC CPU2006 and show historical Core i7-920 results. CPU2006 is a historical, now-retired suite [W3]. Its performance ratios compare a reference time with the measured system's time, and its summary combines individual ratios using a geometric mean [W4].

> Ratio for a benchmark = Reference time / Measured time
> Geometric mean for two ratios = square root of (Ratio 1 x Ratio 2)
> For n ratios: take the nth root of their product

Illustrative benchmark	Reference time	Measured time	Ratio
Workload A	100 s	50 s	2
Workload B	400 s	50 s	8

The two-ratio geometric mean is sqrt(2 x 8) = 4. The arithmetic mean would be 5; that is a different operation. These numbers illustrate the calculation, rather than reproducing a current product result.

To understand the slide's large results table, read one row at a time: identify the program; inspect IC, CPI, and cycle time; connect them to execution time; then compare execution time with the reference time. The final geometric mean summarizes ratios, not raw seconds.

Checklist for a comparison

1. Identify the workload and input.
2. Decide whether the question concerns latency, throughput, elapsed time, or CPU time.
3. Use IC, CPI, and frequency together when calculating CPU time.
4. Recalculate the total after changing only one component.
5. Record assumptions about clock, input, parallelism, and other costs.

Remember: A performance claim is useful when you know what was run and what was measured.

Chapter 2 | Learn MIPS one action at a time

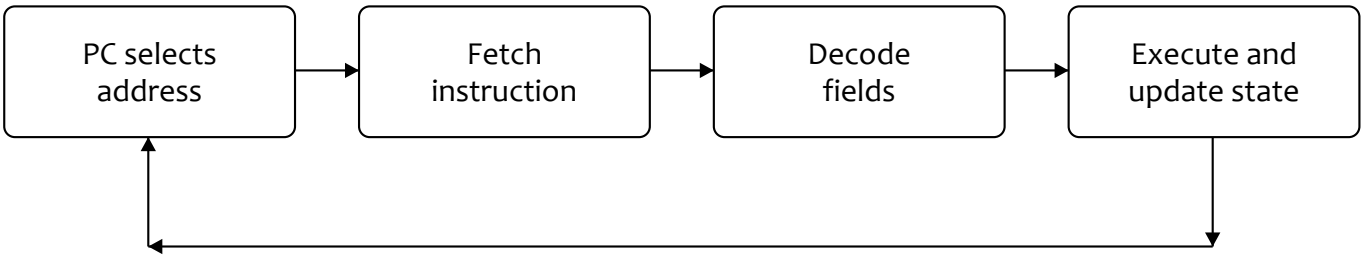
2.1 The ISA and the stored-program machine

Source: Chapter2.pdf pp. 1-10.

Goal: Explain how a processor finds and follows instructions.

An ISA defines machine instructions' meanings: add adds operands and lw loads a word. Different implementations can use different circuits and take different times while keeping those meanings.

The **stored-program concept** puts encoded instructions in the conceptual memory system alongside data. The processor fetches instructions by address. Separate internal instruction/data caches can still preserve this programmer-visible model.



1. **PC** supplies an instruction byte address.

2. Fetch the instruction bits into internal storage.

3. Decode the operation and operands.

4. Execute and update the required state.

5. Choose the next address; sequential instructions are 4 bytes apart.

Slide labels **IR**, **MAR**, and **MBR** name internal storage for an instruction, accessed address, and transferred data. They are not additional general-purpose registers that Assembly can name like $t0.

The slides divide instructions into arithmetic, data transfer, logical operations, conditional branches, and unconditional jumps. An expression such as a + b + c needs intermediate work: first add two values, then add the third. One machine instruction describes a limited operation.

The four design principles

Principle	Beginner interpretation
Simplicity favors regularity	Repeated formats are easier to implement and understand
Smaller is faster	Small, carefully chosen structures can be quicker to access
Make the common case fast	Spend effort on frequent operations
Good design demands good compromises	No single choice optimizes every goal

Remember: PC tracks instruction addresses. Registers such as $t0 usually track data values or addresses needed by instructions.

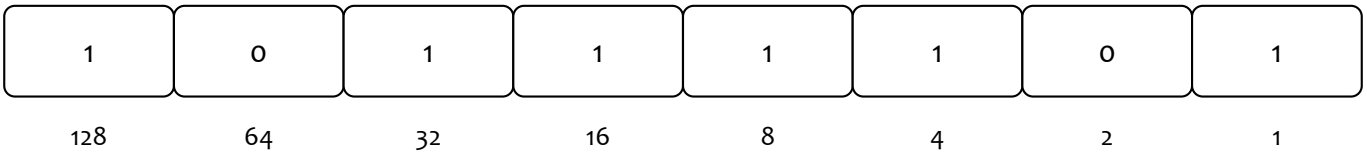
2.2 A small foundation: bits, binary, hex, and signed values

Source: Foundation added for Chapter2.pdf pp. 9, 11 and 53-70.

Goal: Read the bit patterns used in later encoding examples.

A **bit** is a binary digit, 0 or 1. A **byte** contains 8 bits. In this guide a **word** contains 32 bits, or 4 bytes. Binary uses powers of 2. Hexadecimal, usually marked with 0x, uses base 16; one hex digit represents exactly four bits.

One byte: 10111101



For the 8-bit pattern 10111101, the set positions contribute 128 + 32 + 16 + 8 + 4 + 1 = 189. Grouping the bits in fours gives 1011 1101, which is hex BD. Therefore 0xBD is 189 when read as an unsigned byte.

Decimal	Binary nibble	Hex
0 / 1 / 2 / 3	0000 / 0001 / 0010 / 0011	0 / 1 / 2 / 3
4 / 5 / 6 / 7	0100 / 0101 / 0110 / 0111	4 / 5 / 6 / 7
8 / 9 / 10 / 11	1000 / 1001 / 1010 / 1011	8 / 9 / A / B
12 / 13 / 14 / 15	1100 / 1101 / 1110 / 1111	C / D / E / F

The same bits can mean different numbers

Unsigned 8-bit values range from 0 to 255. Signed 8-bit **two's-complement** values range from -128 to 127. If the top bit is 1, the signed interpretation of a byte is its unsigned interpretation minus 256. Thus 10111101 can mean unsigned 189 or signed 189 - 256 = **-67**. The bits do not change; the interpretation changes.

For 32 bits, unsigned values range from 0 to 2^32 - 1. Signed two's-complement values range from -2^31 to 2^31 - 1. 0xFFFFFFFF means 4,294,967,295 unsigned or -1 signed. This matters for slt versus sltu.

To form a negative value in a fixed width, invert the positive pattern and add 1. For -2 in 16 bits: +2 is 0x0002; invert to 0xFFFD; add 1 to obtain 0xFFFE. Always state the width, because -2 in 32 bits is 0xFFFFFFFE.

Remember: Width and signedness are part of the question. A hex pattern alone does not settle its signed meaning.

2.3 Registers, immediates, and instruction syntax

Source: Chapter2.pdf pp. 11-12, 43 and 56.

Goal: Recognize the three operand forms before executing an instruction.

A general-purpose register is a 32-bit working location inside the processor. MIPS has 32 of these registers, numbered 0-31. Names are aliases for numbers: $t0 is register 8; $s0 is register 16. Names like $s0 do not permanently bind the register to a particular C variable; an example or compiler chooses the assignment.

Name	Number(s)	Usual role
$zero	0	Always reads as zero; writes do not change it
$at	1	Assembler temporary
$v0-$v1	2-3	Function results
$a0-$a3	4-7	Function arguments
$t0-$t7	8-15	Temporary values
$s0-$s7	16-23	Values preserved by a callee that changes them
$t8-$t9	24-25	More temporaries
$k0-$k1	26-27	Reserved for kernel use
$gp / $sp / $fp / $ra	28 / 29 / 30 / 31	Global, stack, frame, return-address roles

Operand form	Example	What it describes
Register	$t0	A register location
Immediate	20	A constant encoded in the instruction
Memory operand	12($s0)	A byte address formed using a base and offset

In addi $t0, $t1, 20, the last operand is the number 20, not register 20 and not the contents of memory address 20. In lw $t0, 20($t1), 20 contributes to an address; the instruction loads the contents at that address.

A mnemonic is the operation's short name, such as add. Commas separate operands. A comment beginning with # explains the code but does not execute. A label such as Loop: gives a name to an instruction address.

Remember: First classify the operands. Then interpret them according to the instruction.

2.4 Arithmetic: destination first, then sources

Source: Chapter2.pdf pp. 12-13.

Goal: Translate and trace arithmetic without destroying values you still need.

```
add  $t0, $s0, $s1    # t0 receives s0 + s1
sub  $t1, $s2, $s3    # t1 receives s2 - s3
addi $t2, $t1, 20     # t2 receives t1 + 20
```

Read sub carefully: it subtracts the third operand from the second, then writes the first. For an increment, addi $t0, $t0, 1 reads the old $t0 and replaces it with old $t0 + 1.

The slide asks for f = (g + h) - (i + j), with g, h, i, j, f assigned to $s0, $s1, $s2, $s3, $s4. Split the expression at its parentheses.

```
add $t0, $s0, $s1     # calculate g + h
add $t1, $s2, $s3     # calculate i + j
sub $s4, $t0, $t1     # calculate the difference
```

With g = 9, h = 4, i = 3, j = 2:

Instruction just completed	**$t0**	**$t1**	**f / $s4**
First add	13	unchanged	unchanged
Second add	13	5	unchanged
Subtract	13	5	8

The original g, h, i, and j remain available. The slide also shows a version that overwrites source-variable registers. It computes the expression but changes g and h; use that version only if those original values are no longer required. Temporaries make your first translations easier to check.

add, sub, and addi can raise a signed-overflow exception. Their counterparts addu, subu, and addiu do not raise that exception and keep the low 32 result bits. The u in addiu does **not** mean that its immediate is zero-extended; its immediate is sign-extended, just like addi. All examples here use small values unless otherwise stated.

Remember: Break an expression into one operation per line. Keep an explicit register-to-variable map beside it.

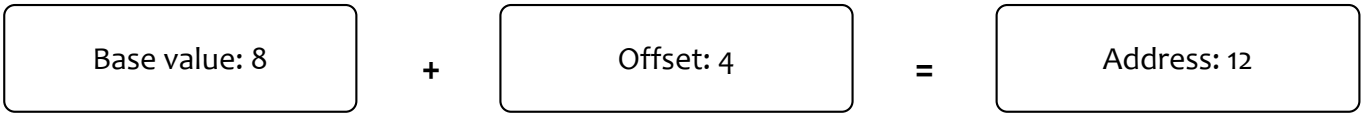
2.5 Memory addresses, offsets, and alignment

Source: Chapter2.pdf pp. 14-16.

Goal: Calculate where a load or store accesses memory.

Memory is byte-addressed. Address 8 identifies one byte; address 9 identifies the next byte. A word uses four consecutive bytes. An address is a location, not the value stored at that location.

Effective address = Value in base register + Sign-extended byte offset



offset(base) supplies a byte address, not the data value

Suppose $s0 contains 8. The operand 4($s0) refers to byte address 8 + 4 = 12. It does not mean "fourth element" unless the surrounding data layout happens to make that interpretation correct. The offset is already measured in bytes.

Operand with base $s0 = 1000	Effective byte address
0($s0)	1000
4($s0)	1004
12($s0)	1012
-4($s0)	996

Alignment

Ordinary lw and sw word accesses need addresses divisible by 4. Ordinary halfword accesses need addresses divisible by 2. Byte accesses can use any byte address. A misaligned ordinary access is an error in this model; it is not a request for the processor to round the address.

Access	Size	Suitable addresses
Byte	1 byte	Any byte address
Halfword	2 bytes	... 8, 10, 12, 14 ...
Word	4 bytes	... 8, 12, 16, 20 ...

For a word array, element zero begins at the base. Element one begins 4 bytes later. For a character array, element one is only 1 byte later. Data type determines the stride.

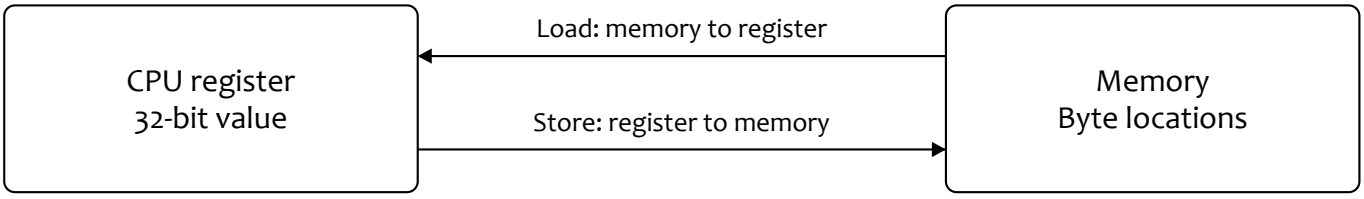
The offset encoded by ordinary loads and stores is a signed 16-bit value: -32,768 to 32,767 bytes. A larger displacement requires address construction using extra instructions. The slide's 2^30 word count refers to the theoretical capacity of a 2^32-byte address space, not a promise that every machine physically contains that much RAM.

Remember: Compute the address first. Check alignment. Only then read or write the contents.

2.6 Loads, stores, extension, and large constants

Source: Chapter2.pdf pp. 17-19; Materials/MIPSreferenceCard.pdf.

Goal: Distinguish the direction and width of a transfer.



Copying does not erase the source

Instruction	Direction	Width	What happens to the register
lw	Memory to register	4 bytes	Replaces all 32 bits
lb / lbu	Memory to register	1 byte	Sign-extend / zero-extend to 32 bits
lh / lhu	Memory to register	2 bytes	Sign-extend / zero-extend to 32 bits
sw	Register to memory	4 bytes	Register itself stays unchanged
sb / sh	Register to memory	1 / 2 bytes	Writes low 8 / 16 register bits

A load copies data; it does not remove the data from memory. A store copies data; it does not empty the register. Arithmetic normally uses register operands, so an expression involving memory needs a load, computation, and possibly a store.

Why extension is necessary

A register has 32 bits, but a loaded byte supplies only 8. **Sign extension** repeats the byte's top bit into the remaining 24 positions. **Zero extension** fills those positions with zeros.

Byte in memory	lb result	lbu result
0x2A	0x0000002A	0x0000002A
0xBD	0xFFFFFFBD, signed -67	0x000000BD, unsigned 189

The source memory byte is identical in both cases. With a halfword 0x8001, lh produces 0xFFFF8001; lhu produces 0x00008001.

lui **is not a memory load**

lui $t0, 0x1234 places the immediate in the upper 16 bits and zeros the lower 16 bits: $t0 = 0x12340000. Follow it with ori $t0, $t0, 0x5678 to construct 0x12345678. There is no base register or memory read in lui.

Logical immediates in andi, ori, and xori are zero-extended. Arithmetic, comparison, branch-displacement, and load/store-offset immediates discussed here are sign-extended where their operation requires extension. sltiu is an especially easy exception to guess incorrectly; Lesson 2.14 explains it.

Remember: The u in lbu changes extension. The u in addiu changes overflow behavior. Learn the rule for each instruction family.

2.7 Byte order and a complete memory trace

Source: Chapter2.pdf pp. 19 and 22; Materials/MipsInstructionSetReference.pdf.

Goal: Reconstruct words from bytes and track which bytes a store changes.

Big-endian stores the most significant byte at the lowest address. **Little-endian** stores the least significant byte there. Both represent the same 32-bit register value. MIPS implementations can support different byte orders; the slides choose big-endian for these examples.

A word value 0x12345678 beginning at address 8	Address 8	Address 9	Address 10	Address 11
Big-endian	12	34	56	78
Little-endian	78	56	34	12

The table's byte entries are hexadecimal. A byte load uses the byte at its exact address. Byte order becomes relevant when combining several bytes into a wider value.

Work through the slide's map

Use big-endian memory with bytes at addresses 8-15 equal to 12 34 56 78 9A BC DE F0. Let $t0 = 8 and $s0 = 0xCAFEFACE. For the first four instructions, none writes memory:

Instruction	Address / bytes read	Result
lw $t1, 0($t0)	8; 12 34 56 78	$t1 = 0x12345678
lw $t2, 4($t0)	12; 9A BC DE F0	$t2 = 0x9ABCDEF0
lh $t6, 4($t0)	12; 9A BC	$t6 = 0xFFFF9ABC
lb $t5, 3($t0)	11; 78	$t5 = 0x00000078

Now execute the stores in sequence. sw $s0, 0($t0) changes addresses 8-11 to CA FE FA CE. sb $s0, 4($t0) writes the lowest byte, CE, to address 12. It leaves addresses 13-15 unchanged. The final bytes are CA FE FA CE CE BC DE F0.

Finally, lh $s0, 7($t0) tries to access address 15. A halfword needs an even address, so this ordinary load raises an alignment error in the teaching model. Do not manufacture a normal loaded result. If the seven exercise items were intended as independent experiments, reset memory before each; the question must specify that convention. The trace above explicitly executes them in order.

Remember: Write out byte addresses. A four-byte word name hides the details you need for partial stores.

2.8 Word arrays: load, calculate, store

Source: Chapter2.pdf pp. 20-21 and 23.

Goal: Convert a constant array index into a byte offset.

For a word array A, address of A[i] = base of A + 4 x i. Index 8 means the ninth element because indexing begins at zero. Its byte offset is 32, not 8.

The slide's g = h + A[8] places g in $s1, h in $s2, and the base of A in $s3:

```
lw  $t0, 32($s3)      # to receives A[8]
add $s1, $s2, $t0     # g receives h + A[8]
```

If $s3 = 1000, h = 7, and the word at address 1032 is 20, the load gives $t0 = 20; the addition gives $s1 = 27. $s3 remains the address 1000. It is not replaced with A[8].

To implement A[12] = h + A[8], the destination is memory:

```
lw  $t0, 32($s3)
add $t0, $s2, $t0
sw  $t0, 48($s3)      # A[12] is 12 x 4 bytes from the base
```

Stage	Value or location	What changes
Locate source	base + 32	No data changes yet
Load source	A[8] into $t0	$t0
Compute sum	h + A[8]	$t0
Locate / store destination	base + 48	Four bytes of A[12]

A common error is trying add $s1, $s2, 32($s3). That is not the syntax for a MIPS register-add instruction. Load the memory value into a register first.

For f = g + h + B[4], load B[4] at offset 16, then perform two additions. Keep track of whether a temporary is an address or a loaded value. They are both 32-bit patterns but play different roles.

Remember: Array indexing describes elements. Memory instructions describe byte displacements.

2.9 Variable indexes and nested array access

Source: Chapter2.pdf p. 23; shift instruction introduced on p. 25.

Goal: Build an address when the index comes from a register or another array.

An ordinary memory operand cannot contain a second register as its offset. To access A[i], calculate 4 x i in a temporary, add the array base, then load or store at offset zero. A logical left shift by 2 multiplies a small nonnegative index by 4.

```
sll $t0, $s1, 2       # t0 = byte displacement, if s1 contains i
add $t0, $s6, $t0     # t0 = address of A[i]
lw  $t1, 0($t0)       # t1 = value A[i]
```

The same $t0 changes roles: first a displacement, then a complete address. Commenting that change helps you avoid using it as though it were the loaded element.

The slide's nested expression: f = g - A[B[4]]

Assume f is $s0, g is $s1, A's base is $s6, and B's base is $s7. B and A are word arrays. B[4] contains an element index into A, not an address.

```
lw  $t0, 16($s7)      # read B[4], an index into A
sll $t0, $t0, 2       # convert that index to a byte displacement
add $t0, $s6, $t0     # form address of A[B[4]]
lw  $t1, 0($t0)       # read A[B[4]]
sub $s0, $s1, $t1     # f = g - that value
```

Given g = 30, A base = 1000, B[4] = 3, A[3] = 8	Temporary meaning
After first load	$t0 = 3, element index
After shift	$t0 = 12, byte displacement
After address add	$t0 = 1012, memory address
After second load	$t1 = 8, element value
After subtract	f = 22

The code assumes valid indexes, sufficient array bounds, aligned addresses, and address calculations that fit the model. C array access does not become automatically bounds-checked when translated into MIPS. Bounds checks must be added when required by the problem.

Remember: A nested expression needs more than one memory access. Resolve the inner index before calculating the outer address.

2.10 Shifts: move bits within a fixed width

Source: Chapter2.pdf pp. 24-25 and 30.

Goal: Understand what enters and leaves a register during a shift.

sll $t0, $t1, k shifts $t1 left by k positions and fills the right with zeros. srl shifts right and fills the left with zeros. Bits pushed beyond the 32-bit register disappear. The immediate shift amount must fit a 5-bit field: 0-31.

Small 8-bit illustration	Before	After
Shift left by 2	00000101	00010100
Logical shift right by 2	00010110	00000101
Shift left by 1 with a lost top bit	10000001	00000010

The first example changes 5 to 20. For a value that fits, shifting left by k multiplies by 2^k. In a fixed width, lost high bits can invalidate the ordinary mathematical multiplication result. For nonnegative indexes in range, shifting left by 2 is a convenient way to form a word-array displacement.

Logical right shift of an unsigned value divides by 2^k and discards the remainder. It is not a general signed division instruction. For example, shifting 0xFFFFFFFF right by 1 logically gives 0x7FFFFFFF, not signed -1.

The reference card also includes sra, arithmetic right shift, which fills the left with copies of the sign bit. For negative inputs it behaves differently from srl; its rounding is not always the same as signed integer division that truncates toward zero.

Correct the source exercise before calculating

Chapter 2 p. 30 prints sll $t2, $t0, 44. A 44-bit immediate shift is not encodable as an ordinary 32-bit MIPS sll. Do not silently reduce it modulo 32. Confirm the intended exercise with your instructor. Valid shifts such as 4 can be calculated normally.

For sll $s2, $s0, 4 with $s0 = 0x12345678, move every hex digit left one position, discard the first digit, and add a zero: $s2 = 0x23456780.

Remember: State the width, direction, fill rule, and shift amount before calculating.

2.11 Bitwise logic and masks

Source: Chapter2.pdf pp. 24 and 26-30.

Goal: Change selected bits without treating the whole value as one Boolean.

Bitwise operations apply the same rule independently at each bit position. They differ from high-level Boolean operators such as C's &&, which test truth values.

Input bit A	Input bit B	A AND B	A OR B	A NOR B
0	0	0	0	1
0	1	0	1	0
1	0	0	1	0
1	1	1	1	0

An AND mask keeps positions whose mask bit is 1 and clears positions whose mask bit is 0. An OR mask sets positions whose mask bit is 1 and leaves the other positions alone.

```
andi $t0, $t1, 0x00FF  # keep only the lowest byte
ori  $t0, $t1, 0x0001  # ensure the lowest bit is 1
nor  $t0, $t1, $zero   # invert all 32 bits of t1
```

nor means NOT(A OR B). Since OR with zero leaves A unchanged, NOR with zero complements A. A not spelling may be accepted as a pseudoinstruction, but this core operation is represented by nor.

andi and ori have 16-bit immediate patterns that are zero-extended. Therefore andi always makes the result's upper 16 bits zero. ori leaves the source's upper 16 bits unchanged. To AND with an arbitrary 32-bit mask, construct that mask in a register and use register and.

Recreate the slide's calculations

Let A = 0x12345678, B = 0xCAFEFACE. Work nibble by nibble using the truth table. The results are A AND B = 0x02345248; A OR B = 0xDAFEFEFE. Decimal 2020 is hex 0x07E4; A AND 0x000007E4 = 0x00000660.

For a valid 16-bit immediate pattern of 0xFFFE, andi clears bit zero and all upper 16 bits. The slide spells this pattern as -2 in one exercise; whether an assembler accepts that spelling for a logical immediate is tool-dependent. The zero-extension semantics apply to the encoded pattern. Use 0xFFFE to make your intent clear.

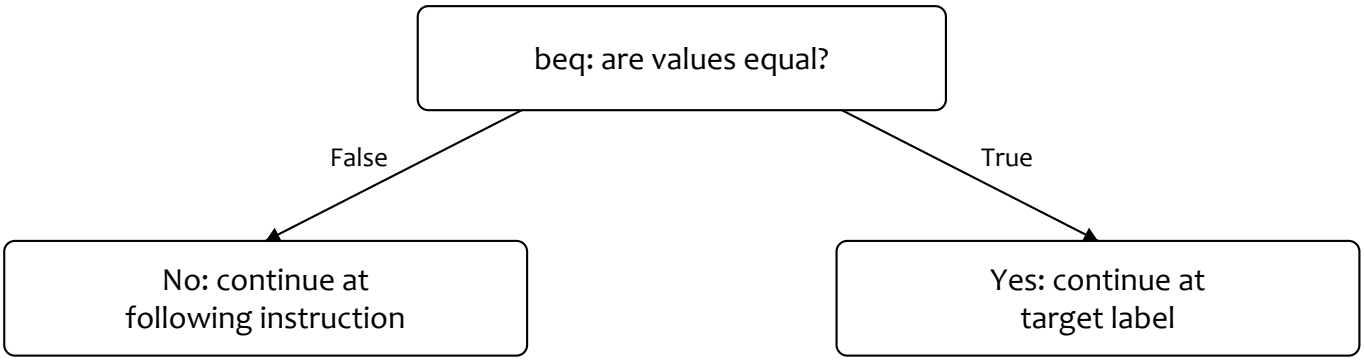
Remember: A mask is a pattern of permissions: AND chooses what survives; OR chooses what must be set.

2.12 Branches, labels, and a tiny loop trace

Source: Chapter2.pdf pp. 31-32.

Goal: Follow a program when the next instruction is not always the next line.

beq branches if two register values are equal. bne branches if they are different. If the condition is false, execution continues at the following instruction in this guide's no-delay-slot model. A label names an instruction address; it does not perform an action by itself.




```
addi $t0, $zero, 0
addi $t1, $zero, 5
Loop: addi $t0, $t0, 1
bne  $t0, $t1, Loop
add  $t0, $t1, $t0
```

Trip through Loop	$t0 after increment	Is $t0 different from 5?	Next action
1	1	Yes	Branch to Loop
2	2	Yes	Branch to Loop
3	3	Yes	Branch to Loop
4	4	Yes	Branch to Loop
5	5	No	Continue to final add

After the loop finishes, the final add computes 5 + 5, so $t0 becomes 10. The branch compares the current values, after the increment. It does not compare a remembered value from the previous trip.

j Label changes control unconditionally. It does not compare registers or save a return address. jal performs a call and saves a return address; jr jumps to an address contained in a register. Procedure lessons explain those roles separately.

When tracing, draw an arrow to the next instruction after each branch. Otherwise it is easy to execute a line that should have been skipped.

Remember: A branch changes the instruction sequence. It does not move a data value to its label.

2.13 If / else: make both paths explicit

Source: Chapter2.pdf p. 33.

Goal: Translate a high-level choice without accidentally executing both alternatives.

Suppose if (i == j) f = g + h; else f = g - h;. Use f, g, h, i, j in $s0 through $s4, respectively. One convenient strategy branches to Else when the high-level condition is false.

```
bne  $s3, $s4, Else    # if i != j, skip the addition
add  $s0, $s1, $s2     # true path: f = g + h
j    Done              # skip the else path
Else: sub $s0, $s1, $s2
Done:
```

Starting values	Branch decision	Instruction that calculates f	Result
g = 7, h = 3, i = 2, j = 2	bne is false	add	10
g = 7, h = 3, i = 2, j = 4	bne is true	sub	4

The unconditional j Done is necessary after the true-path addition. Without it, the processor would continue into Else and overwrite the addition result with a subtraction result.

You can also branch to the true case using beq, placing the false case first. Both arrangements can be correct. When checking a translation, test one input that takes each path rather than looking only at the line count.

A reusable translation recipe

1. Write the true and false actions in words.

2. Choose which condition will jump and which action will be the fall-through path.

3. Place a jump after the first action if it must skip the second.

4. Join both paths at one Done label.

Remember: Labels organize control flow. The path you take determines which assignment happens.

2.14 Less-than tests and signed versus unsigned comparisons

Source: Chapter2.pdf pp. 35-39; Materials/MIPSreferenceCard.pdf.

Goal: Build relational conditions using a 0-or-1 result.

slt $t0, $s1, $s2 writes 1 when $s1 is less than $s2 as signed values, otherwise 0. It does not branch. sltu compares the same 32-bit patterns as unsigned values. Follow the comparison with beq or bne to select a path.

Values: $s1 = 0xFFFFFFFF, $s2 = 0x00000001	**Interpretation**	**Result**
slt $t0, $s1, $s2	-1 < 1	$t0 = 1
sltu $t1, $s1, $s2	4,294,967,295 < 1	$t1 = 0

For the high-level condition i < j, first compute it, then branch to Else if the result is zero:

```
slt $t0, $s3, $s4
beq $t0, $zero, Else
add $s0, $s1, $s2
j   Done
Else: sub $s0, $s1, $s2
Done:
```

To test a >= 5, compute slti $t0, $s0, 5 (a < 5). A zero result means a >= 5. Inverting a condition often avoids extra instructions.

An immediate rule that deserves special care

The correct mnemonic is **sltiu**, not the slide's sltui spelling. sltiu first sign-extends its 16-bit immediate, then compares both 32-bit operands as unsigned values. For immediate -1, the comparison value becomes 0xFFFFFFFF. If $t0 = 5, sltiu $t1, $t0, -1 writes 1. It does not compare 5 with a zero-extended 0x0000FFFF merely because its mnemonic ends in u.

The slides emphasize beq and bne as the two-register equality branches in their chosen core subset. The wider MIPS ISA has other branch instructions too. The design-principle discussion is about why combining a separate compare with a common branch can be a useful compromise, not a claim that no other MIPS branches exist.

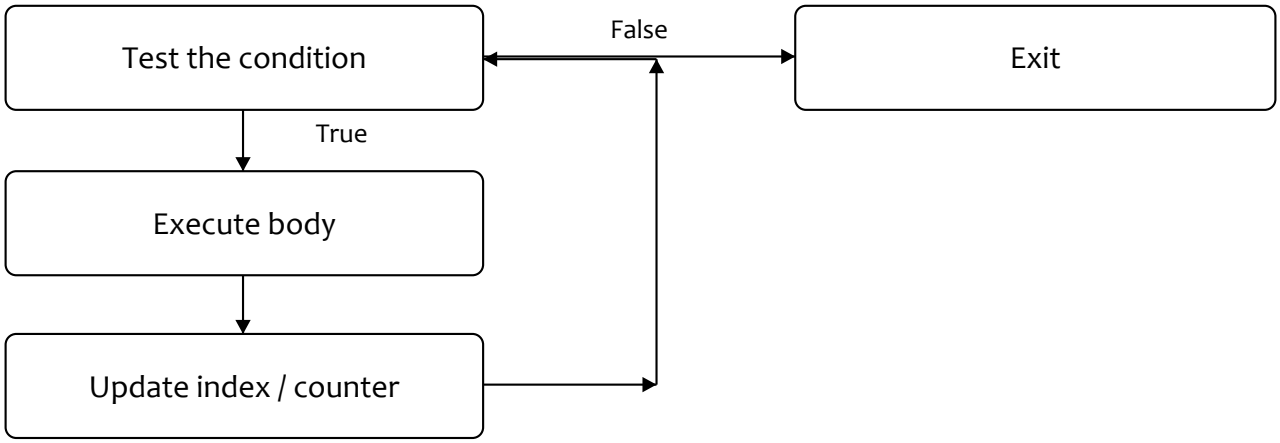
Remember: A comparison creates a Boolean value. A branch acts on a condition. Keep those two actions separate in your trace.

2.15 While loops and for loops

Source: Chapter2.pdf pp. 34 and 38.

Goal: Translate a loop as initialization, test, body, update, and repeat.

A while loop tests before each execution of its body. If the first test is false, the body runs zero times. A for loop usually packages initialization, condition, and update into one high-level line; Assembly makes those actions visible.



The slide's loop is i = 0; while (save[i] == k) i += 1;. Let the word-array base be $s0, i be $s1, and k be $s2.

```
addi $s1, $zero, 0
Loop:
sll  $t0, $s1, 2      # byte displacement of save[i]
add  $t0, $s0, $t0    # address of save[i]
lw   $t1, 0($t0)      # value save[i]
bne  $t1, $s2, Done   # stop when the condition becomes false
addi $s1, $s1, 1
j    Loop
Done:
```

save = [7, 7, 4], k = 7	i at test	Loaded value	Next action
First test	0	7	Increment i
Second test	1	7	Increment i
Third test	2	4	Exit with i = 2

The question assumes a differing element exists within accessible memory. A production routine needs a bound to avoid reading beyond the array. If bounds matter, test i against the array length before loading the element.

Translate the slide's for loop

For for (i = 0, a = 0; i < 10; i++) a++;, use $s0 for a and $s1 for i:

```
addi $s0, $zero, 0
addi $s1, $zero, 0
Loop:
slti $t0, $s1, 10
beq  $t0, $zero, Done
addi $s0, $s0, 1
addi $s1, $s1, 1
j    Loop
Done:
```

The body runs for i = 0 through 9. At the next test i = 10, so it exits with a = 10 and i = 10. The test itself executes 11 times. This distinction matters when counting dynamic instructions.

Remember: Before writing instructions, mark the loop's five jobs on paper.

2.16 Pseudoinstructions and assembler conveniences

Source: Chapter2.pdf p. 40; Materials/MIPSreferenceCard.pdf.

Goal: Distinguish convenient source spelling from actual encoded instructions.

An assembler can accept a **pseudoinstruction** and expand it into one or more real instructions. That makes Assembly source easier to write, but a pseudoinstruction does not necessarily have its own opcode.

Source convenience	One possible real-instruction implementation
move $t0, $t1	addu $t0, $t1, $zero
blt $t0, $t1, Less	slt $at, $t0, $t1; then bne $at, $zero, Less
li $t0, 7	addiu $t0, $zero, 7
li $t0, 0x12345678	lui $t0, 0x1234; then ori $t0, $t0, 0x5678

Exact expansions can depend on the assembler and the constant. For example, a value that fits in one immediate can use one instruction. A larger value may need two. move with a zero source cannot overflow; assemblers commonly use addu or another equivalent move operation.

$at is reserved for assembler-generated work. If you manually store an important value there, a later pseudoinstruction can overwrite it. Use ordinary temporary registers in beginner hand-written code.

Labels and directives are other conveniences. A label gives an address a name. A directive such as .word describes data to the assembler; it is not an instruction the CPU executes. A data label's address differs from the value stored there. This instruction-set deck does not require simulator I/O services for its examples.

Why this matters for machine code

To encode blt, first choose its real expansion. Then assign addresses to the expanded instructions. Then calculate a branch displacement. Counting the source's pseudo lines as though they were always one machine word can give the wrong address.

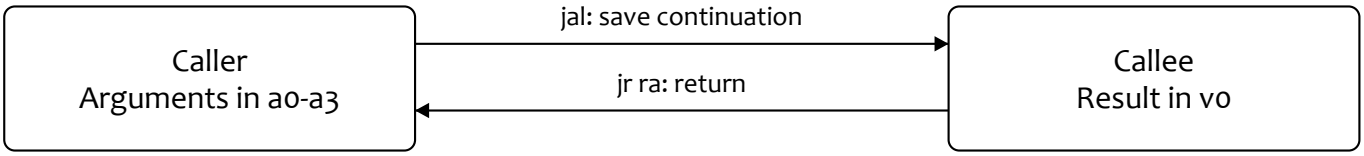
Remember: When asked for machine code or an exact instruction count, expand conveniences before counting or encoding.

2.17 Calls, returns, and the execution-model choice

Source: Chapter2.pdf pp. 32, 41-45; Materials/MIPSreferenceCard.pdf.

Goal: Explain how a procedure receives control and finds its way back.

The **caller** requests a procedure's work. The **callee** is the procedure doing that work. A call transfers control while remembering where execution should resume. An ordinary jump does not remember a return address.



Registers carry values; control transfers choose instructions

Action	MIPS role
Put up to four simple arguments in registers	$a0-$a3
Call the procedure	jal Procedure
Produce a simple return value	$v0, sometimes $v1 too
Return to the stored continuation address	jr $ra

In the slides' no-delay-slot model, if jal Double is at address 1000, $ra receives 1004. A following instruction at 1004 can use the result after the callee returns. jr $ra reads that stored address and changes the PC to it.

```
# Caller has placed an argument in a0
jal  Double
add  $s0, $v0, $zero   # continuation in this guide's model

Double:
add  $v0, $a0, $a0
jr   $ra
```

These are fragments showing the call and callee; a complete layout also needs to prevent normal fall-through from entering the procedure after the caller's code finishes.

Why the reference card gives another return address

Classic delayed MIPS executes one following instruction before transferring control: the **delay slot**. Classic jal saves PC + 8, after that slot, as the reference card records. SPIM has an option for delayed transfers [W5].

For a simple classic fragment, nop (sll $zero, $zero, 0) can fill the slot after a call or return. Put the continuation after the call's slot. Branches and jumps also need safe slots. This guide's traces instead use the slides' no-delay-slot model; use whichever model a question or simulator specifies.

Remember: A call needs a continuation address. The address depends on the stated execution model.

2.18 Calling conventions and the stack

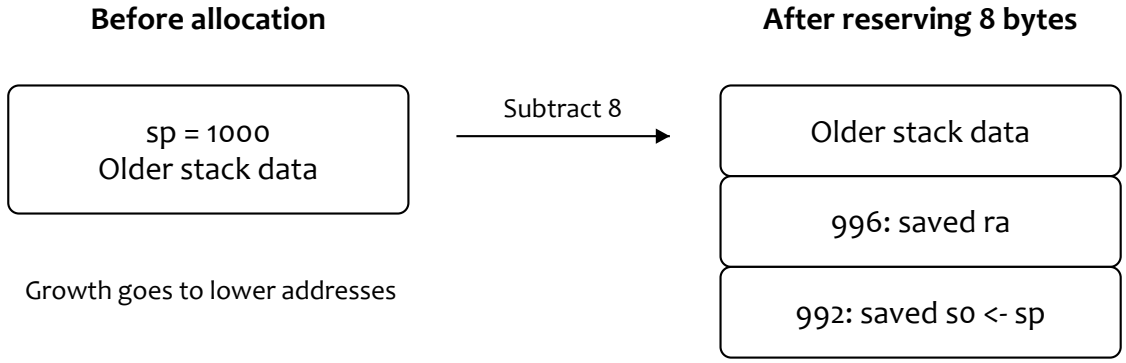
Source: Chapter2.pdf pp. 43-44 and 48.

Goal: Explain which values need saving and where saved values live.

A calling convention is an agreement between caller and callee. The CPU does not automatically preserve every $s register because its name begins with s. The callee's instructions must preserve any callee-saved register that it changes.

Register group	Who must take care when a value is needed?
$t0-$t9	Caller saves a live value before calling code that may clobber it
$a0-$a3, $v0-$v1	Caller should expect calls to change them
$s0-$s7	Callee saves and restores the ones it changes
$ra	A returning non-leaf procedure preserves its incoming return address before a nested call
$sp	A returning procedure restores its stack allocation before returning

The **stack** is a memory region used for temporary procedure storage. In this course model it grows toward lower addresses. $sp points to the current stack location. Allocating space subtracts from $sp; releasing space adds the same amount back.




```
addiu $sp, $sp, -8    # reserve 8 bytes
sw   $s0, 0($sp)    # keep the incoming s0 value
sw   $ra, 4($sp)    # keep the incoming return address if needed
# ... procedure work ...
lw   $s0, 0($sp)
lw   $ra, 4($sp)
addiu $sp, $sp, 8    # release the same space
```

With initial $sp = 1000, allocating 8 bytes makes it 992. Offsets 0 and 4 refer to words at 992 and 996. Restoring the registers must happen before releasing the frame, or you must adjust the offsets accordingly.

These compact teaching frames use 8-byte sizes when possible for alignment. The source leaf example uses 4 bytes for one saved word. A full platform ABI can require additional areas and rules.

Remember: Every returning path must balance its allocation and restore the values its caller is entitled to keep.

2.19 A leaf procedure, one phase at a time

Source: Chapter2.pdf pp. 46-47.

Goal: Read a complete procedure with a prologue, body, and epilogue.

A **leaf** procedure makes no calls to other procedures or to itself. If it does not change $ra, it can return using the incoming $ra without saving it. If it changes a saved register, it must restore that register.

The source function computes (g + h) - (i + j). Arguments g, h, i, j arrive in $a0 through $a3. The implementation below uses $s0 for its local result and reserves 8 bytes; one word is padding.

```
leaf_example:
addiu $sp, $sp, -8
sw    $s0, 0($sp)     # prologue: preserve caller's s0

add   $t0, $a0, $a1   # body: g + h
add   $t1, $a2, $a3   # body: i + j
sub   $s0, $t0, $t1
addu  $v0, $s0, $zero  # put result where caller expects it

lw    $s0, 0($sp)     # epilogue: restore caller's s0
addiu $sp, $sp, 8
jr    $ra
```

Example state	Value
Incoming arguments	9, 4, 3, 2
Incoming $s0 / $sp	99 / 1000
After prologue	sp = 992; memory[992] holds 99
During body	t0 = 13; t1 = 5; s0 = 8; v0 = 8
At return	s0 = 99; sp = 1000; v0 = 8

The result remains in $v0 after $s0 is restored. If you restored $s0 before copying the result, you would copy the caller's old value instead.

A simpler implementation could use only $t registers and $v0, avoiding a saved register and frame. The longer version deliberately teaches preservation. Leaf status alone does not prove that a procedure needs no stack; local storage or saved-register use can still require it.

Remember: Preserve the caller's state and return the new result through the agreed result register.

2.20 Non-leaf calls and recursive factorial

Source: Chapter2.pdf pp. 48-50.

Goal: See why each active call needs its own saved return address and argument.

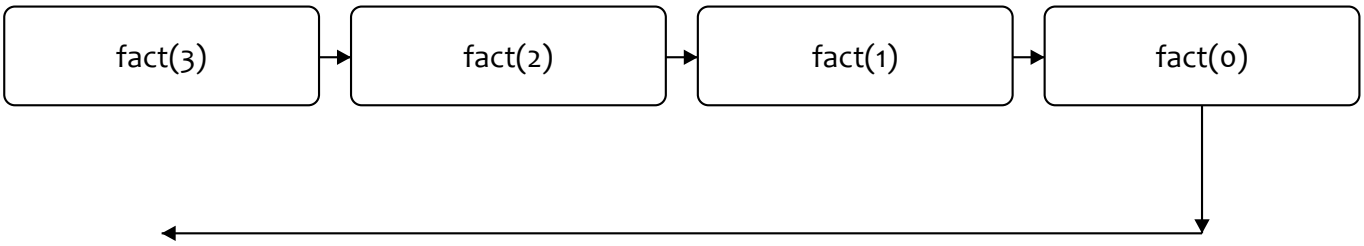
A **non-leaf** procedure calls another procedure or itself. A nested jal overwrites $ra. If the procedure needs to return normally to its own caller, it must save its incoming return address before that nested call.

The slide's factorial has a base case: for n < 1, return 1; otherwise return n x fact(n - 1). Use nonnegative inputs within the 32-bit result range for these examples. Each recursive invocation reserves its own 8-byte frame.

```
fact:
addiu $sp, $sp, -8
sw    $ra, 4($sp)
sw    $a0, 0($sp)     # preserve this call's original n
slti  $t0, $a0, 1
beq   $t0, $zero, Recurse
addi  $v0, $zero, 1
addiu $sp, $sp, 8     # base case made no call
jr    $ra

Recurse:
addi  $a0, $a0, -1
jal   fact
lw    $a0, 0($sp)     # recover original n
lw    $ra, 4($sp)     # recover original continuation
addiu $sp, $sp, 8
mul   $v0, $a0, $v0
jr    $ra
```

The base case does not change $ra after saving it, so it can use the current $ra; it still releases its frame. The recursive path must restore its saved $ra, because the nested call changed it. mul provides a low-32-bit product in the source's dialect; it is a MIPS32 instruction, while some earlier teaching subsets expand a multiplication convenience. A mult / mflo pair is another implementation when that subset is required. Multiplication internals belong to the later arithmetic chapter.



Return upward: 1, then 1, then 2, then 6

Phase of fact(3)	What is waiting or returned
Call downward	fact(3) waits for fact(2); fact(2) waits for fact(1); fact(1) waits for fact(0)
Base case	fact(0) returns 1
Unwind one frame	fact(1) computes 1 x 1 = 1
Unwind another	fact(2) computes 2 x 1 = 2
Unwind final frame	fact(3) computes 3 x 2 = 6

Every frame remembers a different n and a continuation belonging to that invocation. Never use one shared memory slot for all recursive arguments unless you implement a different scheme that preserves each pending value.

Remember: Recursion delays work. The stack records what each unfinished call will need when it resumes.

2.21 Character arrays and string copying

Source: Chapter2.pdf p. 51.

Goal: Use byte transfers and copy the terminator as well as the visible characters.

A C string is a sequence of bytes ending with a zero byte, \0. That terminator is not the printable character '0', whose ASCII value is 48. For a character array, the byte displacement of element i is i itself. Do not multiply it by 4.

The source strcpy exercise receives destination x in $a0 and source y in $a1. It requires enough writable space in x and a properly terminated, non-overlapping source. It copies the byte, tests the copied value, and only then increments i.

```
strcpy:
addiu $sp, $sp, -8
sw    $s0, 0($sp)
addi  $s0, $zero, 0
Copy:
addu  $t0, $a1, $s0    # source byte address
lbu   $t1, 0($t0)
addu  $t2, $a0, $s0    # destination byte address
sb    $t1, 0($t2)
beq   $t1, $zero, Done # terminator has already been copied
addi  $s0, $s0, 1
j     Copy
Done:
lw    $s0, 0($sp)
addiu $sp, $sp, 8
jr    $ra
```

Source bytes for "Hi"	Index	Byte read / written	Action after copy
H	0	0x48	Continue
i	1	0x69	Continue
Terminator	2	0x00	Exit

The destination becomes 48 69 00. If you test before copying and branch around the store for zero, the destination may lack its terminator. lbu is convenient because the byte becomes a nonnegative value; lb would also permit a zero test and low-byte store, but would sign-extend bytes with top bit 1.

This routine is a leaf: it calls no other function. It restores $s0 because it uses that saved register. A version using only temporaries can be shorter.

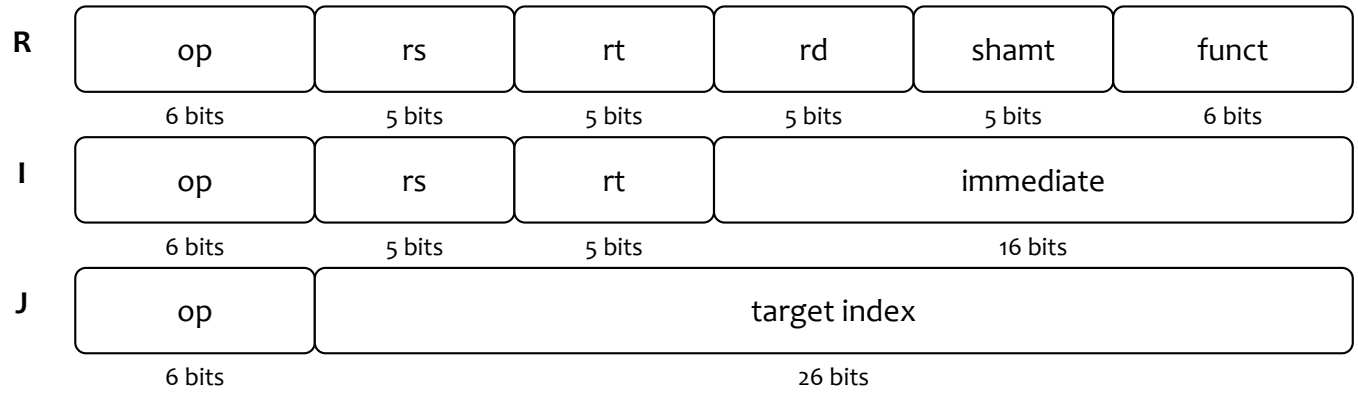
Remember: Bytes for strings; words for the integer arrays used earlier. Copy the terminating byte.

2.22 Machine code and the three formats

Source: Chapter2.pdf pp. 52-56 and 59-60; Materials/MIPSreferenceCard.pdf.

Goal: Treat machine encoding as filling fixed-width fields.

Each ordinary instruction in the source's MIPS subset occupies 32 bits. R, I, and J are layouts for those bits. Their fields do not appear in the same order as the destination-first Assembly syntax.



Format	Fields from most to least significant bits	Widths
R	opcode, rs, rt, rd, shamt, funct	6, 5, 5, 5, 5, 6
I	opcode, rs, rt, immediate	6, 5, 5, 16
J	opcode, target index	6, 26

For core arithmetic such as add, rs and rt identify the inputs and rd identifies the output. For immediate arithmetic, rt is the destination. For a store or two-register branch, rt is a source. Field names identify positions; instruction semantics determine their roles.

The slide gives useful introductory format heuristics. Use the instruction's actual encoding table rather than the rule "any nonzero opcode is I-format": the wider ISA has additional opcode groups. Core R instructions discussed here use opcode zero, but the wider ISA has operations such as MIPS32 mul using other opcode groups.

How to use the reference card

1. Find the instruction's mnemonic and format.

2. Copy its opcode and, if required, function code. Card values marked hex must be converted consistently.

3. Convert register names to register numbers.

4. Form the immediate, displacement, or target index according to that instruction.

5. Check that every field fits its width and the total is 32 bits.

6. Concatenate fields, then regroup into eight hex digits.

Five bits encode register numbers 0-31. Six bits encode values 0-63. Sixteen signed bits encode -32,768 through 32,767, while a logical immediate is a 16-bit pattern 0x0000-0xFFFF. The operation determines its later interpretation.

Remember: Choose the instruction and format before filling any field.

2.23 R-format encoding and decoding

Source: Chapter2.pdf pp. 55-58; Materials/MIPSreferenceCard.pdf.

Goal: Encode an arithmetic instruction and handle the different fields used by a shift.

For add $t0, $s1, $s2, the destination is $t0 (8), rs is $s1 (17), rt is $s2 (18). Opcode is 0, shamt is 0, and add's function code is hex 0x20, decimal 32.

Field	Decimal	Binary
opcode	0	000000
rs	17	10001
rt	18	10010
rd	8	01000
shamt	0	00000
funct	32	100000

000000 10001 10010 01000 00000 100000
0000 0010 0011 0010 0100 0000 0010 0000
Hex: 0x02324020

As a numerical cross-check, combine (rs << 21) | (rt << 16) | (rd << 11) | funct when opcode and shamt are zero. The shifts place each field in its required bit position.

Shifts are R-format too

For sll $t2, $s0, 4, rs is zero because this immediate shift does not use it. rt is $s0 (16), rd is $t2 (10), shamt is 4, and funct is 0. Its encoded word is **0x00105100**. When decoding the slide's binary pattern, split it into fields before reading the register roles.

For jr $ra, rs is 31 and rt, rd, shamt are zero; funct is 0x08. It encodes as 0x03E00008. The instruction reads an address from rs; it does not need an arithmetic destination.

Instruction	Function code, hex
add / sub	20 / 22
and / or / nor	24 / 25 / 27
sll / srl / jr	00 / 02 / 08
slt / sltu	2A / 2B

Chapter 2 p. 56 prints nor's function as 0x28 (39). Correct it to **0x27 (39)**. Hex 0x28 is decimal 40; the supplied reference card agrees with 0x27.

Remember: Read operand roles from the mnemonic, not from the visual order of the Assembly operands.

2.24 I-format, negative immediates, and memory operands

Source: Chapter2.pdf pp. 59-63.

Goal: Place the base register, transfer register, and 16-bit immediate correctly.

For lw $t0, 32($s3), opcode is hex 0x23 (35); rs is the base $s3 (19); rt is the destination $t0 (8); immediate is the byte offset 32.

Field	Decimal	Binary
opcode	35	100011
rs	19	10011
rt	8	01000
immediate	32	0000000000100000

The result is 100011 10011 01000 0000000000100000, or **0x8E680020.** The actual value in $s3 is not encoded in the instruction; its register number is. The same machine instruction can access different locations when the register's value changes.

For the slide's 0xAD280064, opcode is 0x2B (sw), rs is 9 ($t1), rt is 8 ($t0), and immediate is 100. It decodes as **sw $t0, 100($t1)**. rt is a source for sw, even though it is a destination for lw.

Encode a negative constant

For addi $t0, $t0, -1, opcode = 8, rs = 8, rt = 8, and the 16-bit immediate is 0xFFFF. The word is **0x2108FFFF**. Decode the immediate according to the mnemonic: in addi, 0xFFFF sign-extends to -1; in andi, the same immediate bits zero-extend to 65,535.

Some I-format opcodes	Hex values
addi / addiu / lui	08 / 09 / 0F
andi / ori	0C / 0D
slti / sltiu	0A / 0B
lb / lh / lw	20 / 21 / 23
lbu / lhu	24 / 25
sb / sh / sw	28 / 29 / 2B
beq / bne	04 / 05

Remember: A memory operand contributes two fields: base-register number and byte offset.

2.25 Branch displacements: begin at PC + 4

Source: Chapter2.pdf pp. 64-67.

Goal: Encode forward and backward branches without a one-instruction error.

Let P be the byte address of the branch instruction. The target of beq or bne is P + 4 + 4 x signed immediate. Therefore the field to encode is (target - (P + 4)) / 4. The division converts a byte difference to an instruction-word displacement.

Branch immediate = (Target byte address - (Branch address + 4)) / 4

This formula is the same for the classic delay-slot architecture and the simplified execution model. The models differ in what executes on the way to the target, not in this encoded displacement formula.

Forward branch from the slide

Place bne $s3, $s4, Else at x. The next two instructions are at x + 4 and x + 8; Else is at x + 12. The immediate is ((x + 12) - (x + 4))/4 = 2. Fields are opcode 5, rs 19, rt 20, immediate 2. The encoded word is **0x16740002**.

Backward branch from the slide

The loop instruction is at x - 4 and bne $t0, $t1, Label is at x. The immediate is ((x - 4) - (x + 4))/4 = -2. In 16 bits, -2 is 0xFFFE. Fields are opcode 5, rs 8, rt 9, immediate 0xFFFE. The correct word is **0x1509FFFE**.

Chapter 2 p. 67 prints 0x1611FFFE, which would name different registers. Its displayed field values (5, 8, 9, -2) support the corrected word above.

Target relative to a branch at P	Immediate
P + 4, the following instruction	0
P + 8	1
P, the branch's own address	-1
P - 4	-2

The signed displacement range is -32,768 to 32,767 words relative to P + 4, corresponding to byte offsets -131,072 to +131,068. The target must be word-aligned.

Remember: Subtract the following instruction's address, not the branch instruction's address.

2.26 J-format and the shared address region

Source: Chapter2.pdf pp. 68-69.

Goal: Encode a jump using its target index and the upper address bits.

j uses opcode 2; jal uses opcode 3. Their 26-bit target index contributes bits 27-2 of the destination. The processor uses the upper four bits of P + 4 and appends two zero bits at the bottom.

```
Target = upper 4 bits of (P + 4), then 26-bit index, then 00
Target = ((P + 4) & 0xF0000000) | (Index << 2)
Index = (Target >> 2) & 0x03FFFFFF
```

The two low bits are zero because ordinary instructions are word-aligned. The target must be in the same 256 MiB address region selected by the upper nibble of P + 4. A J-format field is not a complete independent 32-bit address.

The slide's decimal-address example

The first instruction, Label, begins at decimal address 80,000. A branch follows at 80,004 and j Label is at 80,008. P + 4 is 80,012, whose upper nibble is zero. The target index is 80,000/4 = 20,000 decimal = 0x4E20. Combine opcode 2 with that index to get **0x08004E20**.

Be careful: 80,000 in this example is decimal. A number beginning with 0x is hexadecimal. Mixing those representations changes the target index.

For an address in the high region, mask away the region bits when forming the 26-bit index. If the desired target lies in another region, an assembler or programmer can construct a full target address in a register and use jr or a suitable call mechanism instead.

Remember: A jump index is an aligned partial address. A branch immediate is a signed relative displacement. They are different calculations.

2.27 Two complete Assembly-to-machine-code examples

Source: Chapter2.pdf pp. 63 and 70.

Goal: Combine C translation, register fields, and label addresses.

First translate A[300] = h + A[300] - 2, with word-array base in $t1 and h in $s2. The byte offset is 300 x 4 = 1,200 = 0x04B0. It fits the signed 16-bit offset field.

Assembly	Field summary	Machine word
lw $t0, 1200($t1)	op 35; rs 9; rt 8; imm 1200	0x8D2804B0
add $t0, $s2, $t0	op 0; rs 18; rt 8; rd 8; fn 32	0x02484020
addi $t0, $t0, -2	op 8; rs 8; rt 8; imm FFFE	0x2108FFFE
sw $t0, 1200($t1)	op 43; rs 9; rt 8; imm 1200	0xAD2804B0

The program loads the old A[300], adds h, subtracts 2, then writes the same array element. Do not accidentally store at element 300 plus another displacement because you have already calculated its byte offset.

Encode the source's label-address exercise

The first instruction is at 0xFC00000C. Every real instruction advances the next address by 4 bytes.

Address	Assembly	Machine word
0xFC00000C	start: add $s0, $s1, $s2	0x02328020
0xFC000010	loop: addi $t0, $t0, -1	0x2108FFFF
0xFC000014	sw $t0, 4($t2)	0xAD480004
0xFC000018	bne $t0, $t3, loop	0x150BFFFD
0xFC00001C	j start	0x0B000003

The branch's reference address is 0xFC00001C. Its target is 0xFC000010, 12 bytes earlier; -12/4 = -3, encoded as FFFD. For the jump, P + 4 is 0xFC000020. Its upper nibble F supplies the high region. The target index is (0xFC00000C >> 2) & 0x03FFFFFF = 0x03000003, giving 0x0B000003 after adding opcode 2.

This is an encoding exercise; it does not claim the sequence eventually terminates. The unconditional jump returns to start, so its behavior depends on the larger surrounding program.

Remember: Assign all addresses first. Calculate branch and jump fields second.

2.28 Far branches and a debugging routine

Source: Chapter2.pdf pp. 71-73; recap of the preceding lessons.

Goal: Recognize a field-size limit and use a repeatable method to check your work.

If a branch target is outside the signed 16-bit displacement range, a single ordinary beq or bne cannot encode it. In the source's simplified model, invert the condition to skip an unconditional jump:

```
# Intended: beq $s0, $s1, FarTarget
bne $s0, $s1, Skip
j   FarTarget
Skip:
```

If the registers differ, skip the jump. If they are equal, fall through to the jump. The jump must itself have a reachable J-format target region. An arbitrary full address may require a register-indirect sequence. Classic delayed MIPS also requires suitable delay-slot instructions, so expansion must respect the execution model.

A checklist for tracing Assembly

1. Record the starting register values and memory bytes or words.
2. Write the role of every register: value, index, displacement, or address.
3. Mark the next instruction. Evaluate its inputs before changing its destination.
4. Update only the locations that instruction writes.
5. After a branch or jump, mark the next instruction explicitly.
6. At a return, check $ra, $sp, and every changed saved register.

A checklist for encoding

1. Expand any pseudoinstructions whose exact words are required.
2. Assign instruction addresses in increments of four bytes.
3. Choose format, opcode, and function from a trustworthy encoding table.
4. Place register numbers in fields according to their roles.
5. Check signed immediate range, alignment, branch reference address, and jump region.
6. Decode your completed word back into Assembly to catch swapped fields.

If a trace is wrong, inspect the earliest row where it diverges. Later wrong values are often consequences of one earlier mistake. For example, if you use an element index as a byte offset, the first load is already reading the wrong location; later arithmetic cannot repair that.

Remember: Reliable Assembly work grows from accurate local steps. You do not need to hold the entire program in your head.

Chapter 3 | Computer arithmetic

3.1 Bits are patterns; an interpretation gives them meaning

Source: Chapter 3 pp. 2-6, 36. Prerequisite: lessons 2.2 and 2.14.

Think of a register as a row of 32 switches. The switches do not say whether they represent an integer, a floating-point number, an address, or an instruction. The operation using them supplies that meaning. This is why an integer addition instruction cannot correctly add two IEEE-754 bit patterns as floating-point values.

A smaller width makes the rules easier to see. With four bits, there are 16 possible patterns. Unsigned interpretation uses all of them for 0 through 15. Signed two's complement uses them for -8 through +7. The pattern 1111 is either unsigned 15 or signed -1; the switches themselves have not changed.

Width	Unsigned range	Signed two's-complement range
4 bits	0 to 15	-8 to +7
8 bits	0 to 255	-128 to +127
32 bits	0 to 4,294,967,295	-2,147,483,648 to +2,147,483,647

For n bits, unsigned range is 0 to 2^n - 1. Signed range is -2^(n-1) to 2^(n-1) - 1. For a signed pattern with its top bit set, subtract 2^n from the unsigned value. For example, four-bit 1101 is unsigned 13, so its signed value is 13 - 16 = -3.

Before an arithmetic question, write down **width, signedness, operation, and destination width**. A mathematically correct answer can fail to fit the chosen storage. We call that overflow. Separately, a floating-point answer can fit the range but still need rounding because too many significant digits are required.

Remember: choose the interpretation before interpreting the top bit. A top bit of 1 means negative only under a signed representation.

3.2 Binary addition and signed overflow

Source: Chapter 3 p. 4.

Binary addition is ordinary column addition with base 2. The tiny rules are 0 + 0 = 0, 0 + 1 = 1, 1 + 1 = 10, and 1 + 1 + 1 = 11. In the last two cases, write the low bit and carry 1 into the next column.

The slide's four-bit example is 7 + 6. The arithmetic sum is 13, but the signed four-bit range ends at 7.

```
  0111    signed +7
+ 0110     signed +6
------
  1101    unsigned 13; signed -3
```

The adder produced the low four bits correctly. They are a wrong signed answer because 13 cannot be represented in four signed bits. **Same-sign inputs giving an opposite-sign result indicate signed addition overflow.**

Input signs	Can signed overflow happen?	What to check
Positive + positive	Yes	Result must not become negative
Negative + negative	Yes	Result must not become nonnegative
Positive + negative	No	The sum lies between the two inputs

Compare two cases. 0111 + 0110 = 1101 overflows signed four-bit storage even though there is no carry out. But 1111 + 0001 = 0000 with a carry out is signed -1 + 1 = 0, which is valid. **Carry out and signed overflow are different tests.** For unsigned addition, a carry beyond the chosen width signals that the exact sum exceeded the range.

On MIPS, add and addi can raise a signed overflow exception. addu and addiu preserve the same low 32 result bits without that signed overflow exception. This instruction choice does not enlarge the destination register.

Remember: first compute the exact mathematical result, then compare it with the destination's range. This method is slower than the sign rule but is an excellent way to verify it.

3.3 Subtraction is addition of a negated operand

Source: Chapter 3 p. 5.

To calculate a - b, the hardware can add a to the two's complement of b. At a fixed width, form the negative pattern by flipping every bit and adding 1. Keep the same width throughout.

For the slide's four-bit 7 - 6, the pattern for +6 is 0110. Flip it to 1001, then add 1 to obtain 1010, representing -6.

```
  0111    +7
+ 1010    -6
------
1 0001    keep four low bits: +1
```

The carry out is discarded here. The signed result +1 is in range, so no signed overflow occurred.

Signed subtraction overflow is possible when the operand signs differ. Think about what subtraction does: a positive value minus a negative value becomes larger; a negative value minus a positive value becomes more negative.

Subtraction	**Overflow warning**
Positive - negative	A negative stored result is a warning
Negative - positive	A nonnegative stored result is a warning
Same-sign operands	No signed subtraction overflow

Example: in four bits, 7 - (-6) should be 13. Its stored pattern is 1101, signed -3, so it overflows. Similarly, -7 - 6 should be -13; the low four bits are 0011, signed +3, also overflow.

MIPS sub checks signed overflow; subu does not. The u form still performs subtraction; it does not force you to change the meanings of all operands. There is no core subi instruction in this subset: adding a suitable negative immediate is a common replacement.

Remember: the most negative signed number has no positive counterpart at the same width. Four-bit -8 can be stored, but +8 cannot. Negating the minimum signed value therefore needs special care.

3.4 What an overflow exception means

Source: Chapter 3 p. 6; local MIPS instruction-set reference. C clarification: [W10].

An exception is a controlled transfer to special handling code because the current instruction encountered a condition that needs attention. It is not a normal procedure call written by the programmer. For an overflowing add, the ordinary result is not committed to its destination; control transfers to an exception handler.

The processor records information such as the cause and a restart-related instruction address. MIPS has an **EPC**, exception program counter, in coprocessor 0. The instruction mfc0 can read a coprocessor-0 register into an ordinary register. Exact exception details depend on the machine and, in classic delayed MIPS, whether the fault occurred in a delay slot.

Instruction family	Signed overflow behavior
add, addi, sub	Can raise a signed overflow exception
addu, addiu, subu	Keep low 32 bits; no signed overflow exception
mult / multu	Produce a wider product in HI and LO
div / divu	Do not provide the automatic checks assumed for add

Correct the slide's spelling addui to **addiu**. Its 16-bit immediate is still sign-extended. For example, addiu $t0, $t0, -1 decrements the low-32-bit value; the immediate does not become +65,535 just because the mnemonic contains u.

Also separate MIPS behavior from C language rules. Standard C unsigned arithmetic wraps modulo its width. Ordinary signed C overflow has undefined behavior; it is not guaranteed to wrap just because the underlying hardware has a non-trapping adder. The slide's statement that C simply ignores overflow is a teaching shortcut, not a reliable rule for writing C.

For this guide's ordinary arithmetic examples, assume exact signed results fit unless overflow is the topic. For pointer adjustments and deliberate low-bit calculations, non-trapping forms are useful. For real error handling, detecting a condition and choosing a recovery policy are separate jobs.

Remember: an exception changes control flow; it is not an extra numeric value written into the result register.

3.5 Multiplication by partial products

Source: Chapter 3 pp. 7-9.

Binary multiplication is easier than decimal multiplication because each multiplier digit is only 0 or 1. For a multiplier bit of 1, include a shifted copy of the multiplicand. For a bit of 0, include zero. Add the included rows.

The slide uses 1000 times 1001: unsigned 8 times 9. From the low bit upward, the multiplier bits are 1, 0, 0, 1.

```
  00001000      8
x    1001      9
----------
  00001000      bit 0: include 8
  00000000      bit 1: include 0
  00000000      bit 2: include 0
+ 01000000      bit 3: include 8 << 3
----------
  01001000      72
```

This is the same computation as 8 x (1 + 8) = 8 + 64. The shift tells you each bit's weight. It does not mean the multiplier must literally be expanded into many Assembly instructions; hardware implements the underlying idea.

Two n-bit unsigned inputs can require **2n product bits**. For example, four-bit 15 x 15 = 225 needs eight bits, 11100001. A four-bit result register would keep only 0001, which is not the complete product.

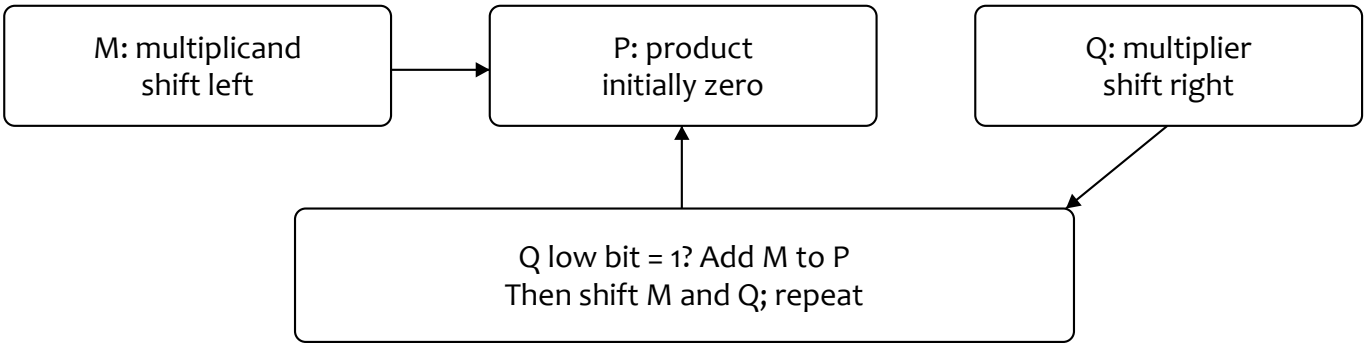
For signed multiplication, signs matter too. You can understand the result by multiplying magnitudes and choosing the sign, although actual circuits may use more efficient signed algorithms. Do not treat a negative two's-complement pattern as an unsigned magnitude without changing the calculation.

Remember: multiplying two 32-bit inputs can require a 64-bit answer. This is the reason MIPS provides HI and LO.

3.6 Shift-and-add multiplication hardware

Source: Chapter 3 pp. 7-10.

Imagine three working boxes: a product accumulator P, a shifted multiplicand M, and a remaining multiplier Q. Initially P = 0. At each iteration, inspect Q's least significant bit. If it is 1, add M to P. Then shift M left and Q right to move to the next multiplier bit.



For the slide's four-bit 2 x 3, use an eight-bit product and multiplicand. The table shows state **after** each complete iteration, including the shifts.

Step	Bit tested	Add to P?	P after add	M after shift	Q after shift
Initial	-	-	00000000	00000010	0011
1	1	Add 2	00000010	00000100	0001
2	1	Add 4	00000110	00001000	0000
3	0	No	00000110	00010000	0000
4	0	No	00000110	00100000	0000

The final product is 00000110, decimal 6. Zero multiplier bits still consume iterations in this simple fixed-iteration algorithm.

The optimized slide keeps the multiplicand in 32 bits and combines the partial product and multiplier in a 64-bit register. It conditionally adds into the upper half, then shifts the combined register right. The multiplier's low bit still controls whether to add. An adder must preserve the necessary carry during the operation.

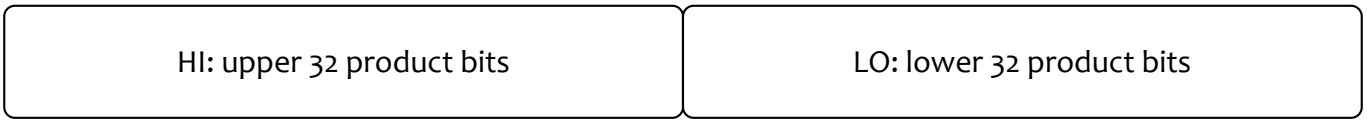
The benefit is smaller hardware: the adder and multiplicand storage can be narrower. It does not automatically mean fewer iterations or faster execution. That is why the slide says optimized in hardware usage, not performance.

Remember: state whether a trace row is before or after shifting. Two correct algorithms can have different-looking intermediate registers.

3.7 HI and LO store a wide multiplication result

Source: Chapter 3 p. 11; Materials/MIPSreferenceCard.pdf.

HI and LO are special result registers, separate from the 32 ordinary integer registers. Think of them as the upper and lower halves of one 64-bit product.



HI:LO is one 64-bit multiplication result

Instruction	Meaning
mult rs, rt	Signed 32 x 32 multiplication; 64-bit result in HI:LO
multu rs, rt	Unsigned multiplication; 64-bit result in HI:LO
mfhi rd	Copy HI into an ordinary register
mflo rd	Copy LO into an ordinary register
mul rd, rs, rt	Keep the low 32 product bits in rd; wider result is not retained there

For a small positive example, 6 x 7 = 42 gives HI = 0x00000000 and LO = 0x0000002A. Copy LO before a later multiply or divide changes the special result registers.

```
mult $s0, $s1
mflo $t0        # low product bits
mfhi $t1        # high product bits
```

For unsigned overflow into more than 32 bits, test whether HI is nonzero. For example, 0xFFFFFFFF x 2 gives HI = 0x00000001 and LO = 0xFFFFFFFE.

For a signed product, **HI must equal the sign extension of LO** for the complete result to fit in a signed 32-bit value. A negative product may correctly have HI = 0xFFFFFFFF. Example: -2 x 3 = -6 gives HI = 0xFFFFFFFF and LO = 0xFFFFFFFA, which fits. Checking only whether HI is zero would wrongly reject it.

mul is a real instruction in some MIPS versions and an assembler convenience in others. It is useful for the slide's factorial code, but consult the selected ISA before machine-encoding it. Do not infer its format from its three written register operands.

Remember: HI is a product's upper half after multiplication; after division, HI instead stores the remainder.

3.8 Division: build the quotient one bit at a time

Source: Chapter 3 p. 12.

Division asks how many copies of the divisor fit in the dividend. The leftover amount is the remainder. For nonnegative integers, the check is **dividend = divisor x quotient + remainder**, with 0 <= remainder < divisor.

For 7 / 2, quotient 3 and remainder 1 satisfy 7 = 2 x 3 + 1. Binary long division reaches this result by bringing down one dividend bit at a time.

Input bit from 0111	Partial remainder after bringing it down	Does 2 fit?	Remainder after step	Quotient prefix
0	0	No: write 0	0	0
1	1	No: write 0	1	00
1	3	Yes: write 1; subtract 2	1	001
1	3	Yes: write 1; subtract 2	1	0011

Bringing down a bit b means new partial remainder = 2 x old remainder + b. The multiplication by 2 is a left shift. The final quotient is 0011 and remainder is 0001.

If the divisor is larger than the dividend, the quotient is zero and the dividend itself is the remainder. For 2 / 7, the answer is quotient 0, remainder 2. Division by zero does not have a valid integer quotient; software must prevent or handle it.

For signed division in this MIPS model, the quotient is truncated toward zero. Start with magnitudes, then apply signs. -7 / 2 gives quotient -3 and remainder -1, because -7 = 2 x (-3) + (-1). A nonzero remainder has the dividend's sign.

Remember: a remainder is not a fractional decimal part. The remainder 1 in 7 / 2 corresponds to the additional fraction 1/2, but it is stored as the integer 1.

3.9 Restoring division and its hardware variants

Source: Chapter 3 pp. 13-15.

Restoring division means trying a subtraction and undoing it if the result is negative. Imagine asking whether a box contains enough money to pay a cost: subtract the cost; if the balance would be negative, restore it and record that the payment did not fit.

One slide implementation starts the divisor in the upper half of a wide register, keeps the dividend in a remainder register, and shifts the divisor right after each trial. For the four-bit 7 / 2 example, the trial divisors are 32, 16, 8, 4, and 2.

Trial divisor	R before trial	Trial R - divisor	Decision	R after decision	Q after shift / bit
32	7	-25	Restore; quotient bit 0	7	0
16	7	-9	Restore; quotient bit 0	7	0
8	7	-1	Restore; quotient bit 0	7	0
4	7	3	Keep; quotient bit 1	3	1
2	3	1	Keep; quotient bit 1	1	3

Here the divisor begins shifted by n bits, so the teaching trace has n + 1 trials. The leading quotient bit is zero. The previous lesson's bring-down method uses n trials. Both reach quotient 3, remainder 1; do not combine one method's initialization with the other method's iteration count.

The optimized design keeps a fixed-width divisor and shifts the combined remainder/quotient register left. For each original dividend bit, bring it into the remainder, subtract the divisor, restore on a negative result, and place 0 or 1 into the quotient's low bit.

The combined storage and narrower subtractor reduce hardware requirements. Signed division can use unsigned magnitude division followed by sign adjustments; the most negative integer and division by -1 need explicit range consideration.

Remember: name what each register currently holds. During division, the remainder register may temporarily contain a trial negative value before restoration.

3.10 MIPS division, quotient, remainder, and checks

Source: Chapter 3 p. 16; Materials/MIPSreferenceCard.pdf.

MIPS core division writes two results: **LO receives the quotient; HI receives the remainder**. Neither is automatically copied to a named ordinary destination register in the two-source form used here.

```
div $s0, $s1
mflo $t0        # quotient
mfhi $t1        # remainder
```

Inputs	Signed quotient in LO	Signed remainder in HI
7 / 2	3	1
-7 / 2	-3	-1
7 / -2	-3	1
-7 / -2	3	-1

Every row satisfies dividend = divisor x quotient + remainder. Truncation toward zero differs from rounding down: -3.5 becomes -3, not -4.

divu interprets both source patterns as unsigned. Thus 0xFFFFFFFF divided by 2 gives quotient 2,147,483,647 and remainder 1, while signed div with the same first pattern calculates -1 / 2, quotient 0 and remainder -1.

Classic MIPS div does not raise the signed overflow or divide-by-zero exceptions that a beginner might expect from add. A zero divisor and the signed minimum divided by -1 require software checks; do not assign a portable useful result to HI/LO for these exceptional operands. Some assemblers expand convenient division spellings into a sequence with checks, so distinguish the actual instruction from its pseudo form.

For a simple nonzero-divisor guard, branch around the division when the divisor equals $zero. What the error path should do depends on the program. Read HI/LO before another operation overwrites them.

Remember: multiply and divide reuse the same special registers but give their halves different meanings.

3.11 Binary fractions and scientific notation

Source: Chapter 3 pp. 17-19.

Floating point is a storage method for values such as 12.75 or very large and very small magnitudes. It resembles decimal scientific notation, but the base is 2. A normal nonzero binary value is written as **sign x 1.fraction x 2^exponent**.

Bits to the right of the binary point have weights 1/2, 1/4, 1/8, 1/16, and so on. Therefore 0.11 in binary is 1/2 + 1/4 = 0.75. The number 1100.11 is 8 + 4 + 0.5 + 0.25 = 12.75.

To convert a fractional decimal part, multiply it by 2 repeatedly. Each resulting integer part gives the next binary bit; continue with the new fractional part.

Fraction before step	Times 2	Next bit	Fraction left
0.75	1.50	1	0.50
0.50	1.00	1	0.00

Normalize 1100.11 by moving the point three places left: 1.10011 x 2^3. Moving the point left makes the exponent positive. Moving it right makes the exponent negative. For example, 0.00101 = 1.01 x 2^-3.

Some decimal fractions never terminate in binary. 0.1 repeats, just as 1/3 repeats in decimal. Finite storage must round such a value. This is a representation limitation, not proof that the computer's adder is broken.

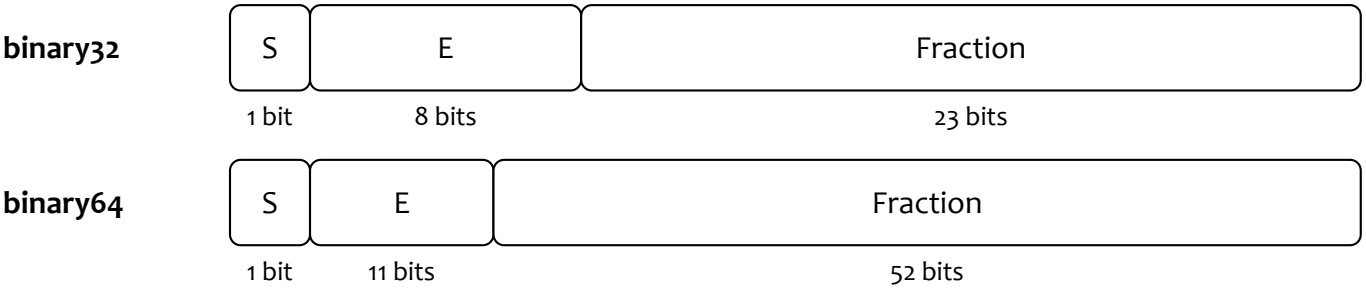
Do not confuse the **significand** 1.10011 with the **fraction field** 10011. The normal leading 1 is understood and need not be stored. The exponent records the scale; the fraction records the significant pattern.

Remember: changing the exponent changes where the binary point belongs, not the stored fraction's positional weights within 1.fraction.

3.12 IEEE-754 normal formats: sign, exponent, fraction

Source: Chapter 3 pp. 19-20; reference card.

IEEE-754 standardizes floating-point representations and arithmetic. For this course, focus on binary32, called single precision, and binary64, called double precision. This lesson handles **normal finite values**; reserved exponent patterns are in lesson 3.15.



Format	Total bits	Sign bits	Exponent bits	Fraction bits	Bias
Single / binary32	32	1	8	23	127
Double / binary64	64	1	11	52	1023

Let S be the sign bit, E the stored unsigned exponent, and F the fraction's numeric value. For a normal number, **value = (-1)^S x (1 + F) x 2^(E - bias)**. The fraction bits have weights 2^-1, 2^-2, and so on. Equivalently, if f is the fraction field interpreted as an integer, F = f / 2^23 for binary32.

The bias lets negative actual exponents be stored using an unsigned field. Actual exponent -3 is stored as -3 + 127 = 124 in single precision. To decode it, subtract the bias: 124 - 127 = -3.

For single-precision 1.0, S = 0, actual exponent = 0, E = 127, and every fraction bit is zero. Its word is 0x3F800000. Do not conclude that an all-zero fraction means the entire value is zero: the hidden leading 1 remains for normal numbers.

Normal single precision has 24 significant binary bits: the 23 stored fraction bits plus the leading 1. Double has 53. Precision counts significant bits; it does not count all 32 or 64 stored bits.

Remember: only use the leading-1 formula after checking that the stored exponent is neither all zero nor all one.

3.13 Decode a floating-point word and understand its range

Source: Chapter 3 pp. 21-23.

The slide asks for the value of 0x414C0000 in single precision. Convert the hex to bits, then split it using field widths, not convenient-looking hex boundaries. The exponent crosses a hexadecimal digit boundary.

```
0x414C0000:
0 | 10000010 | 10011000000000000000000
S     E              fraction
```

1. Sign S = 0, so the value is nonnegative.

2. Stored exponent E = 130, so actual exponent is 130 - 127 = 3.

3. Fraction 10011 followed by zeros means 1/2 + 1/16 + 1/32 = 0.59375.

4. Restore the leading 1: significand = 1.59375.

5. Apply the scale: 1.59375 x 2^3 = **12.75**.

For normal binary32, E runs from 1 through 254, so actual exponents run from -126 through +127. The smallest positive normal is 2^-126, about 1.175 x 10^-38. The largest finite is (2 - 2^-23) x 2^127, about 3.403 x 10^38. These are magnitude limits, not a promise to represent every real number between them.

Normal binary64 has actual exponents -1022 through +1023, smallest positive normal 2^-1022, and largest finite (2 - 2^-52) x 2^1023. Double offers both wider range and finer precision.

The slide's "smallest value" means **smallest positive normal magnitude**. Subnormal values extend below it; negative values obviously have a different ordering. Always read the wording of a range question carefully.

Remember: decode the exponent as unsigned, then subtract the bias. It is not a two's-complement exponent field.

3.14 Encode a decimal number, including rounding

Source: Chapter 3 pp. 24-25.

Encoding reverses decoding. First find the sign, then convert the magnitude to binary, normalize it, bias the exponent, and store the fraction after the leading 1. Keep extra bits until the rounding decision.

For 12.75, integer 12 is 1100 and fraction 0.75 is .11. Thus 12.75 = 1100.11 = 1.10011 x 2^3.

Field	Calculation	Stored result
Sign	Positive	0
Exponent	3 + 127 = 130	10000010
Fraction	Remove leading 1; pad to 23 bits	10011000000000000000000

Join the fields and group the final 32 bits into nibbles. The result is **0x414C0000**. For -12.75, only the sign bit changes, producing 0xC14C0000.

The slide also asks for 6.3. Its exact decimal fraction repeats in binary: 6.3 = 110.0100110011... = 1.100100110011... x 2^2. Therefore S = 0 and E = 129. The first 23 fraction bits are 10010011001100110011001. The following bit is 1 and later discarded bits contain another 1, so round upward under round-to-nearest.

```
Before rounding fraction: 10010011001100110011001
After rounding fraction:  10010011001100110011010
Sign / exponent:          0 / 10000001
Final single word:        0x40C9999A
```

This stored value is approximately 6.300000190734863, not exactly 6.3. Encoding a decimal number as binary32 can change its exact value slightly. If rounding carries out of an all-one fraction, renormalize and increase the exponent.

Remember: never simply cut the first 23 fraction bits unless the problem requests truncation. IEEE default rounding considers the discarded tail.

3.15 Reserved patterns, rounding modes, and extra bits

Source: Chapter 3 pp. 22-23, 35-36. Special-value explanation supplements the reserved-pattern slides; checked with [W8] and [W9].

The exponent's all-zero and all-one patterns have special meanings. Their purpose includes representing zero and continuing gradually toward very small magnitudes instead of abruptly losing every value below the normal range.

Binary32 exponent E	Fraction field	Meaning
0	All zero	+0 or -0, selected by sign
0	Nonzero	Subnormal: (-1)^S x (f / 2^23) x 2^-126
1 to 254	Any	Normal: (-1)^S x (1 + f / 2^23) x 2^(E-127)
255	All zero	+infinity or -infinity
255	Nonzero	NaN: not a number

The smallest positive binary32 subnormal is 2^-149, stored as 0x00000001. It has no hidden leading 1. Examples: 0x00000000 is +0, 0x80000000 is -0, 0x7F800000 is +infinity, and 0x7FC00000 is one NaN pattern.

The usual default is **round to nearest, ties to even**. Choose the nearest representable result; if exactly halfway, choose the one whose retained least significant bit is 0. Other modes round toward zero, toward positive infinity, or toward negative infinity.

Hardware can keep a guard bit G, a round bit R, and a sticky bit S after the retained significand. Sticky is the OR of all further discarded bits. For nearest-even, increment the retained significand when G = 1 and either R = 1, sticky = 1, or the retained low bit = 1. G = 1 with R = sticky = 0 is exactly a halfway case.

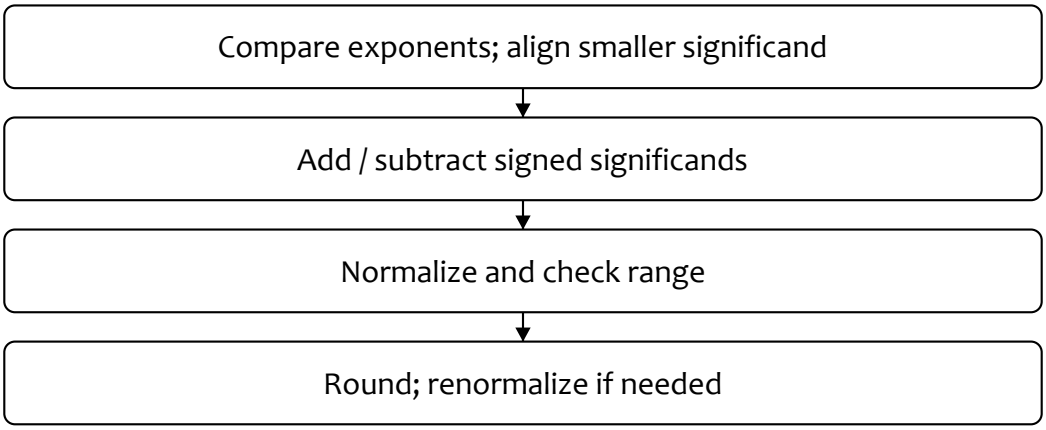
Tiny toy example: keep four significant binary bits. 1.0101000... is halfway between 1.010 and 1.011, so choose 1.010, whose last bit is even. 1.0111000... is halfway and rounds to 1.100.

Remember: range, precision, and rounding are different ideas. A finite in-range computation can still be inexact.

3.16 Floating-point addition: align before adding

Source: Chapter 3 pp. 26-28.

You cannot add significands until their exponents agree. In decimal, 1.2 x 10^3 and 3.0 x 10^1 use different place values. Binary floating-point addition has the same issue. Shift the smaller-exponent significand right, keeping extra rounding bits, until both scales match.



Follow the slide's binary example: 0.5 + (-0.4375). These are 1.000 x 2^-1 and -1.110 x 2^-2, using a toy four-significant-bit format.

Step	What changes	Result
Align	Raise second exponent from -2 to -1; shift its significand right	-0.111 x 2^-1
Add signed significands	1.000 + (-0.111)	0.001 x 2^-1
Normalize	Move point three places right; lower exponent by 3	1.000 x 2^-4
Round	No nonzero discarded bits here	0.0625

Cancellation can require a large left shift after adding opposite signs. Adding two positive significands may instead produce 10.xxx and require a right shift with an exponent increase. Exact cancellation produces zero, which uses the special zero representation.

The decimal slide example 9.999 x 10^1 + 1.610 x 10^-1 demonstrates the same recipe. With sufficient intermediate digits, the exact sum is 100.151; rounding to four significant decimal digits gives 100.2, or 1.002 x 10^2. Its early 0.016 display is rounded shorthand; keep the additional digits when checking the result.

A hardware diagram may have separate pieces for exponent comparison, significand alignment, addition/subtraction, normalization, and rounding. Read it by identifying which of these four jobs each block performs.

Remember: shift the smaller-exponent operand's significand, not its entire encoded IEEE word. The sign/exponent/fraction fields have separate roles.

3.17 Floating-point multiplication: combine signs and scales

Source: Chapter 3 pp. 29-30.

For normal floating-point multiplication, multiply the significands and add the actual exponents. The result sign is negative exactly when the input signs differ. Then normalize and round, just as after addition.

For the slide's example, 0.5 x (-0.4375) = (1.000 x 2^-1) x (-1.110 x 2^-2).

1. Actual exponents add to -1 + (-2) = -3.

2. Significands multiply to 1.000 x 1.110 = 1.110.

3. The significand is already in [1, 2), so no scale adjustment is needed.

4. The toy result is exact, so rounding does not change it.

5. Opposite signs make the result negative: -1.110 x 2^-3 = **-0.21875**.

With stored biased exponents, adding E1 and E2 counts the bias twice. For single precision, the candidate stored exponent is **E1 + E2 - 127**, followed by any normalization adjustment. Here 126 + 125 - 127 = 124, representing actual exponent -3.

Another example: 1.5 x 1.5 = 1.1 x 1.1 = 10.01 in binary. Normalize to 1.001 x 2^1, decimal 2.25. The extra leading product bit increases the exponent by 1.

The decimal slide example works similarly: (1.110 x 10^10)(9.200 x 10^-5) = 10.212 x 10^5 = 1.0212 x 10^6. Four-significant-digit rounding gives 1.021 x 10^6.

At the extremes, a result may overflow to an infinity or become subnormal/zero according to rounding and exception settings. These are floating-point conditions, not the signed integer overflow test from lesson 3.2.

Remember: add exponents when multiplying; align exponents when adding. Mixing these two rules is a common source of wrong answers.

3.18 Floating-point registers and MIPS operations

Source: Chapter 3 pp. 31-32; Materials/MIPSreferenceCard.pdf.

MIPS floating-point operations use coprocessor 1, often called the FPU, and registers $f0 through $f31. They are distinct from $s0, $t0, and the other integer registers. Writing $f0 does not change $zero or $s0.

Job	Single precision	Double precision
Read memory	lwc1	ldc1
Write memory	swc1	sdc1
Arithmetic	add.s, sub.s, mul.s, div.s	add.d, sub.d, mul.d, div.d
Compare	c.eq.s, c.lt.s, c.le.s	c.eq.d, c.lt.d, c.le.d
Branch on FP condition	bc1t / bc1f	bc1t / bc1f

In the course's classic 32-bit FP-register model, a double uses an even/odd pair such as $f4/$f5; use the even register as its named operand. A single occupies one register. Wider modern FP-register configurations exist, so follow the course's stated model.

lwc1 $f0, 0($s0) computes a memory address using integer $s0 and copies 32 bits into FP $f0. It does **not** convert an integer value in memory into a float. If those bits encode 7.5, the float operation sees 7.5; as an unsigned integer, 0x40F00000 would instead be 1,089,470,464.

Similarly, mtc1 moves bits from an integer register to an FP register; conversion needs a conversion instruction such as cvt.s.w. This supplementary distinction helps explain the slide's "bits have no inherent meaning" conclusion.

An FP comparison sets a condition bit rather than writing 0 or 1 into an integer register. bc1t branches when it is true; bc1f branches when it is false. Ordered less-than is false if a NaN participates, so FP comparison is not always identical to integer comparison.

Remember: .s means single, not signed. .d means double, not an integer destination.

3.19 Trace an FP if / else and Fahrenheit conversion

Source: Chapter 3 pp. 32-33. No delay slots in the following teaching code.

Let a be in $f0 and b in $f1. To implement if (a < b) a = a + b; else a = a - b;, perform the FP comparison, branch on its condition, and prevent the first path from falling into the second.

```
c.lt.s $f0, $f1
bc1t AddPath
sub.s $f0, $f0, $f1
j Done
AddPath: add.s $f0, $f0, $f1
Done:
```

For a = 2.0 and b = 3.0, the comparison is true; only add.s executes and a becomes 5.0. For a = 4.0 and b = 3.0, the comparison is false; sub.s executes and a becomes 1.0. Compare the old inputs before overwriting a.

The Fahrenheit function computes (5.0 / 9.0) x (fahr - 32.0). Suppose the constants are stored at byte offsets 0, 4, and 8 from $gp, the input is $f12, and the result is $f0.

```
f2c:
   lwc1 $f16, 0($gp)    # float 5.0
   lwc1 $f18, 4($gp)    # float 9.0
   div.s $f16, $f16, $f18
   lwc1 $f18, 8($gp)    # float 32.0
   sub.s $f18, $f12, $f18
   mul.s $f0, $f16, $f18
   jr $ra
```

For fahr = 212, subtraction gives 180.0 and multiplication gives approximately 100.0 under the chosen FP arithmetic. The coefficient 5/9 is not exactly representable, so reason about rounding rather than promising every intermediate is exact. For fahr = 32, the difference is zero and the result is zero.

This is a leaf function. It does not issue another jal, so its incoming $ra survives. Its temporary FP-register choices follow the classroom fragment; a full platform ABI may impose additional preservation rules.

Remember: memory must contain floating-point encodings of 5.0, 9.0, and 32.0. Integer words 5, 9, and 32 are different bit patterns.

3.20 FP machine fields and an arithmetic problem checklist

Source: Chapter 3 pp. 34-36; Materials/MIPSreferenceCard.pdf.

Floating-point instructions share the 32-bit instruction length but use fields with FP-specific meanings. For an FP arithmetic instruction, the fields are opcode, fmt, ft, fs, fd, funct. The field widths are 6, 5, 5, 5, 5, 6, respectively.

Field	Meaning for add.s $f2, $f4, $f6	Decimal value
opcode	COP1 arithmetic group	17
fmt	Single precision	16
ft	Second FP source $f6	6
fs	First FP source $f4	4
fd	FP destination $f2	2
funct	FP addition	0

Joining these fields gives **0x46062080**. Double uses fmt = 17 instead of 16. Arithmetic funct values in this subset are add 0, sub 1, mul 2, div 3. The FP opcode is decimal 17, hex 0x11; this is distinct from the fmt field's decimal value 16.

FP loads/stores use an I-like form with an integer base register, an FP register number, and a signed byte offset. lwc1 $f2, 100($s4) uses opcode decimal 49, rs = 20, rt = 2, immediate = 100, giving 0xC6820064.

Do not reuse ordinary R-format names mechanically: the field position called rd in integer arithmetic is fs in this FP layout, and the ordinary shamt position is fd. FP branch and comparison formats also have their own rules; use the supplied card when encoding them.

For any arithmetic problem, work through this sequence:

1. Identify whether the bits are signed integers, unsigned integers, or IEEE values.

2. State widths and destination: one register, HI:LO, or FP fields.

3. Apply the correct numeric operation and range or precision checks.

4. For FP, classify the exponent before restoring a hidden bit.

5. Keep enough intermediate bits to round once at the specified precision.

6. For Assembly, check which registers are inputs and which locations change.

Remember: machine encoding describes the instruction to execute; IEEE encoding describes a floating-point data value. They are two different kinds of 32-bit words.

Chapter 4 | Microarchitecture: datapath, control, and pipelining

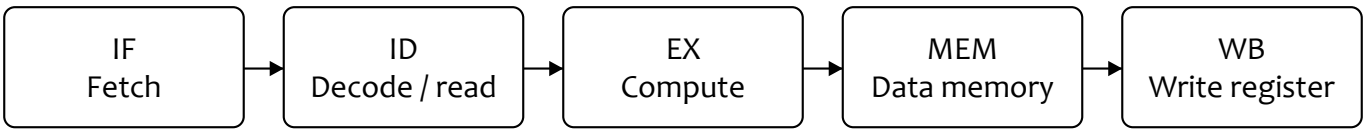
4.1 One instruction, five kinds of work

Source: Chapter 4 pp. 2-5. Prerequisites: Chapter 2 instruction roles and Chapter 1 timing.

The ISA tells the programmer what an instruction does. Microarchitecture tells us how circuits achieve it. The same MIPS lw can run on a single-cycle processor or a pipelined processor while keeping the same visible meaning: calculate an address and copy a word into a register.

The course uses five stages. In a single-cycle machine, their necessary work happens within one long cycle. In a pipeline, different instructions can occupy different stages at the same time.

Stage	Full name	Main question
IF	Instruction fetch	Which instruction is at the PC address?
ID	Instruction decode / register read	What operation and operands are requested?
EX	Execute	What is the arithmetic result or effective address?
MEM	Data memory access	Is a load or store needed?
WB	Write back	Which result should enter which integer register?



One instruction visits the stages; many instructions can overlap

For lw $t0, 12($s0), IF retrieves its instruction bits; ID reads $s0 and prepares 12; EX adds base + 12; MEM reads the addressed word; WB writes $t0. For add, MEM performs no useful data-memory access. For sw, WB performs no register update.

Passing through a pipeline stage does not mean using every circuit in it. This distinction matters when drawing pipeline diagrams: the store may still have a final WB column, but that column does not enable a register write.

Instruction count depends on the program, compiler, and ISA. Hardware affects CPI and clock period. A design with CPI = 1 can still be slower than another design if its one cycle is much longer.

Remember: an instruction's meaning and the number of physical cycles used to implement it are separate ideas.

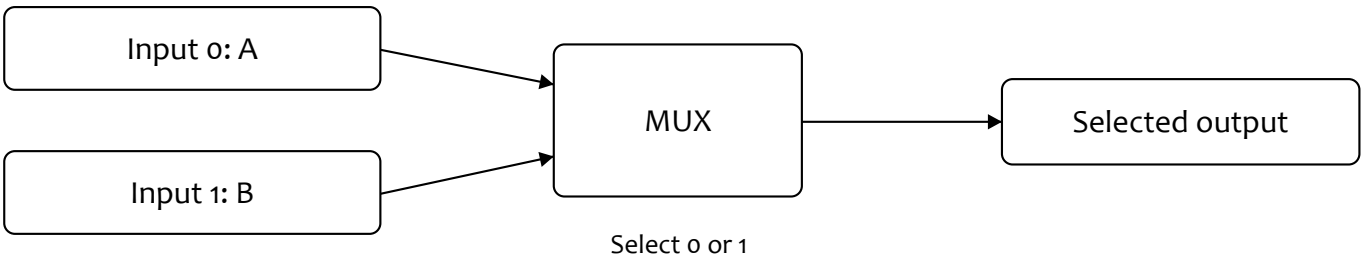
4.2 Read a datapath as boxes, routes, and switches

Source: Chapter 4 pp. 5-9, 15.

A datapath diagram is a map of where information can travel. Start with one instruction, then follow only its active route. Trying to memorize every wire at once creates unnecessary work.

Box	What it holds or does
PC	Holds an instruction's byte address
Instruction memory	Returns the instruction stored at that address
Register file	Reads named registers; writes a result when enabled
ALU	Adds, subtracts, compares, and performs bitwise operations
Data memory	Reads or writes program data
Sign extender	Widens a signed 16-bit value to 32 bits
Adder / shifter	Builds PC + 4 and branch addresses
Multiplexer, or MUX	Chooses one of several candidate inputs
Control unit	Produces selection and enable signals

A two-input MUX is a switch, not an arithmetic operator. In the slide's convention, select 0 chooses input A, select 1 chooses input B. If A = 7 and B = 12, output is either 7 or 12; it does not become 19.



Data wires carry values or bit patterns. Control wires say which path to choose and whether to update storage. RegWrite = 1 allows a register write; it is not the numeric value to write. ALUSrc = 1 selects an immediate input; it does not itself calculate the immediate.

Combinational boxes respond to current inputs. State elements, such as the PC and pipeline registers, preserve information across clock boundaries. Read the current state, compute the next values, then update the state at the appropriate clock event.

Remember: first name the box's job, then identify its inputs and output. Only after that should you trace a wire.

4.3 IF and ID: fetch bits, extract fields, read values

Source: Chapter 4 pp. 10-11, 16-19.

Suppose add $s0, $s1, $s2 is stored at byte address 1000. Its instruction word is 0x02328020. The PC supplies 1000 to instruction memory, which returns that word. A separate adder computes 1004, the normal sequential address.

ID extracts register numbers from fixed bit positions. For this add, rs is 17, rt is 18, and rd is 16. These are **numbers naming registers**, not the data inside them.

Extracted field	Bit positions	Meaning for this add
opcode	31:26	0, chosen core R-format group
rs	25:21	17, read $s1
rt	20:16	18, read $s2
rd	15:11	16, candidate destination $s0
funct	5:0	32, addition

If $s1 contains 34 and $s2 contains 36, the register file's two read outputs are 34 and 36. Reading register number 17 does not produce 17 unless that register happens to contain it.

Hardware may compute outputs that the current instruction ignores. For this R-format word, the low 16 instruction bits are 0x8020, so a continuously connected sign extender could output 0xFFFF8020. That value is irrelevant because this add selects register operands. An unused wire can have a value without affecting the instruction.

The PC is byte-addressed: adjacent ordinary instructions differ by 4. The phrase "word address" in the slide's example should not make you divide the PC by 4 when feeding instruction memory.

Remember: distinguish a field's bits, the register number they encode, and the register's stored value. These are three different things.

4.4 EX and MEM: calculate first, access memory second

Source: Chapter 4 pp. 12-13.

The ALU is reused for several jobs. The control signals decide which operands it receives and which operation it performs.

Instruction	First ALU input	Second ALU input	Operation	Meaning of output
add $t0,$t1,$t2	Value in $t1	Value in $t2	Add	Arithmetic result
lw $t0,12($s0)	Base value in $s0	Sign-extended 12	Add	Byte address
sw $t0,12($s0)	Base value in $s0	Sign-extended 12	Add	Byte address
beq $t0,$t1,L	Value in $t0	Value in $t1	Subtract	Zero indicates equality

For a load with $s0 = 100, EX computes address 112. If memory at 112 contains 900, the ALU result is 112 while the memory read result is 900. These values travel on different wires and must not be confused.

For a store, suppose $t0 = 55. EX still computes 112, but MEM writes 55 at that address. Store data comes from the second register read output, even though the ALU's second input is the immediate 12. Therefore the register-read value must be preserved on a separate route.

For beq, subtracting equal values gives zero. The ALU's Zero output is a Boolean signal: 1 for a zero result, 0 otherwise. A separate branch-target route adds PC + 4 to the shifted signed displacement. The subtraction result itself is not the target address.

An ordinary R-format operation does not need data memory. Do not interpret every ALU result as a memory address. The same physical output wire has different meaning depending on the instruction class.

Remember: ALU result can be an answer, an address, or part of a comparison. Name its meaning before using it.

4.5 WB and the main control signals

Source: Chapter 4 pp. 14, 17-21, 44.

Write back is the final choice of a destination register and a value. R-format arithmetic writes rd using the ALU result. A load writes rt using memory's read data. A store and a branch do not write an ordinary result register.

Use the MUX numbering in this guide and the slide diagrams. X means either choice is harmless because the relevant result is unused; it is not a third electrical value.

Instruction	RegDst	ALUSrc	MemtoReg	RegWrite
R-format arithmetic	1: rd	0: register	0: ALU	1
lw	0: rt	1: immediate	1: memory	1
sw	X	1: immediate	X	0
beq	X	0: register	X	0
addi, if added to subset	0: rt	1: immediate	0: ALU	1

Instruction	MemRead	MemWrite	Branch	ALUOp
R-format arithmetic	0	0	0	10
lw	1	0	0	00
sw	0	1	0	00
beq	0	0	1	01
addi	0	0	0	00

Branch says this instruction is a branch. PCSrc says whether its condition actually selects the target. For this beq-only branch path, PCSrc = Branch AND Zero. A not-taken beq has Branch = 1 but PCSrc = 0.

If another diagram reverses a MUX's input numbering, its select bits may reverse too. The semantic statement "choose the memory result" is more fundamental than memorizing an isolated 1.

Remember: enables prevent unintended updates. A harmless-looking unused memory read is different from an accidental memory write.

4.6 ALUOp and ALU control are two levels of decoding

Source: Chapter 4 p. 20.

The main controller looks at the opcode and gives a broad instruction-class request. The ALU controller turns that request, and sometimes the funct field, into the exact ALU action.

The course calls the broad two-bit request **ALUOp**. It calls the final four-bit action **ALU operation**. They are related signals with different widths, not two names for the same bits.

ALUOp	Instruction class	ALU controller action
00	Loads / stores / addi	Add, regardless of low funct-like bits
01	beq	Subtract to test equality
10	Core R-format	Decode the actual funct field

R-format funct	Instruction	Four-bit ALU operation
0x20	add	0010
0x22	sub	0110
0x24	and	0000
0x25	or	0001
0x2A	slt	0111

For lw, the immediate's low bits may resemble an arithmetic funct. Ignore that resemblance: ALUOp = 00 already requests addition. Only an R-format request uses funct in this simplified design.

For sub, main control outputs ALUOp = 10. The funct 0x22 then selects final ALU operation 0110. For beq, ALUOp = 01 selects the same subtraction action through a different decoding route.

These binary control encodings are a design convention for the depicted circuit. They are not the MIPS ISA's arithmetic opcode values. For example, ALU operation 0010 is not the opcode field for add.

Remember: ask whether a question wants the instruction opcode, ALUOp, or the final ALU operation. Write the signal's name beside its bits.

4.7 Worked functional-unit and control traces

Source: Chapter 4 p. 22.

The slide says registers initially contain twice their register numbers. Keep $zero = 0 as required by the ISA. In the following table, **each instruction is an independent experiment**, starting from that initial state. Memory word at byte address 136 initially holds 2021.

Instruction	Read values / immediate	ALU result	Architectural update
add $t0,$t1,$t2	$t1=18; $t2=20	38	$to becomes 38
addi $s0,$s1,100	$s1=34; imm=100	134	$s0 becomes 134
lw $s0,100($s2)	$s2=36; imm=100	Address 136	$s0 becomes 2021
sw $s0,100($s2)	Base 36; data $s0=32	Address 136	Memory[136] becomes 32
beq $s1,$s0,L1	34 and 32	Difference 2; Zero=0	Continue at PC + 4

For the load, read register 1 is number 18, while its content is 36. The destination is register 16, $s0. ALUSrc = 1, MemRead = 1, MemtoReg = 1, RegWrite = 1. The selected write-back data is 2021, not the address 136.

For the store, read register 2 is number 16 and its content is 32. MemWrite = 1 while RegWrite = 0. Reading the second register does not imply that its value is selected as the second ALU input.

For the branch, Branch = 1, ALUOp = 01, final ALU operation = 0110, and PCSrc = 0 because Zero = 0. Its target adder can still produce a candidate target, but that candidate is not selected.

If these instructions were executed in sequence, later rows would use updated state. In particular, the store after the load would write 2021, not 32. The first step of any trace is therefore to establish independent versus sequential execution.

Remember: functional units can calculate unused outputs. Report which selected values actually update architectural state.

4.8 The next-PC paths: sequential, branch, and jump

Source: Chapter 4 pp. 21, 23-24; reference-card correction for the PC source.

The processor must choose one next instruction address. Sequential execution selects PC + 4. A taken branch selects a PC-relative target. A jump selects a reconstructed target. Adding another choice requires a selection path and control information.

For an instruction stored at byte address P:

Sequential:	P + 4
Taken beq:	P + 4 + 4 x sign_extended_immediate
j / jal target:	{(P + 4)[31:28], index26, 00}

The branch displacement counts instruction words; the load/store displacement counts bytes. Example: a branch at 1000 with immediate 3 targets 1004 + 12 = 1016. A load at any PC with offset 3 accesses base + 3 bytes.

The Chapter 4 slide describes the jump prefix as the old PC's upper four bits. The precise rule is **the upper four bits of P + 4**. Usually both prefixes agree, which hides the error. At P = 0x8FFFFFFC, P + 4 = 0x90000000, so the prefix must be 9 rather than 8.

To jump to 0x91234560 from a region with prefix 8, a single j is insufficient. A register jump can use the full address built with lui and ori, as explained in lesson 2.26. A jr extension would add yet another next-PC candidate from a register.

This single-cycle educational diagram includes j but does not implement every instruction in the larger ISA. Do not assume its existing destination and write-back MUXes already support jal, HI/LO, or FP instructions.

Remember: a datapath supports exactly the instruction paths and control logic actually provided. An ISA mnemonic alone does not create a missing circuit.

4.9 Why a single-cycle clock waits for the slowest instruction

Source: Chapter 4 pp. 25, 28-29.

A single-cycle machine completes one instruction per cycle, but all instruction types use the same clock period. The clock must be long enough for the slowest required path to finish before state updates.

The slide gives IF = 200 ps, ID = 100 ps, EX = 200 ps, MEM = 200 ps, WB = 100 ps. Ignore extra wiring and clock-register overhead for this simplified comparison.

Instruction	Work on its main path	Path delay
lw	IF + ID + EX + MEM + WB	800 ps
sw	IF + ID + EX + MEM	700 ps
R-format	IF + ID + EX + WB	600 ps
beq	IF + ID + EX, in this model	500 ps

The load is longest, so the common clock period is **800 ps**. Even an add waits until the 800 ps cycle ends. Its 600 ps path does not automatically produce a separate 600 ps clock cycle.

For three instructions, single-cycle execution takes 3 x 800 = 2400 ps. For N instructions, time = N x 800 ps. The clock frequency is 1 / (800 x 10^-12 s) = 1.25 GHz.

When solving a real detailed circuit-delay question, include the MUXes, sign extender, adders, register timing, and the actual longest path. Do not sum every box in the drawing indiscriminately: parallel paths do not all occur one after another.

Remember: CPI = 1 is not a universal guarantee of high speed. Instruction time also depends on seconds per cycle.

4.10 Pipelining overlaps work; it does not make a task disappear

Source: Chapter 4 pp. 26-30.

Imagine laundry with separate wash, dry, fold, and put-away stations. While load 1 is drying, load 2 can wash. Each load still visits all stations, but several loads progress together. Pipelining applies the same idea to instructions.

The key improvement is **throughput**: after filling, a well-behaved pipeline can complete one instruction each cycle. The first instruction must still cross all five stages.

Property	Single-cycle example	Five-stage pipeline example
Clock period	800 ps	200 ps, ignoring overhead
First instruction completion	800 ps	5 x 200 = 1000 ps
Later completions in ideal flow	Every 800 ps	Every 200 ps

The pipelined first instruction has greater latency in this unbalanced example. That does not contradict the faster throughput. A conveyor can deliver more boxes per minute even when an individual box spends longer on the conveyor.

The slowest stage sets the pipeline clock: max(200,100,200,200,100) = 200 ps. A 100 ps stage finishes early and waits at its boundary. Pipeline registers preserve completed values until the next cycle.

The textbook slogan "speedup equals the number of stages" assumes balanced stages, many instructions, negligible overhead, and no hazards. This slide's stage delays are not balanced: its ideal long-run speedup is 800/200 = 4, not 5.

Remember: distinguish first-result latency, total time for a finite group, and steady-state instruction spacing.

4.11 Count ideal pipeline cycles and compute time

Source: Chapter 4 pp. 28-30, 35-36.

With k stages and N instructions, an ideal pipeline needs **N + k - 1 cycles** to fill, process the group, and drain. The first instruction takes k cycles; each remaining instruction finishes one cycle later.

For three instructions and five stages:

Instruction	**1**	**2**	**3**	**4**	**5**	**6**	**7**
I1	IF	ID	EX	MEM	WB	-	-
I2	-	IF	ID	EX	MEM	WB	-
I3	-	-	IF	ID	EX	MEM	WB

There are 7 cycles, not 15, because stages overlap. With a 200 ps pipeline clock, time is 7 x 200 = 1400 ps. Compare the single-cycle time 2400 ps: finite-group speedup = 2400 / 1400, about 1.714.

For 100 instructions, ideal cycles = 104; time = 20,800 ps. Single-cycle time = 80,000 ps, giving about 3.846 times faster. The fill/drain overhead becomes a smaller fraction as N grows.

For stage delays d1 through dk and per-stage register overhead h, pipeline period is commonly modeled as max(di) + h. For [200,100,200,200,100] ps and h = 20 ps, it is 220 ps. Do not add h five times into the period: each stage boundary's overhead is already accounted for in one stage's cycle constraint.

The cycle formula assumes no stalls, misses, or wrong-path penalties. Add their actual extra cycles only after stating the machine's behavior. An average CPI model describes long streams and may omit the initial four-cycle fill/drain cost.

Remember: cycles are a count; time is that count multiplied by the relevant clock period.

4.12 Read a pipeline diagram in two directions

Source: Chapter 4 pp. 31-37, 45.

Read a row to follow one instruction through time. Read a column to see which different instructions use the machine in one cycle. These are different questions and lead to different control values.

The slide sequence is:

```
I1: lw  $s0, 20($s1)
I2: sub $t2, $s2, $s3
I3: add $t3, $s3, $s4
I4: lw  $t4, 24($s1)
I5: add $t5, $s5, $s6
```

These instructions do not create a result dependency on one another in the shown fragment, so use the ideal timing first.

	1	2	3	4	5	6	7	8	9
I1	IF	ID	EX	MEM	WB				
I2		IF	ID	EX	MEM	WB			
I3			IF	ID	EX	MEM	WB		
I4				IF	ID	EX	MEM	WB	
I5					IF	ID	EX	MEM	WB

In cycle 5, I1 is in WB, I2 in MEM, I3 in EX, I4 in ID, and I5 in IF. This cycle has an add executing and a load decoding and an earlier load writing back, simultaneously.

Unit / stage in cycle 5	**Instruction controlling it**	**Relevant action**
Instruction memory / IF	I5 add	Fetch I5 bits
Decode / ID	I4 lw	Read $s1; decode load controls
ALU / EX	I3 add	ALUSrc=0; final operation 0010
Data memory / MEM	I2 sub	MemRead=0; MemWrite=0
Register write / WB	I1 lw	RegWrite=1; MemtoReg=1; write $s0

The control block's newly generated bits belong to I4, currently in ID. They do not immediately become the current data-memory controls for I2 or the current WB controls for I1. Each older instruction carries its earlier-decoded controls forward.

Remember: label every requested signal with its stage's instruction before assigning 0 or 1.

4.13 Pipeline registers preserve each instruction's information

Source: Chapter 4 pp. 32-43.

The slide's wholesale-market barriers represent clocked pipeline registers. Fast and slow workers cannot independently overwrite shared information whenever they finish; values cross a boundary at the controlled clock event.

Boundary register	Typical information to preserve
IF/ID	Instruction bits; that instruction's PC + 4
ID/EX	Read operands, immediate, source numbers, destination candidates, controls
EX/MEM	ALU result, store data, chosen destination number, later controls
MEM/WB	Memory result, ALU result, destination number, WB controls

Think of each instruction as a parcel carrying a value and a destination label. If the label is lost, the right value could be delivered to the wrong register.

The original diagram in slide 37 raises this problem. A load in WB must write its own rt destination, not the rt bits of a younger instruction now in ID. The corrected diagram carries the destination number across the pipeline.

Example: an earlier load targets $s0, number 16, while a younger load in ID targets $t4, number 12. If WB accidentally uses the younger rt, the older load's data would corrupt $t4. Preserving destination 16 through ID/EX, EX/MEM, and MEM/WB prevents that error.

Preserve other instruction-specific state too. Branch computation needs the correct instruction's PC + 4 and displacement. A store needs its own data value, even if later instructions change that register. Passing values forward is how the implementation separates overlapping work.

Remember: a pipeline register is not the programmer's register file. IF/ID and EX/MEM are hidden implementation state, not registers you name in Assembly.

4.14 Controls travel with the instruction

Source: Chapter 4 pp. 44-46.

Decode produces a bundle of controls, then the bundle moves along with the instruction's data. A stage uses its part of the bundle; later controls continue onward. The live opcode currently in ID cannot safely control every older stage.

Control group	Signals in the course model	Stage that uses them
EX	RegDst, ALUSrc, ALUOp	Select destination / inputs / ALU action
M	Branch, MemRead, MemWrite	Next-PC decision and data-memory action
WB	RegWrite, MemtoReg	Choose write-back data and allow update

For lw, EX adds base + immediate and selects rt; MEM reads data; WB selects the memory result and writes rt. Its RegWrite bit must remain associated with that load until WB.

Revisit cycle 5 from lesson 4.12. The main control block decodes I4's lw and outputs ALUSrc=1, ALUOp=00, MemRead=1, MemWrite=0, MemtoReg=1, RegWrite=1, RegDst=0, Branch=0. Meanwhile the current ALU acts on I3's add, so its ALUSrc=0 and ALUOp=10. Both answers are correct because they refer to different instructions and stages.

A bubble is an empty piece of work inserted into this flow. Setting its state-changing control bits to zero makes it harmless: no register write, no memory write, and no taken branch. Data bits in a bubble can contain leftovers; disabled controls prevent them from changing architectural state.

Remember: when a question says "control signals in cycle 5," separate the functional units' current controls from the ID control block's new outputs.

4.15 Three hazard types and structural conflicts

Source: Chapter 4 pp. 47-50.

A hazard is a reason the ideal overlapping schedule cannot proceed correctly as planned. It is not automatically a programmer error; the hardware and compiler must preserve the ISA's sequential meaning.

Hazard	Intuitive problem	Example
Structural	Two instructions need one busy resource	IF and a load both request a single shared memory port
Data	A needed value is not yet available	add immediately reads a preceding load's destination
Control	The next address is not yet known	Instructions are fetched before a branch resolves

In a structural conflict, no amount of register-value forwarding creates a second memory port. Either delay one request or provide more resources. Separate instruction and data memories/caches allow IF and MEM accesses in the same cycle in the classroom pipeline.

The register file also needs appropriate read and write ports. In this guide, two operand reads are supported, and a WB write can occur in the same cycle as an ID read. Assume WB writes early enough for ID to see the new value that cycle when stated.

The slide's claim that MIPS has solved all structural hazards applies to this specific simplified implementation and its resources. It is not a universal guarantee for every MIPS processor or every multi-cycle FP unit.

For each possible hazard, ask: which instructions conflict, which stage needs the resource or value, and in which cycle is it available? This identifies whether a stall is needed and which mechanism can fix it.

Remember: a dependency is a property of the program; whether it becomes a timing hazard depends on the implementation.

4.16 RAW dependencies without forwarding

Source: Chapter 4 pp. 49, 52, 54.

RAW means **read after write.** An instruction needs a value that an earlier instruction will write. In a simple pipeline without forwarding, the consumer reads the register file in ID and must wait until the producer's WB has made the value visible.

```
sub $t0, $t2, $t3
add $t3, $t0, $t1
```

The second instruction reads $t0, produced by the first. Without a fix, it reaches ID in cycle 3 while the first does not write $t0 until cycle 5. It could read the old value.

Consumer ID cycle	Producer WB cycle	Safe under same-cycle WB/ID?
3	5	No
4	5	No
5	5	Yes
6	5	Yes

Thus an immediately following dependent instruction needs two cycles of delay in this model. With one independent instruction between them, it needs one more delay. With two independent instructions between them, ID can coincide with producer WB and needs no extra delay.

Find **actual source registers**, not every written register name. addi $t0,$t1,4 reads $t1 but writes $t0. lw $t0,4($t1) reads its base $t1; its rt is a destination. sw $t0,4($t1) reads both data $t0 and base $t1. A branch reads both compared registers.

This in-order five-stage design principally exposes RAW hazards. More advanced out-of-order machines need additional handling for other dependency types; that is beyond this deck's basic timing model.

Remember: identify producer destination and consumer sources, then compare stage timing. Matching a destination to another destination alone is not a RAW dependency.

4.17 Reschedule code only when meaning stays unchanged

Source: Chapter 4 pp. 51-53.

The compiler can sometimes place independent work between a producer and consumer, filling cycles that would otherwise become stalls. This is like preparing another item while waiting for water to boil. The moved work must remain semantically safe.

```
# Original order
lw   $t1, 0($t0)
lw   $t2, 4($t0)
add  $t3, $t1, $t2
sw   $t5, 12($t0)
lw   $t4, 8($t0)
add  $t5, $t1, $t4
addi $t6, $t0, 4
```

The slide moves the load of $t4 and the independent addi after the second load. Under ordinary non-aliasing, valid memory accesses, this produces:

```
lw   $t1, 0($t0)
lw   $t2, 4($t0)
lw   $t4, 8($t0)
addi $t6, $t0, 4
add  $t3, $t1, $t2
sw   $t5, 12($t0)
add  $t5, $t1, $t4
```

In a no-forwarding pipeline, the second load writes in cycle 6; the first add now reads in cycle 6. The $t4 load writes in cycle 7; the final add reads later. This arrangement avoids the shown result hazards.

The store intentionally uses the **old** $t5, before the final add overwrites it. Moving it after that add would change the program. Likewise, moving a load across a store may be wrong if their addresses overlap, and moving work across a branch may execute it on a path where it originally did not run.

For every proposed move, check register reads/writes, memory aliases, and control flow. This lesson assumes ordinary nonfaulting accesses so the slide's scheduling problem can focus on data dependencies.

Remember: the goal is fewer delays with the same result, not simply a prettier pipeline table.

4.18 Insert stalls and count the slide's four-instruction example

Source: Chapter 4 pp. 54-56. Model: no forwarding; same-cycle WB write / ID read.

Consider the slide sequence:

```
lw  $t1, 0($t0)
lw  $t2, 4($t0)
add $t3, $t1, $t2
sw  $t3, 12($t0)
```

Without forwarding, the add must wait for $t2 from the second load. The store must then wait for $t3 from the add. Each immediate dependence requires two stall cycles in this model.

Instruction	IF cycle	ID cycle(s)	EX	MEM	WB / final stage
First lw	1	2	3	4	5
Second lw	2	3	4	5	6
add	3	4, 5, 6	7	8	9
sw	Held behind add	7, 8, 9	10	11	12, no register write

Repeated ID cycles mean the consumer waits; they are not multiple new executions of the instruction. The front end is held so younger instructions cannot overtake it. Older instructions continue toward completion.

Ideal cycles for four instructions in five stages would be 4 + 4 = 8. Four inserted delays give **12 cycles**, matching the slide's pipeline-drain convention. The store changes memory in cycle 11; if a question asks for the last architectural update rather than emptying all five stages, name that endpoint explicitly.

At a 200 ps clock, full drain takes 12 x 200 = 2400 ps. Adding forwarding changes this timing; do not reuse 12 blindly for a different implementation.

Remember: write the stopping condition beside your total: last useful memory update, final WB column, or completely drained pipeline.

4.19 Forwarding supplies a result before WB

Source: Chapter 4 pp. 57-60.

An arithmetic producer computes its result by the end of EX, before it finally writes the register file in WB. Forwarding, also called bypassing, routes that result directly to a younger instruction's input when it is needed.

```
sub $s2, $s1, $s3
and $s7, $s2, $s5
or  $t0, $s6, $s2
add $s0, $s2, $s2
sw  $s5, 100($s2)
```

The slide uses $s8 in the third instruction; that is not a normal saved-register name in the shown register map. Here $t0 provides an ordinary temporary destination without changing the dependency example.

Consumer	When it needs $s2	Where the latest value comes from
and immediately after sub	EX cycle 4	EX/MEM, sub result from cycle 3
or one instruction later	EX cycle 5	MEM/WB, sub's write-back value
add two instructions later	ID cycle 5 / EX cycle 6	Register file, under same-cycle WB/ID
sw still later	Base calculation	Register file

With the stated bypass paths, the five instructions can flow in 9 ideal cycles. The dependencies remain, but they do not require stalls.

Forwarding does not execute the producer twice and does not make its result ready earlier than EX. It changes the delivery route. A consumer's ID stage may read an old register value that is replaced by the correct forwarded value at EX.

Remember: name both the source pipeline register and the destination input. "There is forwarding" alone does not tell you which operand is fixed.

4.20 Forwarding MUX values and the newest-result rule

Source: Chapter 4 pp. 62-67; conditions checked with [W11].

The course uses F1 for the first ALU operand and F2 for the second register operand. Their binary encodings select among three candidate values.

Select bits	Source	Meaning
00	ID/EX	Value already carried from register read
10	EX/MEM	Result from the immediately older pipeline stage
01	MEM/WB	Final write-back value from an earlier instruction

For a genuine source register r, choose the newest older instruction that writes r, excluding $zero. Prefer its EX/MEM result when ready; otherwise a dependency can require waiting. Use MEM/WB only when no newer matching producer supersedes it. A load in EX/MEM has an effective address, not yet its loaded data; do not forward that address or fall back to an older stale value.

```
if EX/MEM writes a nonzero destination equal to source r:
    if its value is ready: use 10
    otherwise: consumer must wait for that value
else if MEM/WB writes a nonzero destination equal to r:
    use 01
else:
    use 00
```

Why prefer the newer producer? In add $s0,$s0,$s1; add $s0,$s0,$s2; add $s0,$s0,$s3, the third add needs the second add's updated $s0, not the first add's older update. If both stages match, EX/MEM wins.

Check RegWrite so a branch/store's coincidental destination-looking bits do not cause a false match. Exclude register 0 because writes to $zero have no effect. Use the carried **actual destination**, rt for a load and rd for core R-format arithmetic.

An I-format instruction is not automatically exempt from rt-related dependencies: stores and branches read rt. For an immediate ALU operation, the ALU's second selected operand is the immediate, so a matching rt destination is not a second-register input dependency. Store-data forwarding needs the corresponding data route; it is not guaranteed by merely having an ALU input MUX.

Remember: both source use and result availability matter. A register-number match is necessary but not sufficient.

4.21 Why an immediate load-use pair still needs a stall

Source: Chapter 4 p. 61.

For an arithmetic instruction, the answer exists at the end of EX. For a load, EX only calculates an address; the loaded value arrives at the end of MEM. An immediately following ALU consumer would need that value during EX in the very same cycle as the load's MEM. Ordinary forwarding cannot send a result backward in time.

```
lw  $t0, 0($s0)
add $t1, $t0, $s1
```

	1	2	3	4	5	6	7
lw	IF	ID	EX	MEM	WB		
add		IF	ID	hold	EX	MEM	WB

Load data arrives after MEM; consumer EX waits one cycle

Forward from MEM/WB to the delayed EX input

Instruction	**1**	**2**	**3**	**4**	**5**	**6**	**7**
lw	IF	ID	EX	MEM	WB	-	-
add	-	IF	ID	ID held	EX	MEM	WB
Bubble	-	-	-	EX	MEM	WB	-

The one-cycle delay moves the add's EX to cycle 5. The load result is now available from MEM/WB, so forwarding can deliver it. Total drain time for this pair is 7 cycles rather than the ideal 6.

Without forwarding, the add's ID must wait until load WB cycle 5, requiring two delays rather than one. Again, this assumes the register file can write and read the same register in one cycle.

An independent instruction placed between the load and add can fill the needed gap instead of a bubble. For example, an unrelated addi $s2,$s2,1 is suitable only if changing its position preserves the program's meaning.

Not every load/store adjacency needs the same delay. A store's base is needed in EX, but its data is used in MEM; a dedicated late store-data bypass can change the store-data case. State the supported paths before counting.

Remember: ask when the value is produced and when the consumer needs it. The word "forwarding" does not remove every hazard.

4.22 Hazard detection: freeze the front, inject a bubble

Source: Chapter 4 pp. 68-69. Correction: hold PC and IF/ID, not ID/EX.

Detect an immediate load-use hazard while the load is in EX and the consumer is in ID. The load's destination is ID/EX.rt. Compare it with the actual source registers of the decoded younger instruction.

```
load_use = ID/EX.MemRead
        AND ID/EX.rt != 0
        AND consumer actually reads that register
```

The slide writes the simple comparison against both IF/ID.rs and IF/ID.rt. This works for a chosen subset when the fields are genuine sources. A more complete decoder checks source use so an immediate instruction's destination rt does not create an unnecessary stall.

When the hazard is detected, do three different things:

1. **PCWrite = 0:** keep the same front-end PC rather than moving on.

2. **IF/IDWrite = 0:** keep the consumer instruction waiting in ID.

3. **Set the next ID/EX control bundle to zero:** insert a harmless bubble into EX.

Allow the older load to move from EX to MEM. If you freeze ID/EX and leave the load there, it cannot progress to produce its value, which defeats the intended repair. This is the important correction to slide 68's last bullet.

A cycle later, the load is farther along. The front can resume and the consumer enters EX, where its operand is supplied by forwarding. The bubble continues down the pipeline with state-changing enables disabled.

Do not confuse a stall with a flush. A stall preserves a valid instruction to execute later. A flush discards an instruction that should never complete, usually because it came from the wrong control-flow path.

Remember: hold the consumer, advance the producer, and insert an empty slot between them.

4.23 Branch hazards, prediction, and wrong-path flushing

Source: Chapter 4 pp. 50, 70.

The processor can fetch younger instructions before it knows whether a branch is taken. If the guessed next address is wrong, those younger instructions must not update registers or memory. Redirect fetching and flush the wrong-path work.

In the pictured basic pipeline, the branch decision reaches next-PC selection in MEM. Under predict-not-taken, three younger instructions can be present when the branch resolves: one in EX, one in ID, and one in IF. A taken branch then costs three lost instruction slots in this model.

Cycle	Branch stage	Younger work under not-taken guess
1	IF	None yet
2	ID	One younger fetch
3	EX	Younger instructions in ID and IF
4	MEM, resolves	Younger instructions in EX, ID, IF
5	Past decision	Correct target begins IF after redirect

This is a **no architectural delay-slot teaching pipeline**. In classic delayed MIPS, one specified following instruction must execute rather than being treated as ordinary wrong-path work. Do not mix those two models when counting flushed instructions.

For a long stream, extra branch CPI can be estimated as branch fraction x misprediction fraction x penalty. Example: 20% branches, 25% wrong predictions, and a 3-cycle penalty add 0.20 x 0.25 x 3 = 0.15 to base CPI.

Earlier branch resolution reduces the number of younger slots at risk, but requires earlier comparison and suitable operand handling. A branch resolved in ID can itself need stalls for not-yet-available operands. Always use the stage and penalty specified in a question.

Remember: count penalties from the timing model, not from a memorized universal number.

4.24 Static and dynamic prediction; a complete pipeline recipe

Source: Chapter 4 pp. 71-73.

Static prediction uses a fixed rule. For example, backward branches often close loops, so predicting them taken can work well. Forward branches often skip a block, so a not-taken guess is a simple heuristic. Neither rule guarantees the actual outcome.

Dynamic prediction records behavior and changes its guess. A one-bit toy predictor remembers whether the previous occurrence was taken. Suppose outcomes are T, T, T, N and the initial guess is N: it gets the first and last wrong, two mispredictions. If the loop repeats, the last N can also make the next first T wrong. More elaborate predictors reduce some of these changes of mind.

Prediction chooses which address to fetch. It does not change the branch's real condition. Incorrectly fetched instructions are discarded; the architectural result must still match sequential program execution.

Use this recipe for a pipeline exercise:

1. State five stages, clock period, WB/ID timing, forwarding paths, branch resolution, and endpoint.

2. List each instruction's real sources and destination.

3. Draw the ideal diagonal schedule.

4. Mark when each producer's value becomes available.

5. Add the minimum supported forwarding or delays; keep younger instructions behind a stall.

6. Add wrong-path flushes or miss delays under the specified model.

7. Count to the requested endpoint and multiply by clock period if time is requested.

Example: 100 instructions, five stages, 10 load-use bubbles, and 6 mispredicted branches with 3-cycle penalties require 100 + 4 + 10 + 18 = **132 cycles**, assuming penalties do not overlap. At 220 ps per cycle, time is 29,040 ps = 29.04 ns.

The ISA and microarchitecture influence one another. Simple fixed-length encodings and separate load/store operations help build a regular pipeline. Hardware choices then determine which dependencies require delay and how short each cycle can be.

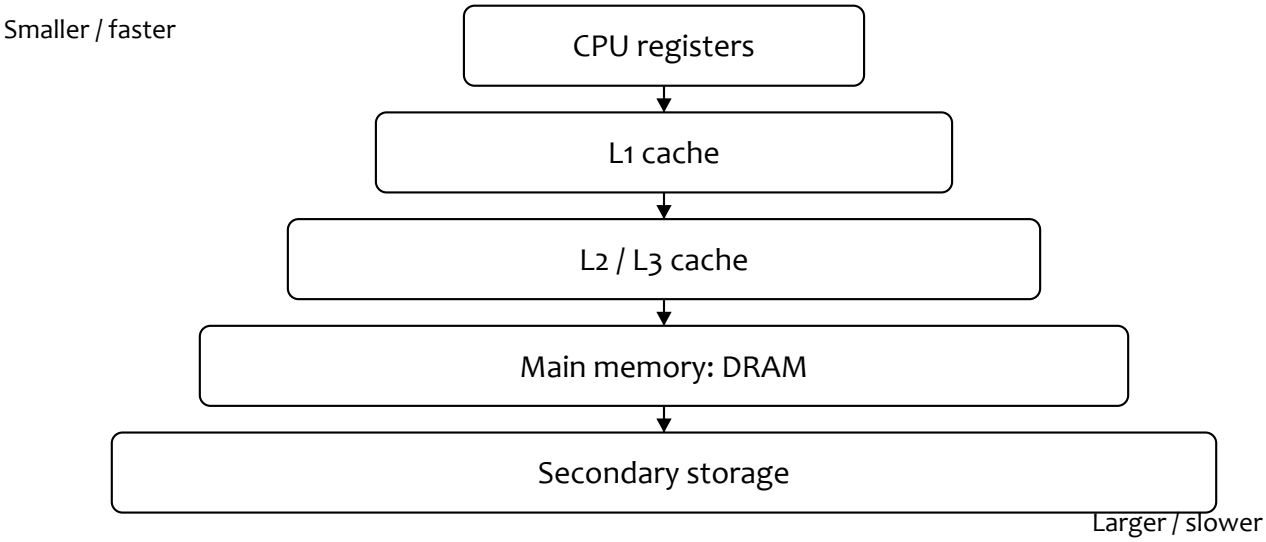
Remember: a pipeline table is an explanation of time, not only a final count. Every added blank slot should have a reason.

Chapter 5 | Memory hierarchy: caches and virtual memory

5.1 Why memory has several levels

Source: Chapter 5 pp. 2-7, 76.

Imagine studying with a few books on your desk, more books on a bookshelf, and a publisher that can supply books you do not own. The desk is small but easy to reach. The shelf holds more but takes longer to search. Ordering from the publisher takes much longer. The slides use this analogy for cache, main memory, and secondary storage.



Level	Typical role	General tradeoff
CPU registers	Current operands and results	Very small, very fast
L1 / L2 / sometimes L3 cache	Nearby copies of memory blocks	Increasing size, generally slower down the levels
Main memory / DRAM	Active program code and data	Much larger than cache
Secondary storage	Persistent files / backing storage	Larger, slower to access

We want the speed of a small fast memory and the capacity of a large cheap one. A hierarchy approximates that combination by keeping useful copies close to the processor. It does not make every possible access equally fast.

Moving information upward generally copies a **block**, not just the one requested byte. That helps when neighboring data is used next. The program still uses addresses; hardware checks whether a nearby copy exists.

A cache is not an extra set of Assembly registers. lw still refers to a memory address. The cache changes how quickly that access can be served, without changing the instruction's intended result.

The hierarchy relies on locality, explained next. Without recurring or neighboring accesses, the effort of bringing blocks closer may provide little benefit.

Remember: smaller fast levels hold a useful portion of information, not an independent replacement for all of memory.

5.2 Temporal and spatial locality

Source: Chapter 5 pp. 4-5.

Temporal locality means **reuse the same thing soon**. Spatial locality means **use nearby things soon**. The first concerns repeated time; the second concerns neighboring addresses.

Consider a loop summing an integer array. The loop's instruction addresses repeat each iteration, giving temporal locality. Its accumulator is reused, also temporal. Reading A[0], A[1], A[2], and so on gives spatial locality because consecutive word elements are four bytes apart.

Access behavior	**Locality you should recognize**
Read one variable repeatedly	Temporal
Execute the same loop instructions	Temporal
Walk through consecutive array elements	Spatial
Read nearby instructions sequentially	Spatial
Randomly visit unrelated blocks once each	Weak locality

If a cache block contains four integer words and a loop reads those words in order, the first access might miss and bring all four in. The next three can hit if the block remains present. A one-byte-at-a-time string scan also benefits from fetching neighboring bytes together.

Repeated access does not guarantee a hit: the block may have been evicted. Neighboring addresses do not guarantee they are in the same block: a boundary can separate them. Locality predicts useful behavior; placement and capacity determine whether the cache preserves it.

Arrays and linked structures can differ. An array stores elements consecutively. Linked nodes may be scattered, so following pointers can offer less spatial locality even if the abstract data structure contains the same values.

Remember: temporal = same item again; spatial = neighboring item next. A program can exhibit both.

5.3 SRAM, DRAM, flash, and storage

Source: Chapter 5 p. 7.

The hierarchy uses different technologies because no one technology offers every desirable property at low cost. Learn the relative roles rather than treating the slide's dated price figures as current shopping information.

Technology	Main idea	Common course role
SRAM	Keeps a bit while powered without periodic DRAM-style refresh	Fast CPU caches
DRAM	Stores charge and needs refresh	Main memory
Flash	Nonvolatile electronic storage	SSDs and other persistent storage
Magnetic disk	Mechanical magnetic storage	Large secondary storage / historical backing-store example

SRAM is generally faster and more expensive per stored bit than DRAM. DRAM provides large main-memory capacity. Flash retains data without power, but its access/write behavior differs from ordinary RAM. A magnetic disk adds mechanical access delay.

The slide's illustrative ranges place SRAM in nanoseconds, DRAM in tens of nanoseconds, flash in microseconds, and disk in milliseconds. The useful lesson is the scale difference. Specific latency and cost depend on the device, operation, and era.

Be comfortable with units: 1 microsecond = 1000 nanoseconds; 1 millisecond = 1,000,000 nanoseconds. A millisecond storage access can take millions of cycles on a GHz processor. This is why a page fault is much more expensive than a cache hit.

For capacities in this guide, **KiB = 1024 bytes, MiB = 1024^2 bytes**. The slides often write KB for a power-of-two capacity; the exercise statements here use KiB to remove ambiguity. Decimal kB = 1000 bytes is a different unit.

Remember: being nonvolatile does not mean being fast enough to replace CPU registers or an L1 cache.

5.4 Blocks, hits, misses, and miss penalties

Source: Chapter 5 pp. 6, 8, 20.

A cache line is a slot that holds a block of data plus identifying state. A memory block is the copied chunk belonging to a range of addresses. Different memory blocks can occupy the same cache line at different times.

For byte address A and block size B bytes:

```
Memory block number = floor(A / B)
Byte offset in block = A modulo B
Block's first byte address = floor(A / B) x B
```

If B = 16 and A = 36, block number is 2, offset is 4, and the block covers addresses 32 through 47. A = 44 belongs to the same block with offset 12.

A **hit** means the requested block is already present in the relevant cache. A **miss** means it must be obtained from the next level. Hit rate = hits / accesses; miss rate = misses / accesses = 1 - hit rate.

Example: eight accesses with six hits and two misses have hit rate 6/8 = 75% and miss rate 25%. The miss penalty is extra time needed to handle a miss beyond the initial lookup time under the convention used here.

In a basic blocking-cache pipeline, a cache miss holds the needed operation until the block arrives. An instruction-cache miss delays fetching; a data-cache miss delays the load/store's data access. More advanced processors can overlap some other work, discussed in lesson 5.19.

Remember: cache-line count, block size, and total data capacity are connected: capacity = number of lines x bytes per line.

5.5 Direct mapping: one possible line per memory block

Source: Chapter 5 pp. 9-10.

In a direct-mapped cache, a memory block has exactly one allowed line. Compute its index by taking the block number modulo the number of cache lines. Think of numbered lockers: every visitor is assigned one locker by a rule, even if another visitor is already using it.

```
L = number of cache lines
index = block_number modulo L
tag = floor(block_number / L)
```

For eight lines, blocks 6, 14, 22, and 30 all map to line 6. Their tags differ: 0, 1, 2, and 3. If block 22 occupies line 6 and block 30 is requested, block 30 replaces it even when another line is empty.

The restriction makes lookup simple: select one line, compare its tag, and check its valid bit. It can also create conflicts between frequently used blocks that share an index.

In byte-addressed memory, first divide by the block's byte size. If B = 4 bytes and address A = 88, the block number is 22, which maps to line 6. Do not use byte address 88 directly as the block number unless B = 1.

The early slides label their inputs **word addresses** with one word per block. There, word number 22 is already block number 22. The corresponding byte address is 22 x 4 = 88. This distinction explains why the same example can show index 6 even though 88 modulo 8 is 0.

Remember: determine whether the given number is a byte address, a word address, or a block number before applying modulo.

5.6 Tags, valid bits, and the slide's eight-access trace

Source: Chapter 5 pp. 10-16.

An index tells you where to look, but not which block currently occupies that line. The tag identifies the block. The valid bit says whether the line contains usable data. Initially invalid lines must miss even if their uninitialized tag bits happen to match.

Use an initially empty, eight-line direct-mapped cache with one word per line. The inputs below are **word/block numbers**, following the slides: 22, 26, 22, 26, 16, 3, 16, 18.

Access	Block	Index = block mod 8	Tag = floor(block/8)	Outcome / update
1	22	6	2	Miss; line 6 becomes valid, tag 2
2	26	2	3	Miss; line 2 becomes valid, tag 3
3	22	6	2	Hit; matching valid tag
4	26	2	3	Hit
5	16	0	2	Miss; fill line 0
6	3	3	0	Miss; fill line 3
7	16	0	2	Hit
8	18	2	2	Miss; replace block 26 in line 2

There are three hits and five misses. Hit rate is 3/8 = 37.5%. Final occupied lines contain blocks 16 at index 0, 18 at index 2, 3 at index 3, and 22 at index 6. Other lines remain invalid.

Block 18 and block 26 collide because both use index 2. A valid bit alone would incorrectly call the last access a hit; the tag comparison reveals that the line holds the wrong block.

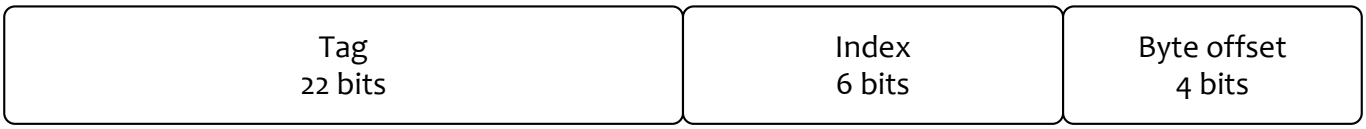
Trace one request at a time: compute fields, inspect the old line state, decide hit/miss, update that line, then move to the next request. Deciding every request from the initial empty state loses the effects of previous fills.

Remember: hit = valid AND matching tag. Valid is necessary, but it is not sufficient.

5.7 Split a byte address into tag, index, and offset

Source: Chapter 5 pp. 17-18, 25, 38.

For a power-of-two organization, the low address bits select a byte within a block. The next bits select the line or set. The remaining high bits form the tag.



Example: 32-bit address, 64 lines, 16-byte blocks

Let A be address width, C data capacity in bytes, B bytes per block, and W associativity. Then lines = C/B, sets = C/(B x W), offset bits = log2(B), index bits = log2(sets), and tag bits = A - index bits - offset bits.

Example: a 32-bit byte address, 1 KiB direct-mapped cache, and 16-byte blocks. C = 1024, B = 16, W = 1. There are 64 lines, offset width 4, index width 6, and tag width **22**.

Field	Bit positions	Width
Tag	31:10	22
Index	9:4	6
Byte offset	3:0	4

If blocks contain words, you may split the total offset further. With 16-byte blocks and 4-byte words, the offset has two word-select bits and two byte-within-word bits. Their total is still four.

Metadata uses storage too. For this cache, data requires 1024 x 8 = 8192 bits. Tags and valid bits require 64 x (22 + 1) = 1472 bits. Total is **9664 bits**, before dirty bits, replacement state, and other overhead. The advertised 1 KiB normally refers to data capacity, not the entire physical circuit.

For a fully associative cache, there is one set, so index width is zero. Do not reserve index bits simply because the cache has multiple lines; placement freedom changes the address split.

Remember: compute sets after dividing by associativity. Then check tag + index + offset = address width.

5.8 Larger blocks and the address-1200 example

Source: Chapter 5 pp. 18-19.

The slide's cache has 64 lines and 16-byte blocks. For byte address 1200:

1. Block number = floor(1200/16) = 75.

2. Index = 75 modulo 64 = 11.

3. Tag = floor(75/64) = 1.

4. Byte offset = 1200 modulo 16 = 0.

The addressed block starts at 1200 and covers bytes 1200 through 1215. The line chosen is 11; the byte chosen within that line is 0. The block number 75 is not the cache-line index.

Larger blocks can improve spatial locality: one miss supplies more neighbors. However, for fixed data capacity, increasing B reduces the number of lines. That increases competition and can evict useful data sooner.

Increase block size	Possible benefit	Possible cost
More neighboring bytes per fill	Fewer misses on sequential scans	Fetch unused bytes, called pollution
Fewer lines at fixed capacity	-	More placement competition
More bytes transferred per miss	Better use of transfer bursts	Longer miss service time

Critical-word-first requests the word the processor currently needs before the rest of the block. **Early restart** resumes the processor as soon as that required word arrives, without waiting for the whole fill. These can reduce the observed wait even though the complete block transfer still needs time.

The best block size depends on locality, cache capacity, and transfer cost. "Larger is always better" is not a safe conclusion.

Remember: answer every requested unit explicitly: memory block, cache line/set, and byte offset are separate results.

5.9 Split caches and the FastMATH example

Source: Chapter 5 pp. 20, 24-25.

A split cache organization has one cache for instructions, I-cache, and one for data, D-cache. This permits fetching an instruction while another instruction accesses data, supporting the pipeline's separate-resource model.

The historical FastMATH example has 16 KiB in each cache, organized as 256 blocks x 16 words per block. With four-byte words, each block is 64 bytes and 256 x 64 = 16,384 bytes.

For its shown direct-mapped 32-bit address split, offset bits = log2(64) = 6; index bits = log2(256) = 8; tag bits = 32 - 6 - 8 = 18. The six offset bits divide into four word-select bits and two byte-select bits.

Field	Width	Selects
Tag	18	Identity of the memory block
Index	8	One of 256 lines
Word within block	4	One of 16 words
Byte within word	2	One of 4 bytes

The slide's I-cache and D-cache miss rates differ because instruction and data streams have different behavior. A weighted average must use **numbers of accesses**, not simply the average of two percentages. If each instruction causes one instruction fetch but only a quarter cause a data access, their weights are not equal.

On an I-cache miss, the next required instruction cannot be supplied immediately. On a D-cache miss, an already-fetched load/store waits for data. A later CPI calculation should count the relevant access frequency for each cache separately.

This processor example is a historical illustration; its 12-stage pipeline is not the five-stage timing model used in Chapter 4's exercises.

Remember: changing the example processor changes its timing assumptions even when both processors implement MIPS instructions.

5.10 Write-through and a write buffer

Source: Chapter 5 p. 21, p. 74.

A store changes data. If the cache updates its copy, what happens to the next level? **Write-through** sends each cache write onward to the next level as well. This keeps the next level up to date after pending writes finish.

Without buffering or overlap, slow lower-level writes can dominate execution. The slide assumes base CPI = 1, stores = 10% of instructions, and an extra 100 cycles for every store. Extra CPI = 0.10 x 100 = 10; total CPI = **11.**

A **write buffer** is a queue of writes waiting to reach the next level. The processor places a write in the buffer and can continue when there is space. The buffer drains separately. It hides some delay; it does not eliminate the lower level's work.

Buffer situation	**Consequence**
Free entry available	Enqueue write and continue under the model
Buffer full	Processor may have to wait
Load needs a location with a pending write	Must receive the newest value, not stale lower-level data

The last case requires ordering or checking the buffer. Otherwise a store followed by a load of the same address could incorrectly return the old value. A real implementation includes mechanisms to preserve program-visible memory behavior.

Write-through describes what happens to lower memory **when writing**. It does not by itself determine whether a missing block should be brought into the cache. That is a separate write-allocation policy.

Remember: lower memory may temporarily lag while a write is buffered. The design must still deliver the correct latest value to the program.

5.11 Write-back, dirty bits, and write allocation

Source: Chapter 5 pp. 22-23, 58, 74.

Write-back changes the cached copy and postpones updating the next level until the modified block is evicted. A dirty bit records whether the cache contains changes absent from the next level.

Event	Valid bit	Dirty bit in a write-back cache
Initial empty line	0	Irrelevant
Read miss fills clean block	1	0
Store changes cached block	1	1
Clean block evicted	Replacement occurs	No data write-back needed
Dirty block evicted	Replacement occurs	Send old data to next level first / safely buffer it

Suppose memory X = 10. Load X into cache, then store 20 to cached X. Cache now has 20, dirty = 1, while lower memory can still hold 10. A later load of X hits and returns 20. On eviction, write 20 back so the newer value is not lost.

On a **write miss**, write-allocate fetches a block into the cache, then modifies it. No-write-allocate, or write-around, sends the write onward without filling that cache. Write-back commonly pairs with write-allocate; write-through often pairs with no-write-allocate, but these pairings are policies, not logical necessities.

Two questions must therefore be answered separately: on a write hit, do we update the lower level immediately? On a write miss, do we allocate a line? Four combinations can be discussed even if only some are common.

A dirty bit is not a valid bit. A valid clean block is usable. A dirty block is normally valid and contains newer data; "dirty" does not mean corrupt.

Remember: account for dirty-eviction time when a problem includes it in the miss penalty. Do not silently treat every replacement as a clean fill.

5.12 Memory bandwidth: narrow, wide, and interleaved transfers

Source: Chapter 5 pp. 26-27.

Latency measures how long one transfer takes. Bandwidth measures bytes transferred per unit time. A cache fill's cost depends on the memory width, bus width, and whether accesses can overlap.

The slide models a four-word, 16-byte block. It assumes one bus cycle for the address, 15 bus cycles per DRAM access, and one bus cycle per returned word.

Organization	Modeled fill time	Bandwidth for 16 bytes
One-word-wide memory, serial words	1 + 4 x 15 + 4 x 1 = 65 cycles	16/65 = 0.246 B/cycle
Four-word-wide memory and transfer	1 + 15 + 1 = 17 cycles	16/17 = 0.941 B/cycle
Four interleaved banks, serial returns	1 + 15 + 4 x 1 = 20 cycles	16/20 = 0.800 B/cycle

The wide arrangement returns all four words together. Interleaving lets different banks work on different words in parallel, but the narrower return path still sends them one by one. These timings describe the slide's assumed overlap, not every possible DRAM system.

If the bus runs at 100 MHz, one bus cycle is 10 ns. The 65-cycle transfer then takes 650 ns. Its bandwidth is 16 bytes / 650 ns, about 24.6 million bytes per second. Do not multiply bus cycles by the CPU's different clock period without first relating the clocks.

Doubling CPU frequency alone does not necessarily halve physical DRAM latency. It can instead make the same nanosecond wait equal more CPU cycles.

Remember: label whether a count is in bus cycles or CPU cycles before converting it to time.

5.13 Cache misses add to CPI

Source: Chapter 5 pp. 28-29, 31.

Base CPI describes execution with the assumed hit behavior already included. Misses add stall cycles. In a simple blocking model, extra cycles = number of relevant accesses x miss rate x miss penalty.

If every instruction is fetched once, I-cache extra CPI = I-miss rate x penalty. D-cache extra CPI also needs the fraction of instructions that access data memory.

```
Actual CPI = base CPI
        + I_miss_rate x I_miss_penalty
        + data_accesses_per_instruction
          x D_miss_rate x D_miss_penalty
```

Use the slide's values: base CPI 2; I-miss rate 2%; D-miss rate 4%; loads/stores 36%; both penalties 100 cycles. I contribution is 0.02 x 100 = 2. D contribution is 0.36 x 0.04 x 100 = 1.44. Actual CPI = **5.44**.

For 1000 instructions, base cycles = 2000, I-miss stalls = 2000, D-miss stalls = 1440, total = 5440. At 2 GHz, CPU time = 5440 / (2 x 10^9) = 2.72 microseconds. The ideal-cache version takes 1 microsecond, so it is 5.44/2 = 2.72 times faster under the model.

Do not multiply D-miss rate by all instructions if only loads/stores access data. Do not add hit time again if it is already in base CPI. If a miss metric is already **misses per instruction**, no access-frequency factor is needed.

This model assumes penalties can be added without useful overlap. For advanced out-of-order execution, some waits overlap and the formula needs refinement.

Remember: percentages are fractions in calculations: 4% = 0.04, not 4.

5.14 Average memory access time is not automatically CPI

Source: Chapter 5 p. 30.

AMAT, average memory access time, averages the service time of memory accesses. With miss penalty defined as extra time after the initial lookup, **AMAT = hit time + miss rate x miss penalty**.

The slide uses hit time 1 ns, miss rate 5%, and extra penalty 20 ns. AMAT = 1 + 0.05 x 20 = **2 ns per access**. This can also be seen as 95% taking 1 ns and 5% taking 21 ns: 0.95 x 1 + 0.05 x 21 = 2.

Quantity	Unit	What it averages
AMAT	ns/access or cycles/access	Memory access service
CPI	cycles/instruction	Complete instruction execution
Misses per instruction	misses/instruction	Miss frequency relative to program work

The slide's "2 cycles per instruction" needs additional assumptions, such as one serial instruction-memory access dominating each instruction with a 1 ns cycle. In a pipelined system, memory access frequency, overlap, and base execution time must be handled separately. Do not universally equate AMAT with CPI.

If a problem defines "miss time" as the **total** miss service time rather than extra penalty, use hit_rate x hit_time + miss_rate x total_miss_time. Example: total miss time 20 ns gives 0.95 x 1 + 0.05 x 20 = 1.95 ns, not 2 ns.

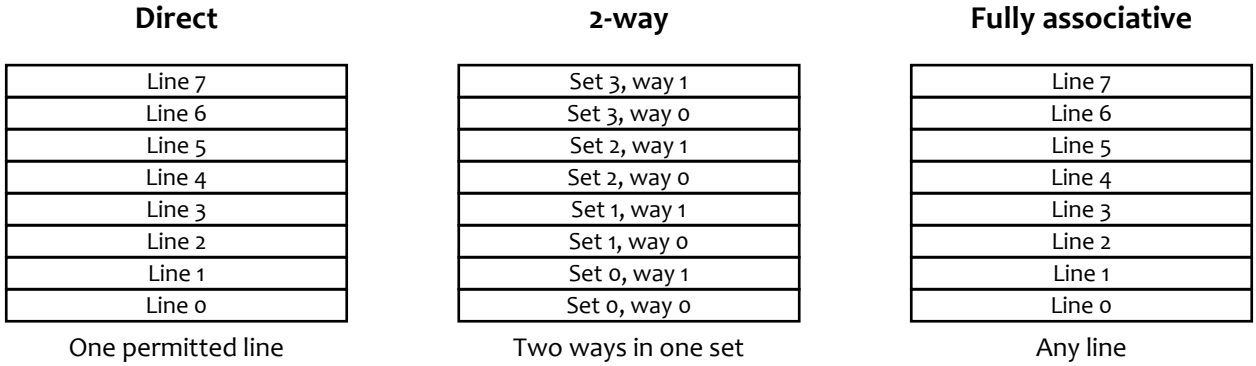
Increasing associativity might lower miss rate but increase hit time. Compare complete AMAT: design A, hit 1 ns, miss 8%, penalty 20 ns gives 2.6 ns; design B, hit 1.2 ns, miss 5%, same penalty gives 2.2 ns. B is better for this metric despite its slower hit.

Remember: write the miss-time convention and units before substituting values.

5.15 Associativity changes where a block may go

Source: Chapter 5 pp. 32-34, 38, 71-72.

Direct mapping gives one choice. Fully associative placement permits any line. W-way set associativity gives W choices in one selected set. The address chooses the set; a replacement policy chooses a way within it when necessary.



Organization with 8 total lines	Number of sets	Choices for a block	Tag comparisons on lookup
Direct mapped / 1-way	8	One line	1
2-way	4	Two lines in selected set	2
4-way	2	Four lines in selected set	4
Fully associative / 8-way	1	Any of eight lines	8

For block b, set = b modulo number_of_sets. In an eight-line 2-way cache, b = 6 maps to set 6 mod 4 = 2, but either way in that set is allowed. Block 10 also maps to set 2; both can coexist if its two ways are available.

Only tags in the selected set need comparing. A fully associative cache searches all lines, which requires more comparators or another lookup structure. Increased freedom can reduce conflicts but adds hardware and may increase access time.

A tag is still needed in an associative cache. More possible placements do not remove the need to identify which block each line contains. With one set, no set-index bits are needed; more high address bits become tag bits.

Remember: W is lines per set, not the number of sets. Compute sets = total lines / W.

5.16 Trace direct, 2-way, and fully associative caches

Source: Chapter 5 pp. 35-37.

The slide compares four total cache lines for block sequence **0, 8, 0, 6, 8**. Start empty. For associative versions use least recently used, LRU, within the selected set.

Direct mapped has four sets. Blocks 0 and 8 both map to index 0; block 6 maps to index 2. The first three requests replace one another at index 0. All five requests miss.

A 2-way cache has two sets. All these even blocks map to set 0, so only that set's two ways help. The table writes its order from least recent to most recent.

Block	Direct outcome	2-way outcome	2-way set-0 LRU order	Fully associative outcome
0	Miss	Miss	0	Miss
8	Miss	Miss	0, 8	Miss
0	Miss	Hit	8, 0	Hit
6	Miss	Miss; evict 8	0, 6	Miss
8	Miss	Miss; evict 0	6, 8	Hit

The fully associative cache has four available lines and only three distinct blocks, so none needs eviction. Results: direct 0 hits; 2-way 1 hit; fully associative 2 hits. Hit rates are 0%, 20%, and 40%, respectively.

A cache hit changes recency for LRU. The third access to 0 makes 8 older, explaining why 8 is evicted when 6 arrives. If you record recency only on misses, you will choose the wrong victim.

The simulation figures on slide 37 illustrate diminishing returns from additional associativity. They are workload-specific results, not universal miss rates.

Remember: more total lines in the other set do not help when the placement rule sends all requested blocks to one set.

5.17 Replacement: invalid first, then LRU or an approximation

Source: Chapter 5 pp. 39-40, 73.

When an associative set misses, first use an invalid way if one is available. Only choose a victim among valid lines when the set is full. In a direct-mapped cache there is no victim choice: the one indexed line must be replaced.

LRU evicts the line whose latest access happened furthest in the past. Maintain a list from oldest to newest. On a hit, move the accessed block to the newest end. On a miss, insert the new block there after using an empty line or removing the oldest.

Example with a two-way set: after A, B, A, the order is B, A. Request C and evict B, producing A, C. A was loaded earlier than B but used more recently, so arrival time alone is not LRU.

Policy	Decision basis	Tradeoff
LRU	Exact recent-use order	More state/update logic at high associativity
Pseudo-LRU	Approximate recent use	Less hardware; can choose a different victim
Random	Random eligible way	Simple; workload-dependent outcome

The slide's "one bit per line" describes an approximation idea, not a complete universal exact-LRU implementation. With many ways, exact ordering needs more information than a single independent recently-used bit. Tree-based or bit-based approximations use different representations.

For a classroom LRU trace, write the order explicitly rather than trying to remember it mentally. Decide hit/miss from **old state**, then update recency. Never evict a block merely because its tag value is numerically smallest.

Remember: replacement operates within the chosen set. Fully associative placement has only one set, so its policy can choose among all lines.

5.18 L1 and L2: distinguish local and global miss rates

Source: Chapter 5 pp. 41-44.

An L1 miss first consults L2. An L2 hit is slower than an L1 hit but avoids main memory. An L2 miss goes farther down. Larger slower levels serve fewer requests because the earlier level handles most of them.

Local L2 miss rate = L2 misses / L2 accesses. **Global miss rate** = misses reaching memory / original top-level accesses, or per instruction when explicitly specified. If L1 miss rate is m1 and local L2 miss rate is m2, global rate is m1 x m2.

For access-time modeling with extra times t2 and tmem:

AMAT = L1_hit_time + m1 x (t2 + m2 x tmem)

The slide instead gives misses per instruction, directly suitable for CPI. At 4 GHz, one cycle is 0.25 ns. Base CPI = 1, L1 misses/instruction = 2%, main memory extra time = 100 ns = 400 cycles. Without L2, CPI = 1 + 0.02 x 400 = **9**.

With L2, its extra access time is 5 ns = 20 cycles. Global misses/instruction reaching memory are 0.5%. Every L1 miss pays the L2 access, and the subset reaching memory pays the additional memory cost: CPI = 1 + 0.02 x 20 + 0.005 x 400 = **3.4**.

The corresponding local L2 miss rate is 0.005 / 0.02 = 25%. Performance improvement is 9/3.4, approximately 2.647. Do not multiply 0.005 by 0.02 again: 0.005 already has the global denominator.

L1 tends to prioritize low hit latency; L2 can prioritize reducing expensive memory misses. Their optimal sizes and blocks need not match.

Remember: beside every rate, write its denominator: top-level accesses, L2 accesses, or instructions.

5.19 Cache stalls interact with CPU speed and program dependencies

Source: Chapter 5 pp. 31, 45-46.

Making arithmetic faster can expose memory as the bottleneck. If base CPI falls but miss stalls stay the same, memory becomes a larger share of total execution time. This is another application of partial-speedup reasoning from Chapter 1.

Suppose base CPI = 2 and miss-stall CPI = 3, total 5. Halving base CPI produces 1 + 3 = 4, so overall speedup is only 5/4 = 1.25, not 2.

Increasing clock rate can also make a fixed physical wait cost more cycles. A 50 ns access is 100 cycles at 2 GHz but 200 cycles at 4 GHz. The seconds spent waiting are unchanged. A new CPI must use penalties converted with the new period.

Out-of-order processors may continue independent work while a load waits. Dependent operations must still wait for that load's value. A pending store remains tracked until it can safely complete. Therefore the visible delay depends on data flow, not only the miss count.

Sequence after a missing load	Opportunity
Several independent calculations	Some delay can overlap useful work
Next address depends on loaded pointer	Little opportunity; chain must wait
Many concurrent misses with enough resources	Potential overlap / memory-level parallelism

The simple CPI formulas earlier assume blocking additive waits. Use them when a question gives that model. For an advanced processor, simulation or a richer model may be required; do not claim a simple formula captures every overlap.

Algorithm and compiler choices also change memory access order. The next lesson shows a software method for making reuse occur while data is still cached.

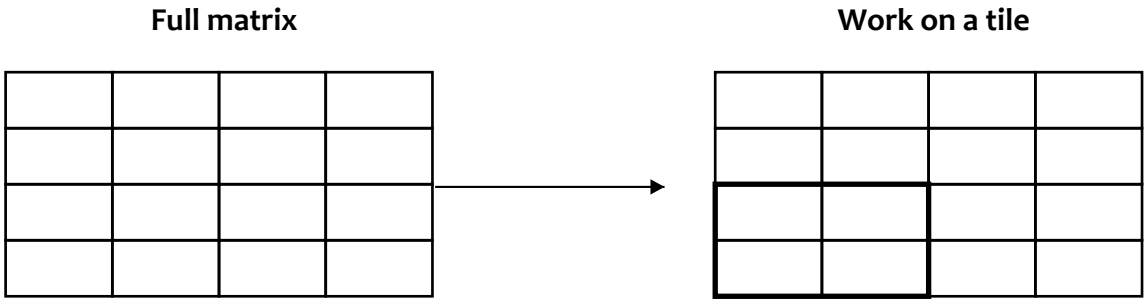
Remember: a nanosecond delay and its cycle count are linked through clock period; they are not interchangeable constants.

5.20 Matrix access and blocking keep useful data nearby

Source: Chapter 5 pp. 47-51.

DGEMM is double-precision matrix multiplication. Each C element accumulates products of elements from A and B. The slides store matrices using **column-major** indexing: address position for row i, column j is i + j x n. Moving i by 1 moves one element; moving j by 1 jumps n elements.

The slide's inner calculation is C[i+j*n] += A[i+k*n] x B[k+j*n]. As k increases, B accesses consecutive elements, while A steps by n elements. A stride can pull many different blocks into cache.



Reuse a small group before moving to the next group

For three n-by-n double matrices, total data size is 3 x n^2 x 8 bytes. With n = 32, it is 24,576 bytes = 24 KiB. With n = 64, it becomes 98,304 bytes = 96 KiB. These data totals explain why a small cache may stop retaining everything as n grows; tags, associativity, and competing accesses also matter.

Blocking divides the computation into tiles and reuses the tile's data before moving on. It changes access order, not the intended mathematical matrix product. To keep boundary handling correct for arbitrary n, clamp each tile's end.

```
for each row tile si, column tile sj, inner tile sk:
  for i from si to min(si+B, n)-1:
    for j from sj to min(sj+B, n)-1:
      cij = C[i+j*n]
      for k from sk to min(sk+B, n)-1:
        cij += A[i+k*n] * B[k+j*n]
      C[i+j*n] = cij
```

Accumulate over **all** sk tiles. Do not reset cij to zero on every inner tile, which would erase earlier partial products. For initial C = 0, initialize once before accumulating; the slide's formula otherwise performs C += A x B.

A rough three-tile footprint is 3 x B^2 x 8 bytes. B = 32 gives 24 KiB. It is a useful estimate, not a guarantee that any 32 KiB cache will hit perfectly. The original full-tile loop also needs an n-multiple-of-B assumption or boundary checks.

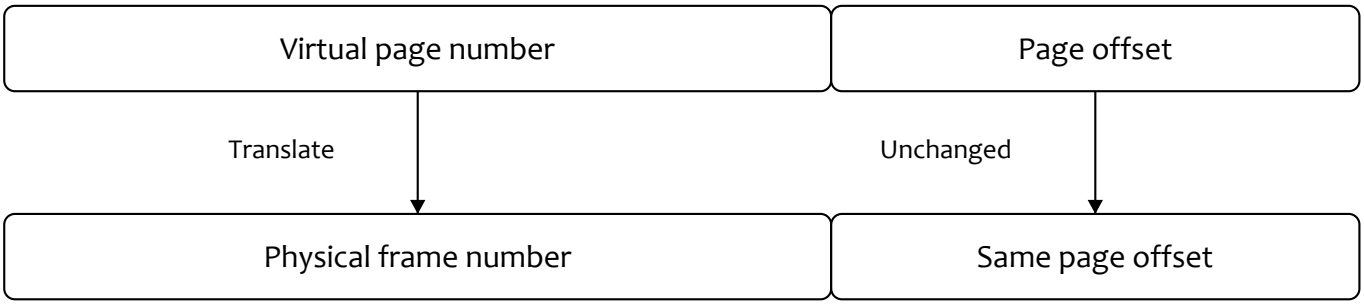
Remember: use the array layout actually given. Row-major A[i*n+j] and column-major A[i+j*n] have different fast traversal directions.

5.21 Virtual addresses, physical addresses, and pages

Source: Chapter 5 pp. 52-54. The deck labels virtual memory self-learning; it is fully explained here.

Virtual memory gives a program its own address-space view. The CPU and operating system translate a virtual address to a physical memory location, while also checking permissions. Two processes can use the same virtual address without sharing the same physical data.

Memory is divided into fixed-size **pages**. Physical memory's corresponding slots are often called frames. A virtual page can be placed in a physical frame; the offset within the page remains unchanged.



For page size P bytes and virtual address VA:

```
VPN = floor(VA / P)          # virtual page number
offset = VA modulo P
PA = physical_frame_number x P + offset
```

Example: with 4 KiB pages, VA = 4669 gives VPN = 1 and offset = 573. If virtual page 1 maps to physical frame 13, PA = 13 x 4096 + 573 = **53,821**.

With 4 KiB pages, offset width is 12 bits. A 32-bit virtual address has 20 VPN bits. If physical addresses are 30 bits as in the slide illustration, physical frame number width is 18 bits. The offset still has 12 bits; changing page identity does not change the selected byte within a page.

The page size changes the split. With 16 KiB pages, offset width is 14 and the same VA = 4669 belongs to VPN 0, offset 4669. Do not reuse the 4 KiB page-number sequence for the 16 KiB variant.

Remember: translation replaces the page number, not the page offset.

5.22 Page tables and permission information

Source: Chapter 5 pp. 55-57, 65.

A page table maps virtual page numbers to physical frames and status. A page-table entry, PTE, can say whether a page is present, which frame holds it, whether writing is permitted, and whether it was recently used or modified.

PTE information	Why it is needed
Present / resident bit	Does this mapping currently have a usable frame?
Physical frame number	Where is the page in main memory?
Read / write / execute permission	Which accesses are allowed?
Referenced / use bit	Evidence for replacement decisions
Dirty bit	Does backing storage need an update before eviction?

For a simple single-level table, use VPN as an array index. The page-table base register locates the table. A PTE address is base + VPN x PTE_size. Its exact layout and width are machine-specific.

Example: 32-bit virtual addresses and 4 KiB pages give 2^20 possible VPNs. If each PTE uses 4 bytes, a full flat table would use 2^20 x 4 = **4 MiB** per address space. This motivates more compact or multi-level arrangements, a supplementary explanation of why actual OS tables can be more elaborate.

Presence and permission are different checks. A present page can still reject a write if it is read-only. A page not currently resident can need OS work. A completely invalid unmapped address is not necessarily a valid page that can simply be fetched from disk.

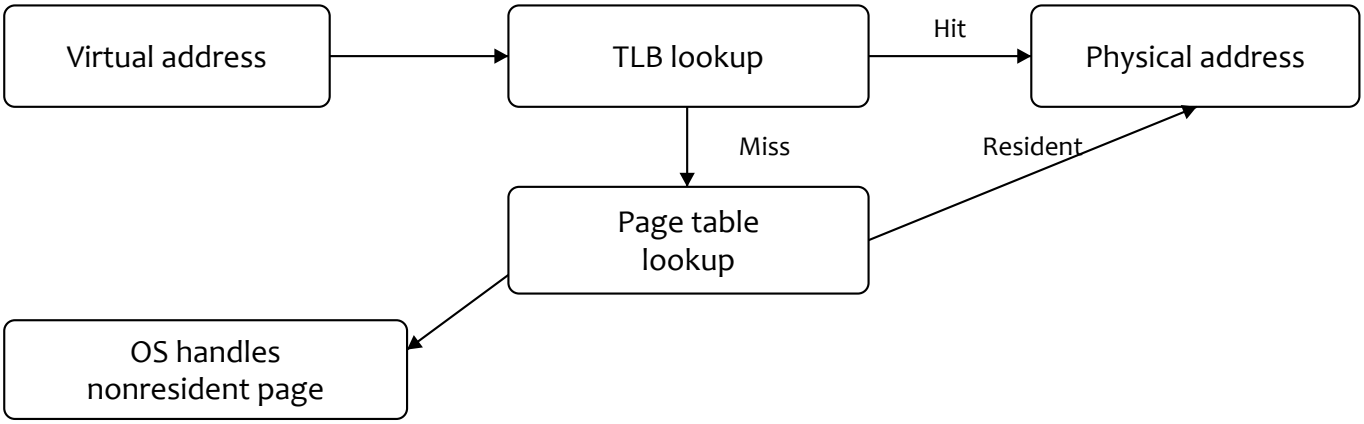
The operating system controls mappings and permissions using privileged execution. User programs cannot freely edit these protected structures. A system-call instruction requests an intentional transition into OS services; a forbidden access causes an exception.

Remember: a page-table entry identifies and protects a page. It is not a cache copy of all that page's data.

5.23 TLB miss, page fault, and cache miss are different

Source: Chapter 5 pp. 59-63.

A TLB, translation lookaside buffer, is a small fast cache of address translations. It remembers recent VPN-to-frame mappings so the CPU does not need a full page-table lookup for every reference.



Event	What is missing?	Typical next action
TLB miss, page resident	A cached translation	Obtain PTE, refill TLB, retry
Page fault for valid nonresident page	A resident physical page	OS supplies the page, updates mapping, retries
Data-cache miss	A nearby copy of a memory block	Fetch block from next cache / memory level

A **TLB miss does not imply a page fault**. The required page can already be in DRAM; only its translation is absent from the small TLB. A TLB hit also does not imply a data-cache hit: knowing the frame does not guarantee the data block is cached.

With a basic physically addressed cache model, first translate VA to PA, then use the physical address for cache lookup. A virtually tagged cache can look up a virtual identity but must handle aliases: different virtual addresses referring to the same physical data. Some modern designs overlap indexing and translation; this deck's simple serial view is the starting model.

The slide's typical TLB capacities and timing ranges are illustrative, not universal device specifications. A miss can be handled by hardware page-table walking or by a software handler, depending on the architecture.

Translation failures must be recognized before an incorrect destination is committed. For a valid page serviced by a handler, restart the faulting instruction after the mapping is ready.

Remember: translation caches mappings; data caches blocks. Never combine their hit counts without knowing which lookup each count describes.

5.24 Page faults, replacement, dirty pages, and protection

Source: Chapter 5 pp. 54, 58, 62-65.

In the slide's disk-backed model, a nonresident page can require a very slow storage read. Real valid faults can also involve demand-zero allocation or other OS work. The essential idea is that the operating system must make a usable permitted mapping before the access can complete.

For a valid page stored on disk, the handler follows this sequence:

1. Use the faulting virtual address to find the PTE and backing location.

2. Obtain a free physical frame or select a victim page.

3. If the victim is dirty, preserve its changes by writing it back.

4. Read the required page into the chosen frame.

5. Update the page table and translation state consistently.

6. Restart the faulting instruction when the process can run again.

Pages have flexible frame placement, resembling fully associative data placement. A page table finds the selected frame directly rather than comparing the requested VPN against every frame. Flexibility and lookup cost need not be implemented the same way as an associative hardware cache.

Exact LRU for all pages is costly. The OS can approximate recency with referenced bits set by hardware and periodically cleared. A page without recent references becomes a possible victim. A dirty bit records writes; a clean victim may be discarded without a storage write if a valid backing copy exists.

Write-through to disk on every ordinary store would be impractically slow, so paging uses write-back behavior. Page-table reference and dirty bits are not the same as a data-cache line's valid/dirty bits, although their meanings are analogous at different levels.

Permissions remain necessary even when a page is resident. Protected kernel mappings and read-only pages cannot be bypassed merely by having a TLB entry. Hardware and OS cooperate to enforce access rights.

Remember: restart the instruction whose access could not complete, not automatically the following instruction.

5.25 Work through the deck's TLB exercise one request at a time

Source: Chapter 5 pp. 67-70.

The slide supplies a four-entry fully associative TLB and a page table. The valid initial TLB mappings are VPN 11 -> frame 12, VPN 7 -> frame 4, and VPN 3 -> frame 6; the row for VPN 4 is invalid. The page table additionally has resident VPNs 0 -> 5, 4 -> 9, 5 -> 11, and 10 -> 3. VPNs 1, 2, 6, 8, 9 are initially on disk.

To make a reproducible trace, use these explicit assumptions: initial valid TLB recency from oldest to newest is 11, 7, 3; use the invalid slot first; page faults allocate new frames 13, 14, and so on with no physical eviction. The table shown ends at VPN 11, so **assume the later requested VPN 12 starts nonresident**. Without that assumption, its outcome is not fully specified by the initial slide table.

For 4 KiB pages, calculate VPN = floor(address/4096) before searching. The TLB order below is oldest to newest after the access. A "PTE hit" means the page is already resident even though the TLB missed.

Address	VPN	Event	Frame	TLB order after access
4669	1	TLB miss; page fault	13	11, 7, 3, 1
2227	0	TLB miss; PTE hit	5	7, 3, 1, 0
13916	3	TLB hit	6	7, 1, 0, 3
34587	8	TLB miss; page fault	14	1, 0, 3, 8
48870	11	TLB miss; PTE hit	12	0, 3, 8, 11
12608	3	TLB hit	6	0, 8, 11, 3
49225	12	TLB miss; page fault	15	8, 11, 3, 12
12948	3	TLB hit	6	8, 11, 12, 3
49419	12	TLB hit	15	8, 11, 3, 12
46814	11	TLB hit	12	8, 3, 12, 11
13975	3	TLB hit	6	8, 12, 11, 3
40004	9	TLB miss; page fault	16	12, 11, 3, 9
12707	3	TLB hit	6	12, 11, 9, 3
52236	12	TLB hit	15	11, 9, 3, 12

There are six TLB misses, eight TLB hits, and four page faults under these assumptions. Evicting a TLB entry does not evict its physical page: VPN 11's translation can miss while frame 12 still holds its data.

Repeat the experiment with 16 KiB pages

Restart from the same initial mappings and LRU order, now using 14 offset bits. The VPN sequence changes because each page contains four times as many bytes. The first VPN 0 reference misses in TLB but is resident. The first VPN 2 reference faults and receives frame 13.

Address	VPN	Event	Frame	TLB order after access
4669	0	TLB miss; PTE hit	5	11, 7, 3, 0
2227	0	TLB hit	5	11, 7, 3, 0
13916	0	TLB hit	5	11, 7, 3, 0
34587	2	TLB miss; page fault	13	7, 3, 0, 2
48870	2	TLB hit	13	7, 3, 0, 2
12608	0	TLB hit	5	7, 3, 2, 0
49225	3	TLB hit	6	7, 2, 0, 3
12948	0	TLB hit	5	7, 2, 3, 0
49419	3	TLB hit	6	7, 2, 0, 3
46814	2	TLB hit	13	7, 0, 3, 2
13975	0	TLB hit	5	7, 3, 2, 0
40004	2	TLB hit	13	7, 3, 0, 2
12707	0	TLB hit	5	7, 3, 2, 0
52236	3	TLB hit	6	7, 2, 0, 3

There are two TLB misses, twelve TLB hits, and one page fault. Final TLB recency is 7, 2, 0, 3. The invalid initial fourth row did not count as an existing translation. Different page sizes produce different offset values as well as VPNs, so a final physical address must use the new page size.

Remember: changing page size changes which addresses share a VPN. Reset the initial state before solving the alternative page-size experiment.

5.26 Four hierarchy questions and the three kinds of cache miss

Source: Chapter 5 pp. 66, 71-76.

Every hierarchy level needs answers to four questions: where may a block go, how is it found, what is replaced on a miss, and when are writes sent downward? The same themes recur in data caches, TLBs, and paging, though their mechanisms differ.

Design question	Hardware-cache example	Virtual-memory example
Placement	Direct / set / fully associative	Page may use a selected available frame
Finding	Index plus tag comparison	Page-table lookup, often cached in TLB
Replacement	LRU / pseudo-LRU / random	OS recency approximation
Writes	Write-through or write-back	Dirty-page write-back

Cache misses are often described using the **three Cs**:

Miss type	Why it happens	Small example
Compulsory / cold	First-ever access to that block	First request for block A
Capacity	Working set exceeds total available data lines	Three repeatedly used blocks in a two-line cache
Conflict	Placement restriction causes competition beyond same-size fully associative behavior	Alternate blocks 0 and 8 in a four-line direct cache

Classify the requested block's miss, not merely the cause of the most recent eviction. A first access to a new block is compulsory even if it happens to replace another block. A later miss can be a conflict if a same-capacity fully associative cache would have kept it.

For precise trace comparisons, use a fully associative cache of the same capacity and replacement convention as a reference. If even that cache misses after the block's first access, insufficient capacity is the important limitation under the model.

Finish a memory problem by checking address units, data capacity versus metadata, initial valid state, associativity, replacement, write policy, rate denominators, and time units. The correct formula is useful only after these assumptions are clear.

Remember: caches exploit reuse; virtual memory adds translation and protection. They share hierarchy ideas but solve different immediate problems.

Source map and corrections

The guide follows all six supplied decks by Cuong Pham-Quoc / HCMUT: Introduction and Chapters 1-5, totaling 302 slides. Source references at the beginning of lessons use **PDF page numbers**, counted from the first page, which match the numbered slides in these decks. This guide paraphrases and expands their content; its diagrams and extra examples were created for teaching rather than copied from the slides.

Source pages	Guide lessons	Main subjects
Introduction 1-9	I.1-I.3	Course orientation, computer, architecture, learning goals, recorded administration
Chapter 1 1-8, 10, 14	1.1	History, Moore's Law, technology and historical charts
Chapter 1 9-11	1.2	Computer classes and components
Chapter 1 12-13	1.3	Software layers and representations
Chapter 1 15-19	1.4-1.5	Performance metrics, time and relative performance
Chapter 1 20-22	1.6	Clock timing and a redesign example
Chapter 1 23-26, 28	1.7-1.8	IC, CPI, weighted instruction costs
Chapter 1 27	1.9	Partial improvement exercise
Chapter 1 29-31	1.10-1.11	Power and parallelism
Chapter 1 32-34	1.12	Benchmarks and conclusions
Chapter 2 1-10	2.1	ISA, stored programs, fetch / execute and design principles
Chapter 2 11-13	2.3-2.4	Operands, register roles and arithmetic
Chapter 2 14-23	2.5-2.9	Memory, extension, endianness and arrays
Chapter 2 24-30	2.10-2.11	Shifts and bitwise operations
Chapter 2 31-40	2.12-2.16	Branches, loops, comparisons and pseudo instructions
Chapter 2 41-50	2.17-2.20	Calls, stack frames, leaf / recursive procedures
Chapter 2 51	2.21	String-copy exercise
Chapter 2 52-63	2.22-2.24, 2.27	Machine formats, fields, encoding and decoding
Chapter 2 64-70	2.25-2.27	Branch and jump addresses; combined encoding
Chapter 2 71-73	2.28	Far branches, summary and end page
Chapter 3 1-6	3.1-3.4	Integer representations, addition/subtraction, overflow and exceptions
Chapter 3 7-11	3.5-3.7	Multiplication algorithms, optimized hardware, HI/LO

Source pages	Guide lessons	Main subjects
Chapter 3 12-16	3.8-3.10	Long/restoring division, quotient/remainder and MIPS instructions
Chapter 3 17-25	3.11-3.15	Binary fractions, IEEE fields, range, encode/decode, special values
Chapter 3 26-30, 35	3.15-3.17	Rounding, addition hardware and steps, multiplication
Chapter 3 31-34, 36-37	3.18-3.20	FP registers, instructions, programs, encoding and conclusions
Chapter 4 1-9	4.1-4.2	Implementation, stages, hardware components and MUXes
Chapter 4 10-22	4.3-4.7	Datapath routes, instruction fields, controls and worked traces
Chapter 4 23-25	4.8-4.9	Next PC, jump extension and critical path
Chapter 4 26-30	4.9-4.11	Pipelining, latency/throughput and speedup
Chapter 4 31-46	4.12-4.14	Cycle diagrams, pipeline registers and carried control signals
Chapter 4 47-56	4.15-4.18	Hazard types, rescheduling and no-forwarding stalls
Chapter 4 57-67	4.19-4.21	Forwarding, availability, MUX values and priority
Chapter 4 68-73	4.22-4.24	Load-use detection, freeze/bubble, branches and prediction
Chapter 5 1-8	5.1-5.4	Hierarchy, locality, technology and hit/miss vocabulary
Chapter 5 9-20	5.5-5.8	Direct mapping, state traces, tags/offsets and block size
Chapter 5 21-25	5.9-5.11	Split caches, FastMATH, writes and allocation
Chapter 5 26-31	5.12-5.14, 5.19	Bandwidth, CPI, AMAT and CPU interaction
Chapter 5 32-40	5.15-5.17	Associativity, comparators, LRU and approximation
Chapter 5 41-51	5.18-5.20	Multilevel costs, software access, DGEMM blocking
Chapter 5 52-65	5.21-5.24	Virtual memory, page tables, TLBs, handlers and protection
Chapter 5 67-70	5.25	Full address-request trace and page-size variant
Chapter 5 66, 71-76	5.26	Placement/finding/replacement/writes and three-Cs synthesis

Title, divider, and end slides are mapped with neighboring topics. Binary foundations, extra traces, bounds checks, and debugging recipes supplement the decks' explanations.

Corrections and conventions worth keeping beside the slides

Source location	Clarification used in this guide
Chapter 1 p. 6	"First computer" depends on the definition; use the narrower historical description for ENIAC
Chapter 1 p. 5 and historical charts	Treat the dated product and trend examples as historical snapshots
Chapter 2 p. 19	Use big-endian for the slide exercises; MIPS implementations can use either byte order
Chapter 2 p. 22	Halfword access at address 15 is misaligned; state whether experiments are independent or sequential
Chapter 2 p. 30	sll with immediate 44 is not an encodable ordinary 32-bit MIPS shift
Chapter 2 p. 35	Correct sltui to sltiu; its immediate is sign-extended before unsigned comparison
Chapter 2 pp. 32 and 45	The slides use a simplified call model; the classic reference card has delay slots and jal saves PC + 8
Chapter 2 p. 47	The slide's 4-byte frame illustrates one saved word; examples here use 8-byte teaching frames
Chapter 2 p. 56	nor funct is 0x27, decimal 39, rather than 0x28
Chapter 2 p. 67	bne $t0, $t1 with displacement -2 encodes as 0x1509FFFE
Chapter 2 pp. 54 and 62	Format shortcuts describe a chosen core subset; consult actual tables for the wider ISA
Chapter 3 p. 6	Correct addui to addiu; distinguish non-trapping MIPS from standard C signed-overflow semantics
Chapter 3 p. 11	Signed product fit requires HI equal to LO's sign extension; HI != 0 alone is not a signed overflow test
Chapter 3 pp. 13-15	Keep each division algorithm's initialization and iteration count together
Chapter 3 pp. 22-23	Smallest magnitudes shown are normal limits; subnormals extend the range toward zero
Chapter 3 p. 26	Keep extra intermediate digits when aligning; the displayed 0.016 is rounded shorthand
Chapter 4 p. 23	Jump's high prefix is from instruction address + 4; a region boundary exposes the old-PC shortcut
Chapter 4 pp. 28-30	Five unbalanced stages need not give 5x speedup; the 800/200 ps example approaches 4x

Source location	Clarification used in this guide
Chapter 4 pp. 37-38	Carry the destination number with the older instruction into WB
Chapter 4 p. 48	Separate resources eliminate the pictured structural conflict, not every possible MIPS structural hazard
Chapter 4 p. 58	$s8 is outside the shown saved-register map; use an ordinary temporary destination for this example
Chapter 4 pp. 63-68	Check actual source use, RegWrite, zero destination, newest producer and value availability
Chapter 4 p. 68	Freeze PC and IF/ID; zero the controls entering ID/EX to make a bubble; let the load advance
Chapter 5 p. 30	AMAT is time/access, not automatically whole-instruction CPI
Chapter 5 p. 40	Pseudo-LRU is an approximation; one per-line bit is not a universal exact-LRU implementation
Chapter 5 p. 50	Handle partial tiles or assume n is divisible by tile size; keep previous inner-tile sums
Chapter 5 pp. 52-64	Distinguish missing translation, valid nonresident page, forbidden address, and data-cache miss
Chapter 5 pp. 67-70	Explicitly choose initial TLB LRU order and a nonresident VPN-12 assumption to complete the trace

Local supporting references

Materials/MIPSreferenceCard.pdf, one-page reference card: checked register numbers, instruction formats, immediate extension, opcodes, function codes, and classic call behavior.

Materials/MipsInstructionSetReference.pdf, MIPS Technologies instruction-set manual: checked byte-order support, alignment, instruction restrictions, and operation details. Its presence here provides additional verification beyond the optional reference card.

Web verification references

The following primary sources support selected clarifications. The supplied slides and local manuals remain the main sources. Foundations retain the references checked for the earlier guide on 9 October 2026; selected arithmetic and pipeline checks were added on 14 October 2026. Explanations and examples were written for this guide, not copied from web pages.

W1. University of Pennsylvania: ENIAC historical account. Supports the qualified description of ENIAC as a programmable, general-purpose electronic computer.

University of Pennsylvania: ENIAC history

W2. Intel: Moore's Law historical press kit. Supports the distinction between the 1965 projection and its 1975 revision.

Intel: Moore's Law history

W3. SPEC: CPU2006 suite overview. Identifies CPU2006 as a retired suite.

SPEC CPU2006 overview and retired-suite notice

W4. SPEC: CPU2006 overview and run rules. Supports reference-time normalization and geometric-mean ratio summaries.

SPEC CPU2006 reference times and summary metrics

W5. James R. Larus: SPIM documentation. Records simulator support for delayed control transfers.

SPIM documentation by James R. Larus

W6. MIPS Technologies: MIPS32 Architecture for Programmers, Volume II, revision 0.95, hosted by Cornell. Used as a primary-source cross-check of the core instruction rules; the supplied local manual also supports these checks.

MIPS Technologies: MIPS32 Volume II, revision 0.95

W7. Cornell CS3410 course implementation notes. Independently explains that sltiu sign-extends its immediate before an unsigned comparison.

Cornell CS3410: SLTIU implementation notes

W8. Oracle Numerical Computation Guide, single format. Checked reserved binary32 fields, the hidden-bit distinction, and normal/subnormal limits.

Oracle Numerical Computation Guide: single format

W9. Oracle Numerical Computation Guide, IEEE arithmetic model. Checked rounding-mode terminology and default nearest-even behavior.

Oracle Numerical Computation Guide: IEEE arithmetic

W10. GNU C language manual, signed overflow. Checked the distinction between language-level undefined behavior and selected hardware/compiler wrapping behavior.

GNU C language manual: signed overflow

W11. Cornell CS3410 forwarding implementation notes. Checked destination guards, load-result availability, and priority for the most recent matching producer.

Cornell CS3410: forwarding conditions

Practice | Make an attempt before opening the key

There are 112 questions across the practice sections at the end. Questions 1-14 review introduction and performance; 15-35 build Assembly skills; 36-42 cover procedures; 43-48 cover machine encoding; 49-68 cover arithmetic and floating point; 69-88 cover datapath and pipelining; 89-112 cover memory hierarchy and virtual memory. Each question points to its related lessons.

Use 32-bit MIPS, big-endian memory, and the no-delay-slot model unless a question explicitly asks for another convention. Assume signed arithmetic results fit in 32 bits unless a question concerns bit truncation. Do not use pseudoinstructions when a question requests core instructions or machine encodings. Byte entries and words prefixed by 0x are hexadecimal. Other numbers are decimal. State intermediate steps and units. Memory experiments are independent unless explicitly described as a sequence.

The answer key is supplied as a separate PDF. This section contains questions and working space only. Longer programming solutions can be written on extra paper.

Q01. What is architecture?

Related lessons: I.1 and 2.1.

Explain the difference between an ISA and a processor implementation. Can two processors use the same ISA and still have different CPU times? Give a reason.

Q02. Find the computer class and component

Related lessons: 1.2.

Classify a laptop, a washing-machine controller, a network file server, and a scientific simulation system. Then identify which component performs arithmetic, which holds active program data, and which connects a keyboard to the system.

Q03. Connect the representations

Related lessons: 1.3.

Put high-level code, Assembly, and encoded machine instructions in their translation order. Explain the roles of a compiler, assembler, and operating system. Is a label an instruction executed by the CPU?

Q04. Interpret a technology claim

Related lessons: 1.1.

Explain why a doubling of transistor count does not guarantee that the same program runs in half the time. Starting from 500 transistors in an illustrative example, calculate the count after three doublings.

Q05. Distinguish the kinds of time

Related lessons: 1.4.

One task computes for 4 seconds, waits for I/O for 5 seconds, and computes for 2 seconds. There is no overlap or other work. Find elapsed time and CPU time. If only computation becomes twice as fast, find the new elapsed time. Explain which metric describes how many independent tasks a system finishes per hour.

Q06. Relative performance and percentages

Related lessons: 1.5.

The same program takes 12 seconds on A and 18 seconds on B. How many times is A's performance greater than B's? Moving from B to A, calculate both the percentage increase in performance and the percentage reduction in time.

Q07. Clock units

Related lessons: 1.6.

A CPU runs at 2.5 GHz. Find its period in nanoseconds. A small operation requires 4,000 cycles; find its CPU time in microseconds.

Q08. A redesign changes the cycle count

Related lessons: 1.6.

Computer A runs at 3 GHz and takes 8 seconds. B requires 1.25 times as many cycles for the same task and must finish in 5 seconds. Find A's cycles, B's cycles, and B's required frequency.

Q09. The full CPU-time equation

Related lessons: 1.7.

A program executes 1.2 million instructions with average CPI 1.75 on a 2.1 GHz CPU. Find the cycle count and CPU time in milliseconds. Explain why the GHz number alone is insufficient.

Q10. Weighted CPI

Related lessons: 1.8.

For a 10,000-instruction program, class A is 40% with CPI 1, class B is 35% with CPI 2, and class C is 25% with CPI 4. Find each class count and cycle contribution, total cycles, weighted CPI, and CPU time at 3 GHz.

Q11. A partial improvement

Related lessons: 1.9.

Use Q10's workload and clock. Class C's CPI is reduced from 4 to 2; other costs and counts stay fixed. Find the new cycles, new average CPI, new CPU time, and overall speedup. What fraction of the original cycle time belonged to class C?

Q12. Dynamic power ratios

Related lessons: 1.10.

Effective capacitance becomes 90% of its old value, voltage becomes 80%, and frequency becomes 110%. Assume switching activity stays constant. Find the new/old dynamic-power ratio and the percentage reduction. If original dynamic power was 80 W, find the new value.

Q13. Parallelism with a serial portion

Related lessons: 1.11.

25% of the original execution time is serial; 75% parallelizes perfectly across four cores. Ignore overhead. Find normalized new time and overall speedup. Explain why it is below four.

Q14. A benchmark summary

Related lessons: 1.12.

Calculate the three reference/measured time ratios, then their geometric mean. Explain why you should not average the measured seconds to produce that ratio summary.

Benchmark	Reference time	Measured time
A	100 s	50 s
B	120 s	30 s
C	80 s	10 s

Q15. Binary and hex

Related lessons: 2.2.

Convert 0x6D to 8-bit binary and decimal. Show the bit-weight addition. How many bytes are in a 32-bit word?

Q16. The same byte, two interpretations

Related lessons: 2.2 and 2.6.

For byte 0xD6, give its unsigned and signed 8-bit values. Give its 32-bit sign-extended and zero-extended patterns. Also state the signed and unsigned interpretations of 0xFFFFFFFF as a 32-bit word.

Q17. A register trace

Related lessons: 2.4.

Initially $t1 = 8 and $t2 = 3. Trace each instruction, recording $t0, $t1, and $t2 after every line. A register not yet assigned can be marked unchanged or unknown.

```
add  $t0, $t1, $t2
sub  $t1, $t0, $t2
addi $t2, $t1, -2
```

Q18. Translate an expression

Related lessons: 2.4.

Write core MIPS for f = (g - h) + (i + 6), using f, g, h, i in $s0-$s3. Preserve g, h, and i. Trace your code for g = 10, h = 4, i = 3.

Q19. Addresses and alignment

Related lessons: 2.5.

With $s0 = 2048, calculate the address accessed by each instruction and decide whether its alignment is valid: lw $t0, 12($s0); lhu $t1, 3($s0); sb $t2, 7($s0). Do not calculate a loaded value because none is supplied.

Q20. Load widths and extension

Related lessons: 2.6-2.7.

Big-endian memory at addresses 4096-4099 contains bytes 80 F1 7F 00 (hex), and $s0 = 4096. Find the 32-bit result for each independent load: lw $t0, 0($s0); lh $t1, 0($s0); lb $t2, 0($s0); lbu $t3, 0($s0); lhu $t4, 2($s0).

Q21. Partial stores, executed in sequence

Related lessons: 2.6-2.7.

Initially addresses 6000-6007 contain zero bytes, $s0 = 6000, and $t0 = 0x1234ABCD. Execute sw $t0, 0($s0), then sb $t0, 4($s0), then sh $t0, 6($s0). Write all eight final bytes in address order. Does $t0 change?

Q22. Big-endian and little-endian layouts

Related lessons: 2.7.

Write the four bytes of word 0x90ABCDEF at addresses 100-103 in each byte order. For each layout, find the result of lb from address 101. These are two independent layouts.

Q23. A constant array index

Related lessons: 2.8.

Write core MIPS for A[7] = h + A[2], with word-array base in $s3 and h in $s2. State the load and store offsets. If h = 9 and A[2] = 13, describe the memory change.

Q24. A variable array index

Related lessons: 2.9.

Write core MIPS for f = A[i] + 1, with f in $s0, i in $s1, and A's base in $s6. Assume a valid nonnegative index. Trace the address and result when A's base is 1000, i = 3, and A[3] = 20.

Q25. A nested array index

Related lessons: 2.9.

Write core MIPS for f = g - A[B[2]], with f in $s0, g in $s1, A base in $s6, and B base in $s7. Both arrays contain words and the index is valid. Trace the roles of your temporaries when g = 50, B[2] = 4, and A[4] = 7.

Q26. Valid shifts and a malformed instruction

Related lessons: 2.10.

For $t0 = 0x80000011, independently calculate sll $t1, $t0, 4 and srl $t2, $t0, 4. Explain what happens to lost bits. Is sll $t1, $t0, 44 a valid ordinary immediate-shift instruction?

Q27. Bitwise operations

Related lessons: 2.11.

With $t0 = 0x12345678 and $t1 = 0x00FF00FF, independently find: and $t2, $t0, $t1; or $t3, $t0, $t1; nor $t4, $t0, $zero; andi $t5, $t0, 0x00F0. Show at least one nibble-level calculation.

Q28. Build a full-word constant

Related lessons: 2.6 and 2.11.

Use lui and ori to place the bit pattern 0x89ABCDEF in $t0. Give the intermediate value after each instruction. Do these instructions read memory?

Q29. Follow a loop and count its branches

Related lessons: 2.12.

Trace the code. Give final $t0 and $t2, how many times bne executes, how many times it is taken, and the total number of executed instructions shown below.

```
addi $t0, $zero, 0
addi $t1, $zero, 3
Loop: addi $t0, $t0, 1
bne  $t0, $t1, Loop
add  $t2, $t0, $t1
```

Q30. Translate a signed if / else

Related lessons: 2.13-2.14.

Using a in $s0 and b in $s1, translate if (a >= 5) a = a + b; else a = a - b; with core instructions. Test your code for (a, b) = (4, 3) and (5, 3).

Q31. Signed and unsigned comparison

Related lessons: 2.14.

With $t0 = 0xFFFFFFFF and $t1 = 0x00000001, find the results of slt $t2, $t0, $t1 and sltu $t3, $t0, $t1. Explain the interpretations used.

Q32. A sign-extended unsigned comparison

Related lessons: 2.14.

With $t0 = 65,536, find the result of sltiu $t1, $t0, -1. Show the 32-bit immediate after extension. Explain how an incorrect zero-extension assumption would change the comparison.

Q33. A bounded search loop

Related lessons: 2.15.

Translate i = 0; while (i < n && A[i] == k) i++;. Use A base in $s0, i in $s1, k in $s2, and nonnegative n in $s3. Test i < n before accessing A[i]. Trace A = [4, 4, 9], n = 3, k = 4. Also explain what happens if all three elements equal 4.

Q34. A for loop that accumulates a sum

Related lessons: 2.15.

Translate for (i = 0, sum = 0; i < 4; i++) sum += i;, with sum in $s0 and i in $s1. Give a table of i and sum for each body execution, and their final values.

Q35. Expand conveniences

Related lessons: 2.16.

Give a real-instruction expansion for move $t0, $t1 and for blt $t0, $t1, Less. State which reserved register your branch expansion uses and why a pseudoinstruction may affect instruction-address calculations.

Q36. Calls in two execution models

Related lessons: 2.17.

A jal instruction is at decimal address 4000. State the saved return address in the no-delay-slot model and in classic delayed MIPS. Explain which instruction address is the delay slot, and what jr $ra does.

Q37. Who preserves the value?

Related lessons: 2.18.

Before a call, $s0 = 77 and $t0 = 9. The callee uses both registers and returns normally under the convention taught here. Which value must it restore? If the caller needs its old $t0 afterward, who must arrange preservation? Is the preservation performed automatically by the CPU?

Q38. A leaf procedure using a saved register

Related lessons: 2.19.

Write a leaf procedure SumPlusOne with arguments in $a0 and $a1 and result in $v0. It returns a0 + a1 + 1. Use $s0 as its local working register, preserve the incoming $s0, and restore $sp before returning. Use an 8-byte frame. Trace arguments 10 and 20, incoming $s0 = 55 and $sp = 1000.

Q39. A stack-frame address calculation

Related lessons: 2.18.

Initially $sp = 0x1000. A procedure reserves 16 bytes and stores $ra at 12($sp). Find the new $sp and the saved word's address. What must $sp be when this returning procedure has released its entire frame?

Q40. Repair a non-leaf return

Related lessons: 2.20.

Explain the bug below. Rewrite Outer using an 8-byte teaching frame to preserve the incoming return address. Inner returns normally and may overwrite caller-saved registers. Assume Outer needs no incoming data value after calling Inner.

```
Outer:
jal Inner
jr  $ra
```

Q41. Trace recursive factorial

Related lessons: 2.20.

Using the lesson's factorial model, trace fact(3). Every invocation, including fact(0), allocates 8 bytes before testing the base case. Give the call sequence, returned values during unwinding, maximum simultaneous frames, maximum stack bytes, and the lowest $sp if the initial $sp is decimal 1000. What result and $sp should the caller receive?

Q42. Copy a terminator

Related lessons: 2.21.

A source string contains hex bytes 41 42 00. Four destination bytes initially contain FF FF FF FF. After the lesson's copy loop, what are the four destination bytes, how many bytes were copied, and what indexes were used? Explain why testing zero before the store would require an extra terminator store.

Q43. Choose formats

Related lessons: 2.22.

Classify these core instructions as R, I, or J: add, sll, lw, bne, j, jr, lui. Give the field widths of each format and explain why the immediate in sll does not make it I-format.

Q44. Encode R-format instructions

Related lessons: 2.23.

Encode sub $t2, $s0, $s1 and sll $t0, $t1, 3. Show every field and the final eight-digit hex word. Use sub function 0x22 and sll function 0x00.

Q45. Encode I-format instructions

Related lessons: 2.24.

Encode addi $s0, $s0, -8 and sw $t2, 12($s1). Show the signed-immediate representation, each field, and the final hex words. Use opcodes 0x08 and 0x2B, respectively.

Q46. Encode backward and forward branches

Related lessons: 2.25.

Encode bne $t0, $t1, Loop at address 0x100C with Loop at 0x1000. Separately encode beq $s0, $s1, Next at 0x2000 with Next at 0x2010. Show each PC + 4, byte difference, signed displacement, and machine word. Use opcodes 5 for bne and 4 for beq.

Q47. A jump at an address-region boundary

Related lessons: 2.26. Challenge question.

A j instruction is at 0x8FFFFFFC and targets 0x90000040. Find P + 4, the upper nibble used for reconstruction, the 26-bit target index, and the machine word. Could that same instruction address directly use J-format to target 0x80000040? Explain using the upper bits of P + 4.

Q48. Encode and trace a complete sequence

Related lessons: 2.27-2.28. Integration question.

Start is at 0x00400000. Assign addresses and encode every instruction. Then trace the no-delay-slot execution, giving final $s0 and the numbers of executed and taken bne instructions. Use addi opcode 8, bne opcode 5, j opcode 2, and add function 0x20.

```
Start: addi $t0, $zero, 3
Loop:  addi $t0, $t0, -1
bne    $t0, $zero, Loop
j      Done
Done:  add $s0, $t0, $zero
```

Practice | Chapter 3 arithmetic

Use the stated widths. Integer division truncates toward zero. IEEE questions use binary32 and round-to-nearest, ties-to-even unless stated otherwise. Assembly fragments use no delay slots. No solutions appear in this question section.

Q49. Range and interpretation

Related lessons: 3.1-3.3.

Interpret four-bit 1110 as unsigned and signed two's complement. State both four-bit ranges. Show the four-bit pattern for -5 and explain why +8 does not fit signed four-bit storage.

Q50. Carry and signed overflow

Related lesson: 3.2.

Add four-bit 0110 and 0101. Give the low-four-bit result, exact unsigned sum, signed interpretation of the result, carry out, and signed-overflow decision. Then explain why signed -1 + 1 can have a carry out without signed overflow.

Q51. Subtraction using two's complement

Related lesson: 3.3.

Calculate signed four-bit -6 - 5. Show the operand patterns, the negated second operand, and the stored result. Does the exact mathematical result fit? Explain using both range and sign rules.

Q52. Trapping versus non-trapping arithmetic

Related lesson: 3.4.

With $t0 = 0x7FFFFFFF, compare addi $t1,$t0,1 and addiu $t1,$t0,1. State how addiu extends its immediate. Correct the claim that ordinary signed C overflow is guaranteed to wrap.

Q53. Partial-product multiplication

Related lesson: 3.5.

Multiply unsigned four-bit 1101 by 1011 using partial products and an eight-bit answer. Identify which multiplier bits contribute rows. Give the decimal product and explain why the answer needs more than four bits.

Q54. A shift-and-add trace

Related lesson: 3.6.

Trace four iterations with eight-bit P = 0, eight-bit M = 3, and four-bit Q = 5. Test Q's low bit; add M if it is 1; then shift M left and Q right. Show P, M, and Q after each complete iteration.

Q55. Unsigned HI and LO

Related lesson: 3.7.

After multu of 0xFFFFFFFF and 3, what are HI and LO? Does the full product fit unsigned 32-bit storage? Show how to copy both halves to ordinary registers.

Q56. Signed multiplication fit

Related lesson: 3.7.

After signed mult of -3 and 7, give the complete signed product and hexadecimal HI/LO patterns. Does it fit signed 32 bits? Explain why testing HI != 0 alone would give a wrong decision.

Q57. Signed quotient and remainder

Related lessons: 3.8 and 3.10.

For signed -20 / 6, give quotient and remainder and verify the identity dividend = divisor x quotient + remainder. Which half of HI/LO holds each? Compare with unsigned divu of pattern 0xFFFFFFEC by 6.

Q58. Binary long division

Related lessons: 3.8-3.9.

Divide unsigned four-bit 1101 by 0011 using the bring-down-bit method. Show the partial remainder, decision bit, and quotient prefix for each of the four input bits. Verify the decimal quotient/remainder identity.

Q59. Special-result registers and division safety

Related lesson: 3.10.

Explain the problem with performing a multiplication, then a division, and only afterward using mflo to retrieve the multiplication result. Write a no-delay-slot fragment that checks a divisor for zero before a core div and copies the quotient when safe. Name another signed division edge case.

Q60. Normalize a binary fraction

Related lessons: 3.11-3.12.

Convert decimal 5.625 to binary and normalized binary scientific notation. Give its sign, actual exponent, biased binary32 exponent, and the fraction bits before padding.

Q61. Decode IEEE-754

Related lesson: 3.13.

Decode binary32 word 0xC1200000. Give S, stored exponent, actual exponent, fraction value, significand, and decimal value. State whether it is normal or a reserved-pattern case.

Q62. Encode IEEE-754

Related lesson: 3.14.

Encode -2.5 as binary32. Show the normalized magnitude, sign bit, biased exponent, 23-bit fraction field, and complete eight-digit hexadecimal word.

Q63. Reserved floating-point patterns

Related lesson: 3.15.

Classify 0x00800000, 0x00000001, 0x7F800000, and 0x7FC00000. Which normal-formula assumption fails for the second pattern? Give its exact value as a power of two.

Q64. Ties-to-even rounding

Related lesson: 3.15.

In a toy format that retains four significant binary bits including the leading 1, round 1.1011000... and 1.1001000... to nearest, ties-to-even. Explain how the retained low bit affects the two halfway cases.

Q65. Floating-point addition steps

Related lesson: 3.16.

Add 1.5 and 0.375. Show normalized binary operands, exponent alignment, significand addition, normalization, and the exact result. Is rounding needed to store this result in binary32?

Q66. Floating-point multiplication steps

Related lesson: 3.17.

Multiply -1.5 by 2.5. Show sign, actual exponent combination, significand product, normalized result, and binary32 encoding. Explain how to combine the two stored biased exponents.

Q67. FP comparison and control flow

Related lessons: 3.18-3.19.

With a in $f0 and b in $f2, translate if (a <= b) a = a*b; else a = a/b; using c.le.s and FP branches. Use no delay slots. Trace results for (a,b) = (2.0,3.0) and (4.0,2.0). Why is ordinary integer slt unsuitable for comparing these encoded values?

Q68. FP instruction fields

Related lesson: 3.20.

Encode add.d $f8,$f4,$f6. Give decimal opcode, fmt, ft, fs, fd, and funct values, then the complete word. Explain which paired registers hold the operands in the course's classic FP-register model.

Practice | Chapter 4 microarchitecture

Unless a question says otherwise, use IF, ID, EX, MEM, WB; same-cycle WB write / ID read; no architectural delay slots; separate instruction and data memories. Count through the final fifth-stage column even for a store. Forwarding is enabled only when the question explicitly states it. MUX and ALU encodings follow the lesson tables.

Q69. Follow a load through five stages

Related lessons: 4.1-4.4.

For lw $t0,12($s0), $s0 contains 200 and memory word at address 212 contains 37. Describe IF, ID, EX, MEM, WB. Which values are a register number, effective address, loaded data, and destination number?

Q70. MUX selection and write enable

Related lessons: 4.2 and 4.5.

A two-input MUX has input 0 = 88 and input 1 = 37. What is its output for each select? If RegWrite = 0, does putting output 37 on a register-file write-data wire change the destination register? Explain.

Q71. Extract fields versus register contents

Related lesson: 4.3.

Instruction 0x02324020 executes with $s1 = 20 and $s2 = 30. Identify its assembly instruction, read-register numbers, read-data values, selected destination number, ALU result, and write-back data.

Q72. Load path and control

Related lessons: 4.4-4.7.

For lw $t2,-4($s0), $s0 = 0x1000 and memory word at 0x0FFC is 123. Give the sign-extended immediate, effective address, destination number, write-back value, and controls RegDst, ALUSrc, MemtoReg, RegWrite, MemRead, MemWrite, Branch, ALUOp.

Q73. Store path and control

Related lessons: 4.4-4.7.

For sw $t2,8($s0), $s0 = 100 and $t2 = 55. Give both register-read values, selected ALU operands, effective address, store data, and the controls that permit memory writing while preventing register writing.

Q74. Branch signal versus taken signal

Related lessons: 4.5 and 4.8.

A beq is at byte address 1000, has immediate 2, and compares unequal values. Give Branch, Zero, PCSrc, candidate branch target, and selected next PC. Repeat the next-PC decision for equal values.

Q75. Two levels of ALU decoding

Related lesson: 4.6.

Give ALUOp and final four-bit ALU operation for lw, beq, and core R-format sub. Explain why the low six bits of a load's immediate must not be decoded as its funct field.

Q76. Single-cycle critical path

Related lesson: 4.9.

Stage/path delays are IF 80 ps, ID 40 ps, EX 70 ps, MEM 100 ps, WB 30 ps. Ignore other overhead. Calculate path times for lw, sw, R-format arithmetic, and beq under the same stage-use model as the slides. Choose the single-cycle clock period and the time for four instructions.

Q77. Finite pipeline speedup

Related lessons: 4.10-4.11.

Seven independent instructions run through five stages. The largest stage delay is 100 ps and pipeline-register overhead is 10 ps. A single-cycle implementation uses 320 ps. Calculate pipeline period, cycles, execution time, single-cycle time, finite-group speedup, and ideal long-run speedup.

Q78. Which instruction controls cycle 5?

Related lessons: 4.12-4.14.

Use this independent sequence with no stalls:

```
lw  $t0, 0($s0)
sub $t1, $s1, $s2
add $t2, $s3, $s4
sw  $t3, 4($s0)
add $t4, $s5, $s6
```

In cycle 5, name the instruction in each stage. Give the current ALU action, data-memory read/write enables, and WB source. Which instruction's opcode is currently being decoded by the main controller?

Q79. Preserve the destination number

Related lesson: 4.13.

An earlier load to $t0 is in WB while a younger load to $t4 is in ID. What goes wrong if the WB destination is taken from the current IF/ID.rt? Which boundary registers must preserve the earlier load's destination?

Q80. RAW without forwarding

Related lesson: 4.16.

Without forwarding, schedule add $t0,$s0,$s1 followed by sub $t1,$t0,$s2. Show producer WB and consumer ID/EX cycles, number of stalls, and total cycles through the final WB. Explain the same-cycle register-file assumption.

Q81. Fill a load-use gap safely

Related lessons: 4.17 and 4.21.

With full ALU-input forwarding, reschedule lw $t0,0($s0); add $t1,$t0,$s1; addi $s2,$s2,1 to avoid the immediate load-use stall. Assume valid nonfaulting memory and no other dependencies. Explain why placing independent work between the first two instructions is safe here.

Q82. Forward from two different boundaries

Related lessons: 4.19-4.20.

With ordinary ALU-result forwarding, analyze:

```
add $t0, $s0, $s1
sub $t2, $t0, $s2
or  $t3, $s3, $t0
```

Give the forwarding source and F1/F2 selection for each dependent operand at the consumer's EX cycle. Are stalls needed? How many cycles drain the sequence?

Q83. Choose the latest forwarded value

Related lesson: 4.20.

Three consecutive instructions update $t0: add $t0,$t0,$s0; add $t0,$t0,$s1; add $t0,$t0,$s2. When the third is in EX, both EX/MEM and MEM/WB appear to match its source. Which wins and why? Name the RegWrite and $zero guards.

Q84. An immediate load-use pair

Related lesson: 4.21.

Draw the schedule for lw $t0,0($s0); add $t1,$t0,$s1 with ordinary full ALU-input forwarding. Explain the bubble and count total cycles. Compare the stall count and total cycles without forwarding.

Q85. Repair incorrect stall control

Related lesson: 4.22.

A proposed load-use repair says "freeze PC and ID/EX; keep the load there until its result is ready." Explain the problem. State the correct front-end write enables and what enters ID/EX instead.

Q86. A field match that is not a source dependency

Related lessons: 4.16 and 4.22.

Does lw $t0,0($s0) followed by addi $t0,$s1,1 create a load-use hazard? Compare with addi $t2,$t0,1 as the second instruction. Explain why comparing both rs and rt without checking field roles can over-stall.

Q87. Branch penalties and the endpoint

Related lessons: 4.23-4.24.

A five-stage run executes 100 useful instructions. Five branch mispredictions each cost three extra cycles; there are no other delays or overlaps. Count total cycles including fill/drain, then compute time at 2 ns/cycle. Give finite-run cycles per useful instruction and the long-stream CPI estimate without fill/drain.

Q88. A one-bit predictor

Related lesson: 4.24.

A one-bit predictor initially predicts not taken and then records the last actual outcome. For outcomes T, T, T, N, list each prediction, correctness, and final predictor state. If each wrong prediction costs two cycles, how many extra cycles occur?

Practice | Chapter 5 memory hierarchy

Unless stated otherwise, addresses are byte addresses, word size is four bytes, caches/TLBs begin empty, capacities mean data bytes, and pages have valid permitted mappings. Use KiB = 1024 bytes. On an LRU hit, update recency. Miss penalty means extra time after the initial lookup. For write-policy questions, use the explicitly named allocation rule.

Q89. Identify locality

Related lessons: 5.1-5.3.

Classify repeated loop instructions, repeated access to one scalar, a sequential array scan, and one-time accesses to unrelated scattered nodes. Explain why locality makes a hierarchy useful and why a cache hit is not guaranteed merely by temporal locality.

Q90. Set count and address widths

Related lesson: 5.7.

For a 2 KiB, 2-way cache with 32-byte blocks and 32-bit byte addresses, calculate lines, sets, offset bits, index bits, and tag bits. State which quantity associativity divides.

Q91. Map a specific byte address

Related lessons: 5.5-5.8.

For a 1 KiB direct-mapped cache with 16-byte blocks, map byte address 0x1234. Give decimal memory-block number, index, tag, byte offset, and the starting byte address of its block.

Q92. Direct-mapped state trace

Related lesson: 5.6.

A four-line direct-mapped cache has one memory block per line. Inputs are already block numbers: 0, 4, 0, 1, 4, 1. Show index, tag, hit/miss, and changed line for each request. Give final occupied lines and hit rate.

Q93. Valid is part of the hit test

Related lesson: 5.6.

A requested block's index points to a line whose stored tag matches but valid = 0. Is this a hit? What if valid = 1 but the tag differs? Write the complete hit condition.

Q94. Include metadata storage

Related lesson: 5.7.

A 2 KiB, 2-way, write-back cache has 32-byte blocks and 32-bit addresses. Count data bits plus one valid bit, one dirty bit, and all tag bits per line. Ignore replacement and ECC state. Give total bits and bytes.

Q95. Placement choices

Related lesson: 5.15.

An eight-line cache holds block numbers 6 and 14. Compare their allowed locations in direct-mapped, 2-way, and fully associative organizations. Can both coexist in the 2-way case? How many tag comparisons are needed per selected set?

Q96. LRU updates on hits

Related lessons: 5.16-5.17.

Two ways in a single set start empty. Trace A, B, A, C, B using exact LRU. Show outcomes, victims, and oldest-to-newest order after each access. Compare the victim on request C with FIFO arrival order.

Q97. Dirty write-back behavior

Related lesson: 5.11.

Lower memory A starts at 10. A read miss brings it into a write-back cache. A store hit changes A to 20, followed by a load hit and then eviction of A. State cached value, lower-memory value, valid/dirty state where applicable, load result, and whether eviction needs a write-back.

Q98. Allocation is separate from write propagation

Related lessons: 5.10-5.11.

Compare a store miss followed by a load of the same address under (a) write-through/no-write-allocate and (b) write-back/write-allocate. Assume no eviction between them and immediate lower-level completion for this conceptual trace. Which access fills the cache and which load hits?

Q99. Block-transfer bandwidth

Related lesson: 5.12.

A six-word block is 24 bytes. Address transfer costs one bus cycle, each DRAM access costs 12 cycles, and each returned word costs one cycle. Compute modeled fill time and B/cycle for serial one-word-wide memory, six-word-wide memory/transfer, and six interleaved banks with serial word returns. Assume the overlap model from the slides.

Q100. Miss contributions to CPI

Related lesson: 5.13.

Base CPI = 1.5. I-cache miss rate = 1%; D-cache miss rate = 5%; 30% of instructions access data; both miss penalties are 40 CPU cycles. Calculate each extra-CPI contribution, total CPI, and time for 1000 instructions at 2 GHz. Assume additive blocking penalties.

Q101. AMAT and miss-time conventions

Related lesson: 5.14.

Hit time is 2 ns and miss rate is 10%. Calculate AMAT when 30 ns means (a) additional miss penalty, and (b) total miss service time including lookup. Explain why the answers differ and why neither is automatically complete-instruction CPI.

Q102. Local and global L2 miss rates

Related lesson: 5.18.

L1 hit time = 1 ns; L1 miss rate = 8%; extra L2 access time = 5 ns; local L2 miss rate = 25%; additional main-memory time = 100 ns. Calculate global miss rate and AMAT. State both miss-rate denominators.

Q103. More GHz does not halve a fixed memory wait

Related lesson: 5.19.

Base CPI = 1 on both processors. There are 0.02 misses/instruction, each costing an additional fixed 50 ns, without overlap. Compare a 2 GHz CPU and a 4 GHz CPU: miss penalty in cycles, actual CPI, seconds per instruction, and overall speedup.

Q104. Matrix layout and tile footprint

Related lesson: 5.20.

Double matrices use column-major index i + j*n. Which loop index gives adjacent elements when incremented by 1? Calculate the data footprint of three 16-by-16 tiles and of three complete 64-by-64 matrices. Why must blocked loops handle partial tiles and preserve sums over all inner tiles?

Q105. Virtual-to-physical translation

Related lessons: 5.21-5.22.

With 4 KiB pages, translate VA 0x3456 when VPN 3 maps to frame 9. Give VPN, offset in hex and decimal, and PA. For 32-bit VA and 30-bit PA, give VPN/physical-frame widths and offset width.

Q106. Change the page size

Related lesson: 5.21.

Use VA 0x3456 again, now with 16 KiB pages and VPN 0 mapped to frame 2. Calculate the new VPN, offset, and PA. Explain why the earlier 4 KiB translation cannot simply be reused.

Q107. A flat page-table size

Related lesson: 5.22.

For 32-bit virtual addresses, 8 KiB pages, and eight-byte PTEs, calculate offset width, VPN width, number of entries, and size of a complete flat table. With 36-bit physical addresses, how many bits identify a physical frame?

Q108. Three different misses

Related lesson: 5.23.

Explain (a) TLB miss with resident page, (b) page fault for a valid nonresident disk-backed page, and (c) data-cache miss after a successful translation. Can a TLB hit and a data-cache miss happen on the same access? Explain.

Q109. A small TLB LRU trace

Related lessons: 5.23 and 5.25.

A two-entry TLB initially holds VPN 3 -> frame 9 and VPN 1 -> frame 5, oldest to newest. The page table also says VPN 2 -> frame 7 is resident. Pages are 4 KiB. Requests are VA 0x1234, 0x3456, 0x2345, 0x1456. Trace TLB hits/misses, evictions, resulting physical addresses, and final LRU order. Are any page faults needed?

Q110. A dirty page victim

Related lesson: 5.24.

A valid page fault needs VPN 7. The OS selects physical frame 5, currently holding a dirty page for VPN 3. Explain the order of preservation, read, mapping/translation updates, and instruction restart. What changes if the old page is clean with a valid backing copy?

Q111. Classify the three Cs

Related lesson: 5.26.

Classify misses for (a) an empty two-line fully associative cache seeing A, B, C, A, and (b) an empty four-line direct-mapped cache seeing block numbers 0, 4, 0. For each request say compulsory, capacity, conflict, or hit, with a reason.

Q112. Combine translation and a data-cache trace

Related lessons: 5.6-5.8 and 5.21-5.25.

Pages are 4 KiB. VPN 0 maps to frame 2; VPN 1 maps to frame 3; both are resident and permitted. A two-entry fully associative TLB starts empty. A physically addressed, direct-mapped data cache starts empty with four lines and 16-byte blocks. Virtual requests are 0x0034, 0x0038, 0x1034, 0x0034. For each, give VPN, TLB outcome, PA, memory-block number, cache index/tag/offset, and data-cache outcome. Give final cache state, hit rates, and page-fault count.